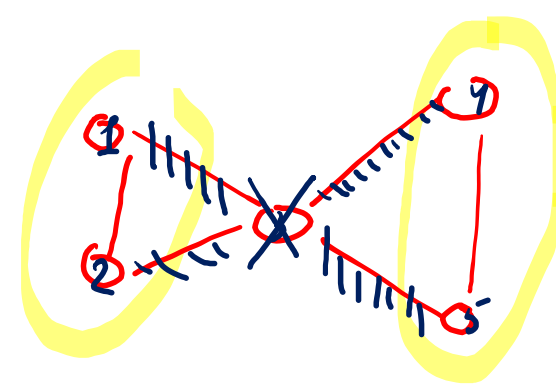
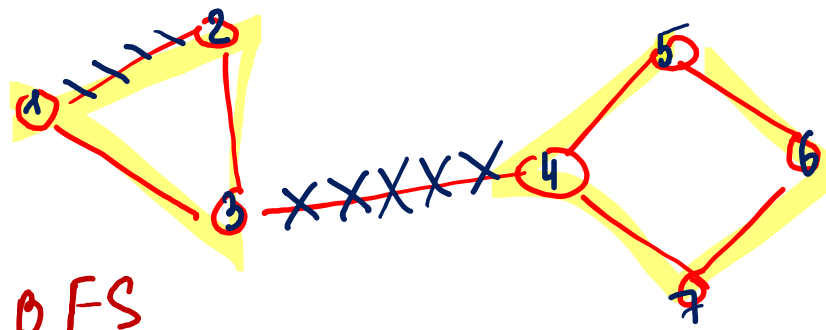


Графы. Мосты. Точки сочленения. Компоненты сильной связности.

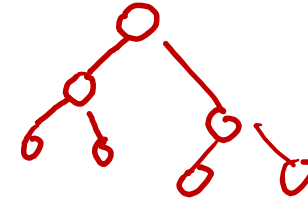
Горденко М.К.

Определения

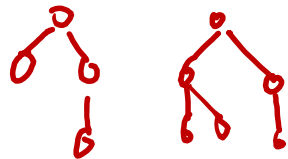
DFS ~ BFS

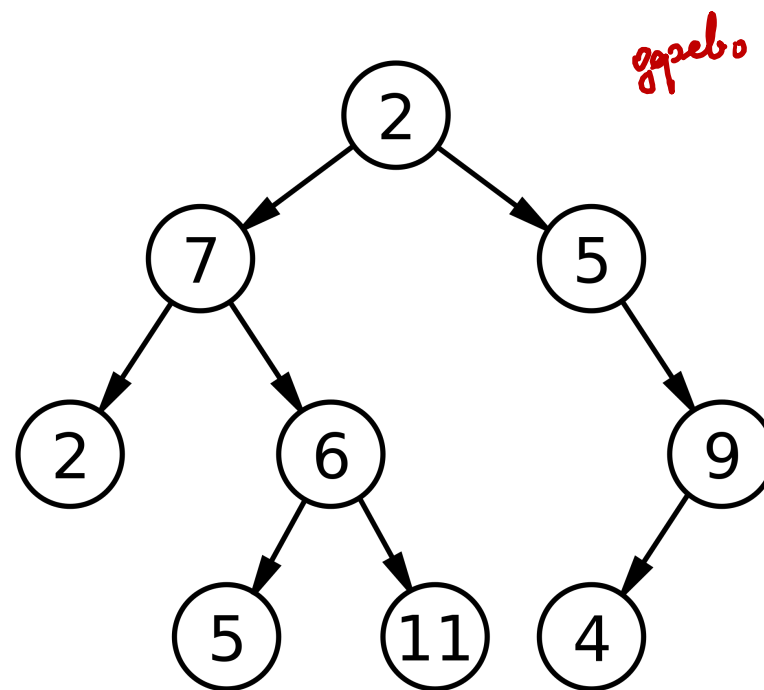
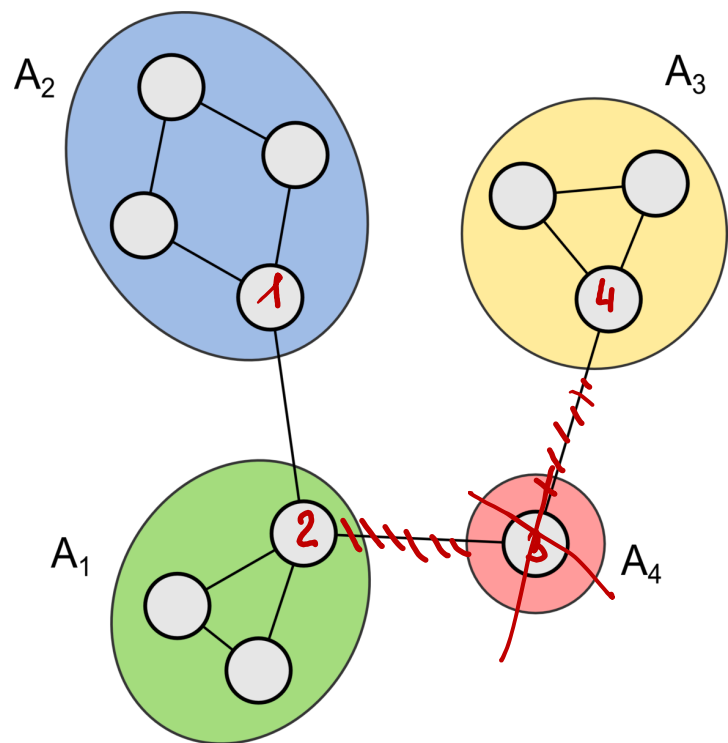


- Мост (разрезающие рёбра, разрезающие дуги или перешейки) — ребро в теории графов, удаление которого увеличивает число компонент связности.
- Дерево — это связный ациклический граф.
- Точкой сочленения (разделяющая вершина или шарнир) в теории графов называется вершина графа, при удалении которой количество компонент связности возрастает.
- Лес — упорядоченное множество упорядоченных деревьев.



n вершин $\rightarrow n-1$ рёбра

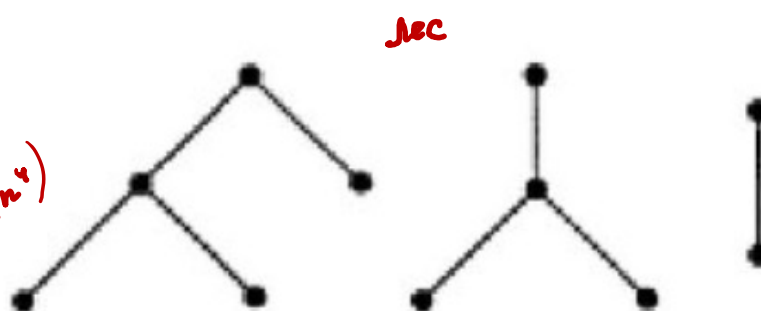




до n^2 перебр

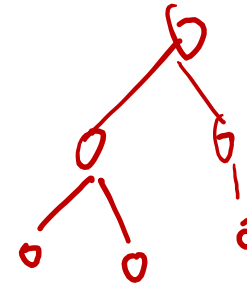
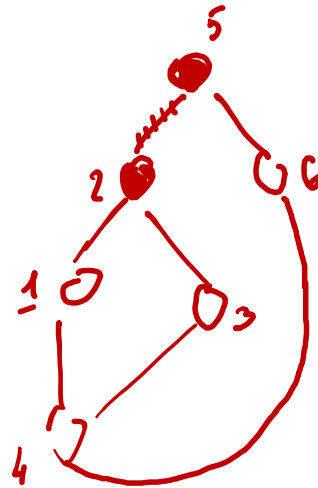
DFS-conv $\rightarrow O(v + n^2)$

$O(n^4)$

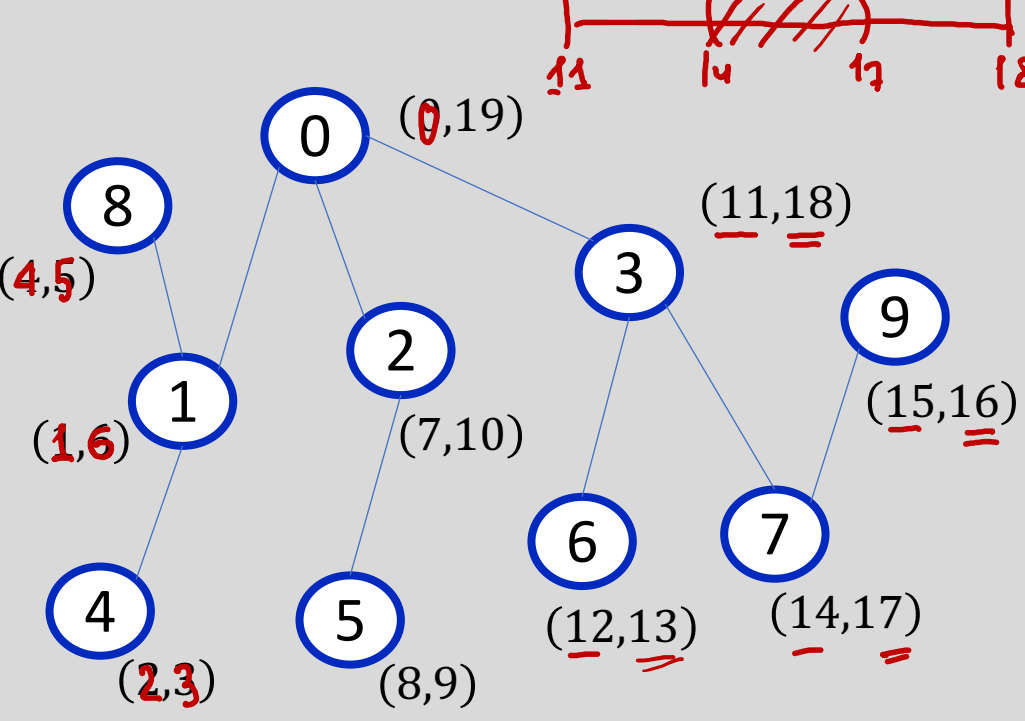


Важно понять!

- Граф с N вершинами может содержать не более $N - 1$ мостов, поскольку добавление ещё одного ребра неминуемо приведёт к появлению цикла. Графы, имеющие в точности $N - 1$ мостов, известны как деревья, а графы, в которых любое ребро является мостом — это леса.



Время начала и конца обработки вершины

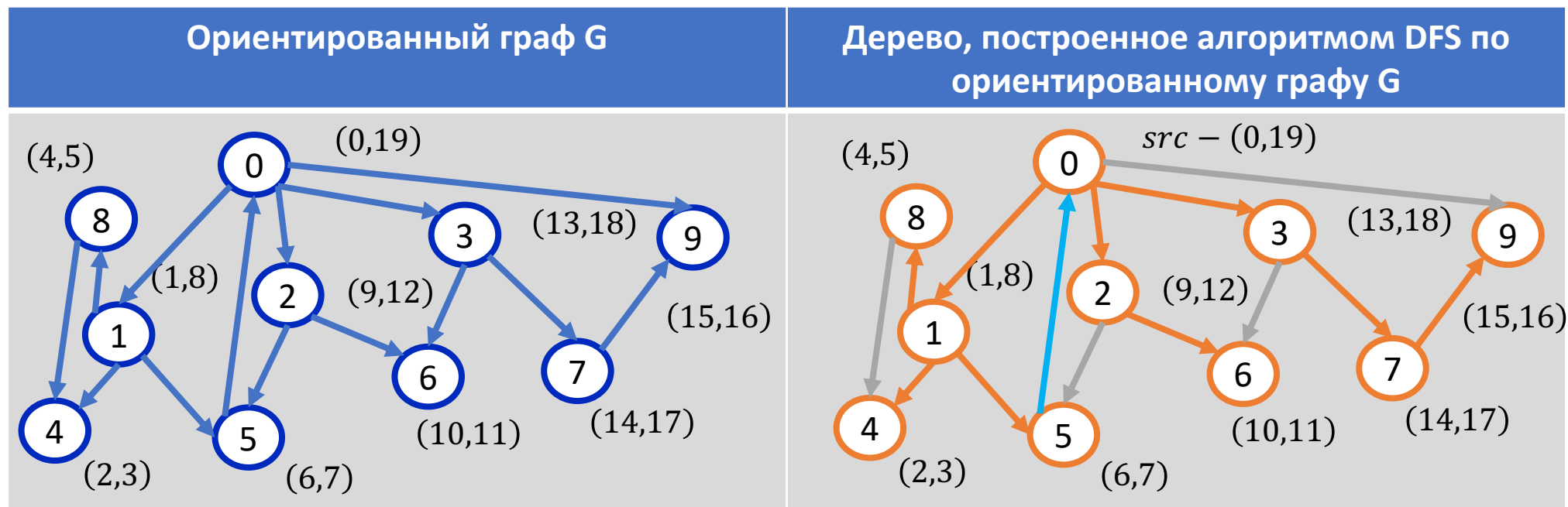
Неориентированный граф G	Функция обхода графа в глубину (DFS) с фиксацией времени начала и конца обработки вершин
 Handwritten red notes above the graph: $u_{in} < \sim \delta_{in} < \delta_{out} < u_{out}$ Time intervals for nodes (v, (start, end)): 0: (0, 19) 8: (4, 5) 1: (1, 6) 2: (7, 10) 3: (11, 18) 9: (15, 16) 4: (2, 3) 5: (8, 9) 6: (12, 13) 7: (14, 17)	<pre>1. void dfs(int v) { 2. used[v] = true; 3. wtime_in[v] = time++; 4. for (auto &vert : g[v]) { 5. if (!used[vert]) { 6. dfs(vert); 7. } 8. } 9. wtime_out[v] = time++; 10. }</pre>

Как с помощью этих меток определить пару «предок-потомок»?

Вспомним DFS с time_in/time_out

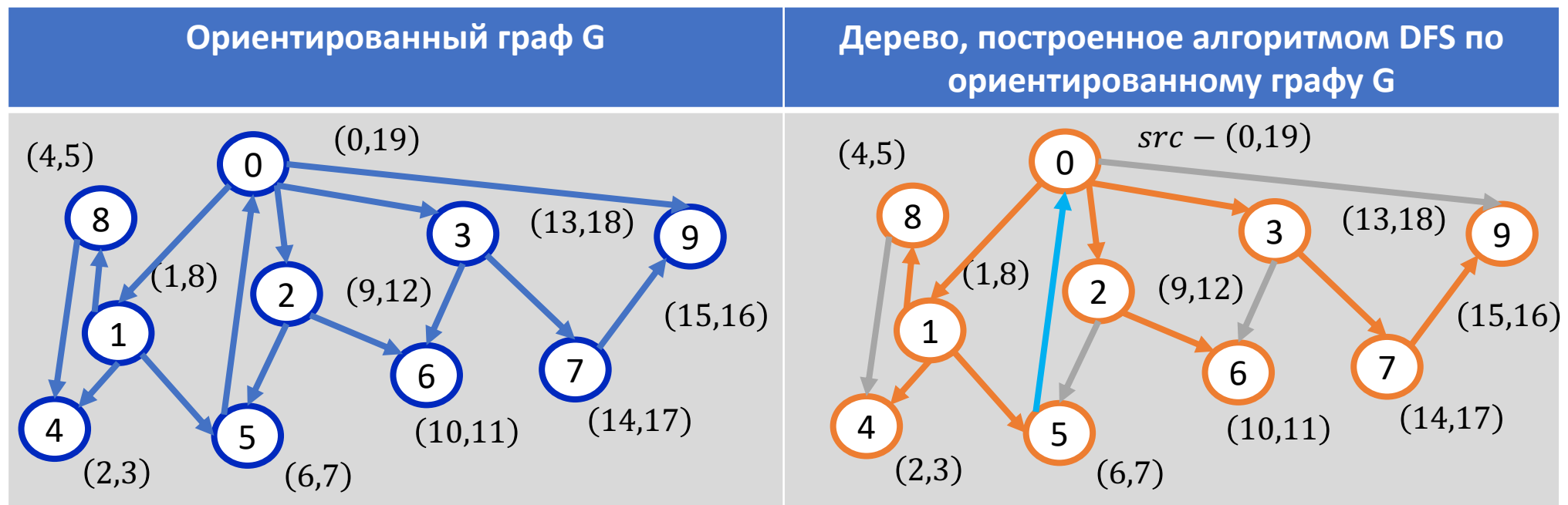
- Древесные ребра. Оранжевым цветом в дереве справа помечены те ребра (u,v) , которые могут быть изображены в соответствии с временем начала и конца обработки вершин. Это так называемые древесные ребра. Для их вершин выполняется:

→ $time_in[u] < time_in[v] < time_out[v] < time_out[u]$.



Вспомним DFS с time_in/time_out

- *Прямые ребра.* Ведут от вершины к потомку, не являющемуся ребенком. Для них выполняется то же условие, что и для древесных ребер (u, v) .
- На рисунке справа ребро $(0, 9)$ – прямое.



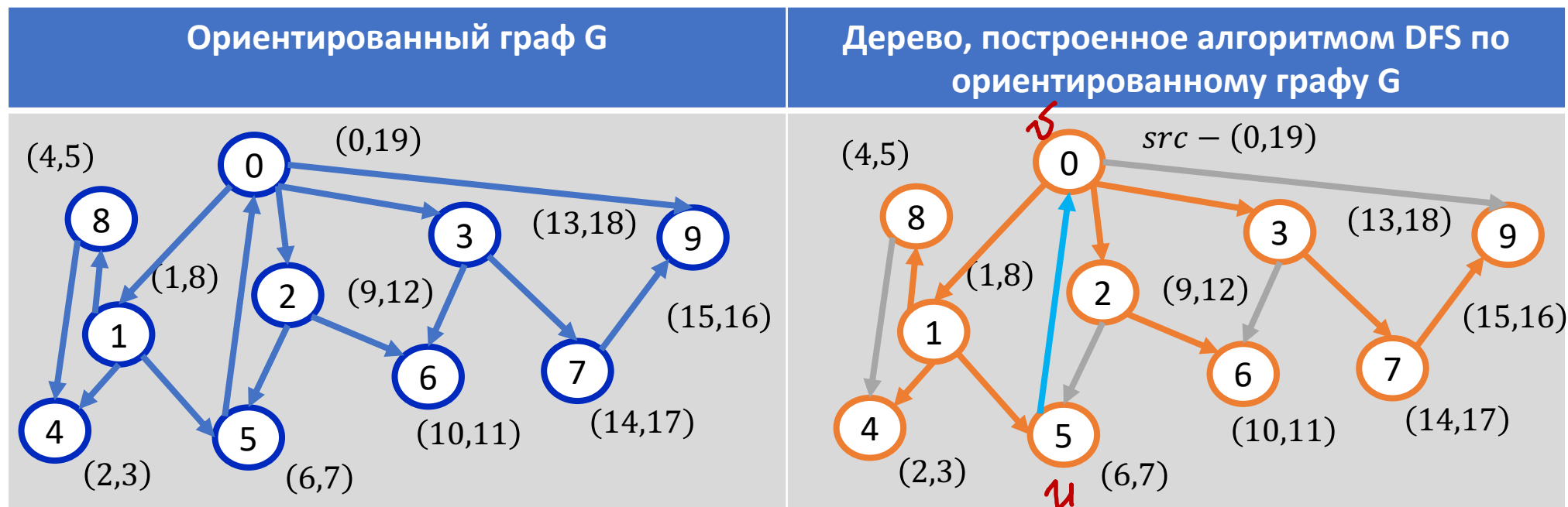
Вспомним DFS с time_in/time_out

- Обратные ребра. Ведут от вершины к ее предку. То есть, для обратного ребра (u,v) : выполняется условие

$$\text{time_in}[\underline{v}] < \text{time_in}[\underline{u}] < \text{time_out}[\underline{u}] < \text{time_out}[\underline{v}].$$

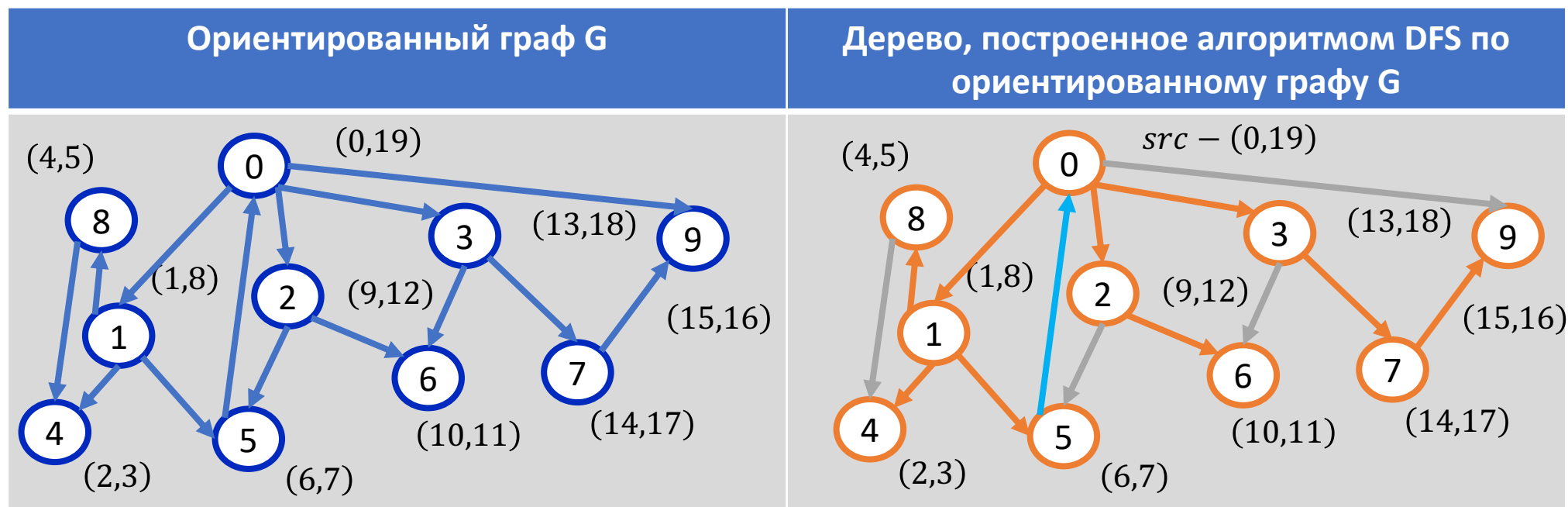
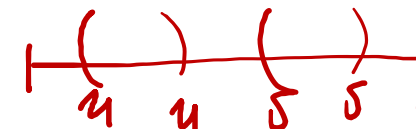
по времени *предок*

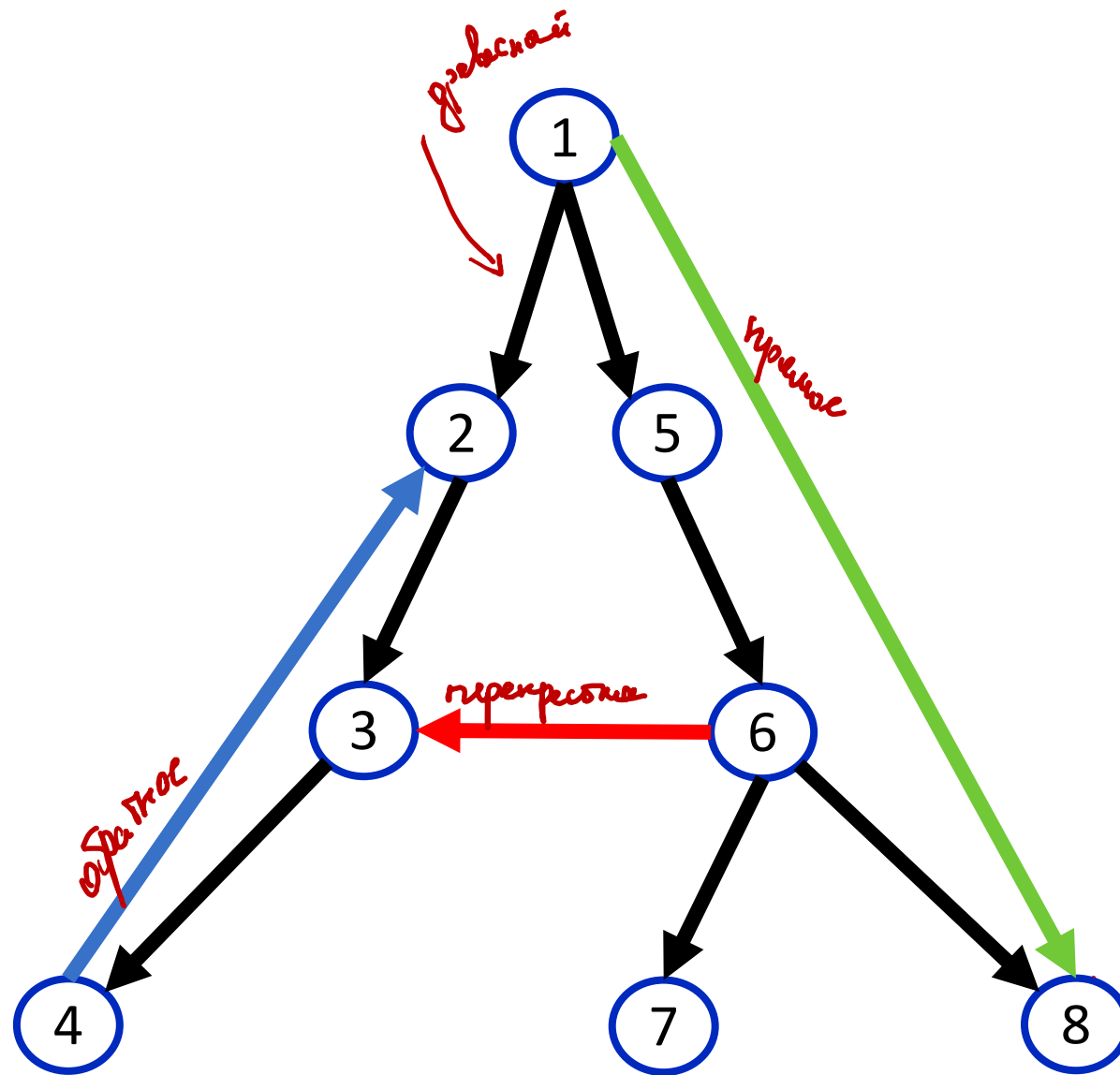
- Синее ребро на рисунке – обратное.



Вспомним DFS с time_in/time_out

- Перекрестные ребра. Для таких ребер выполняется условие
 $time_in[v] < time_out[v] < time_in[u] < time_out[u]$.
- На рисунке справа можно найти перекрестные ребра, например (8,4).



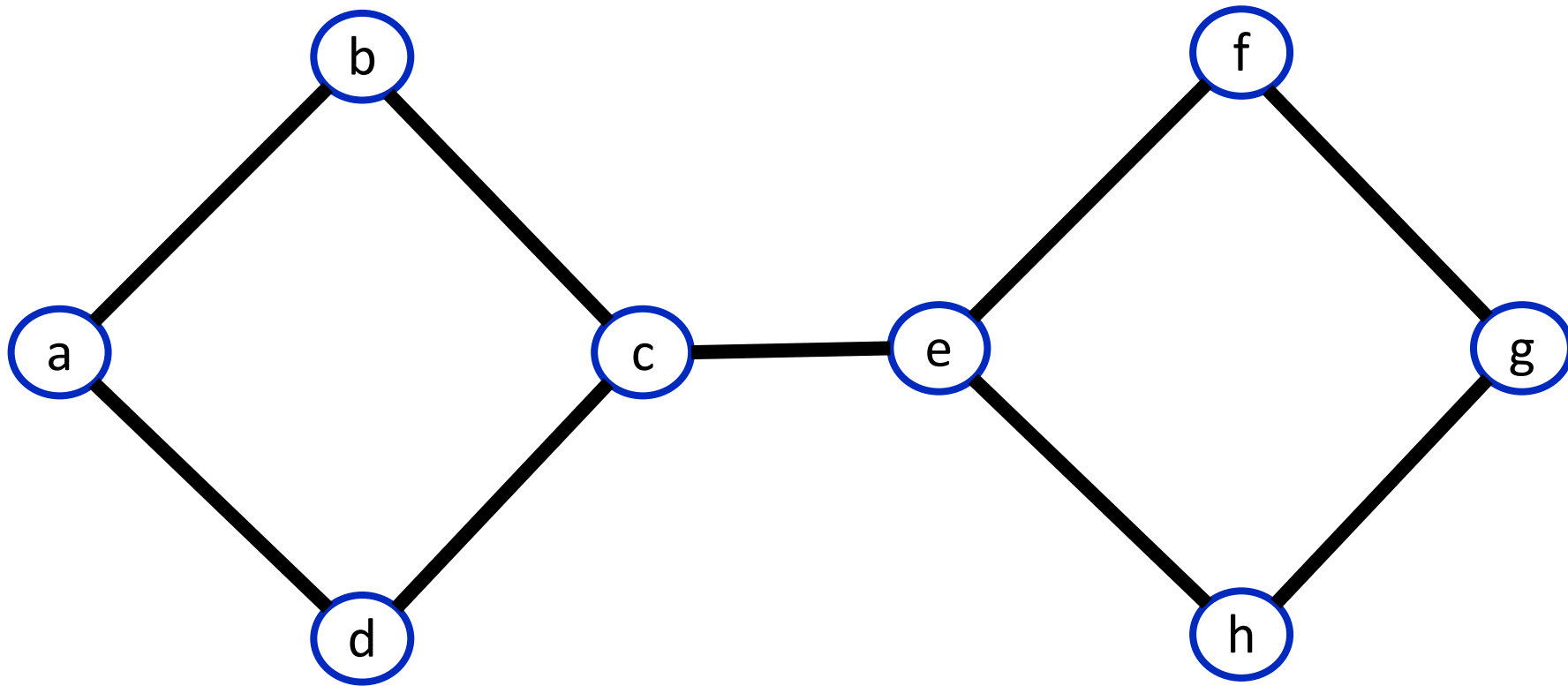


Поиск мостов на основе DFS – $O(V+E)$

- Пусть мы находимся в обходе в глубину, просматривая сейчас все рёбра из вершины v . Тогда, если текущее ребро (v, to) таково, что из вершины to и из любого её потомка в дереве обхода в глубину нет обратного ребра в вершину v или какого-либо её предка, то это ребро является мостом. В противном случае оно мостом не является.
- В самом деле, мы этим условием проверяем, нет ли другого пути из v в to , кроме как спуск по ребру (v, to) дерева обхода в глубину.

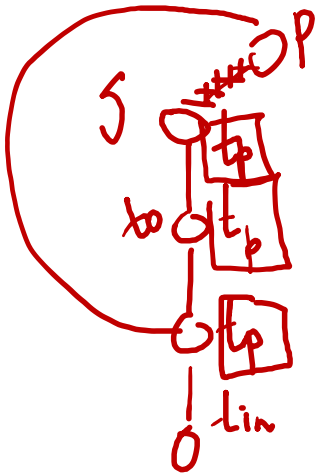


Поиск мостов на основе DFS – $O(V+E)$



Поиск мостов на основе DFS – $O(V+E)$

- Итак, пусть $tin[v]$ — это время захода поиска в глубину в вершину v . Теперь введём массив $fup[v]$, который и позволит нам отвечать на вышеописанные запросы. Время $fup[v]$ равно минимуму из времени захода в саму вершину $tin[v]$, времён захода в каждую вершину p , являющуюся концом некоторого обратного ребра (v, p) , а также из всех значений $fup[to]$ для каждой вершины to , являющейся непосредственным потомком v в дереве поиска:



$$\underbrace{fup[v]} = \min \begin{cases} \checkmark tin[v] \\ tin[p], (v, p) - \text{обратное ребро} \\ fup[to], (v, to) - \text{древесное ребро} \end{cases}$$

Поиск мостов на основе DFS – $O(V+E)$

- Тогда, из вершины v или её потомка есть обратное ребро в её предка тогда и только тогда, когда найдётся такой потомок to , что $fup[to] \leq tin[v]$.
- Если $fup[to] = tin[v]$, то это означает, что найдётся обратное ребро, приходящее точно в v ; если же $fup[to] < tin[v]$, то это означает наличие обратного ребра в какого-либо предка вершины v .
- Таким образом, если для текущего ребра (v, to) (принадлежащего дереву поиска) выполняется $fup[to] > tin[v]$, то это ребро является мостом; в противном случае оно мостом не является.

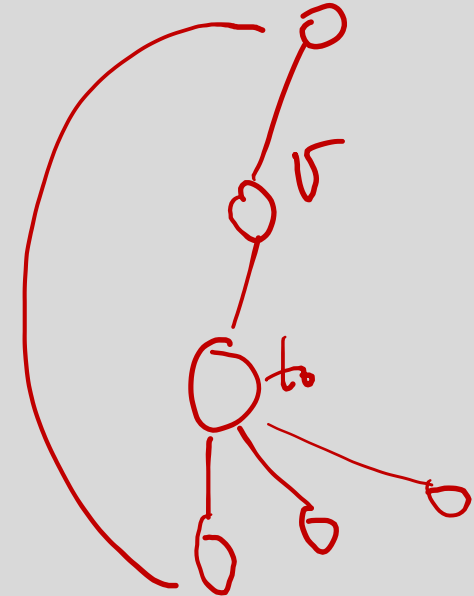
Поиск мостов на основе DFS – $O(V+E)$. Алгоритм

- Если говорить о самой реализации, то здесь нам нужно уметь различать три случая: когда мы идём по ребру дерева поиска в глубину, когда идём по обратному ребру, и когда пытаемся пойти по ребру дерева в обратную сторону. Это, соответственно, случаи:
- $used[to] = false$ — критерий ребра дерева поиска;
- $used[to] = true \ \&\& \ to \neq \ parent$ — критерий обратного ребра;
- $to = parent$ — критерий прохода по ребру дерева поиска в обратную сторону.

```
1.  const int kMaxNum = ...;
2.  vector<vector<int>> g(kMaxNum);
3.  bool used[kMaxNum];
4.  int timer = 0;
5.  int tin[kMaxNum];
6.  int fup[kMaxNum];

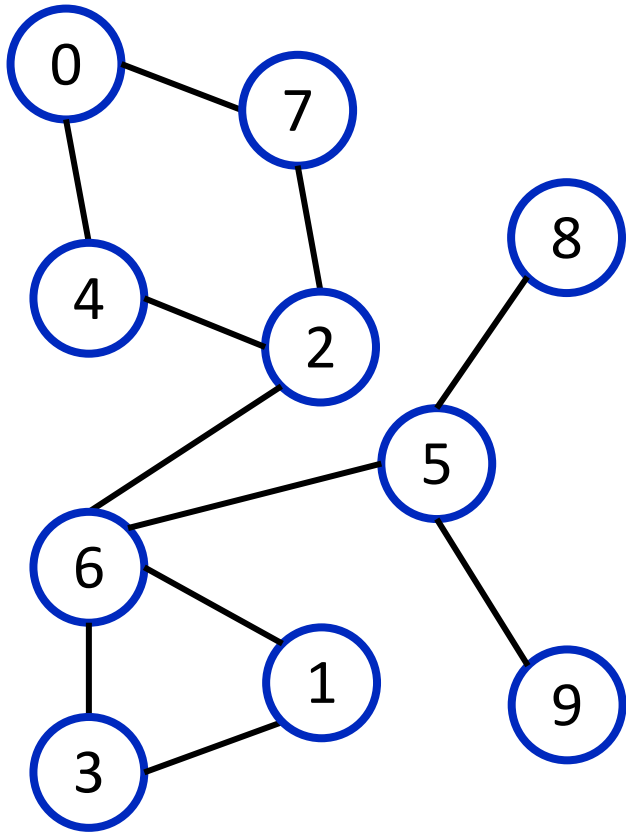
7.  void dfs(int v, int p = -1) {
8.      used[v] = true;
9.      tin[v] = (fup[v] = timer++);
10.     for (auto &to : g[v]) {
11.         if (to == p)
12.             continue;
13.         if (used[to]) обратное
14.             fup[v] = min(fup[v], tin[to]);
15.         else {
16.             dfs(to, v);
17.             fup[v] = min(fup[v], fup[to]);
18.             if (fup[to] > tin[v])
19.                 isBridge(v, to);
20.         }
21.     }
22. }

23. void findBridges() {
24.     for (int i = 0; i < kMaxNum; ++i)
25.         used[i] = false;
26.     for (int i = 0; i < kMaxNum; ++i)
27.         if (!used[i])
28.             dfs(i);
29. }
```



Поиск мостов на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

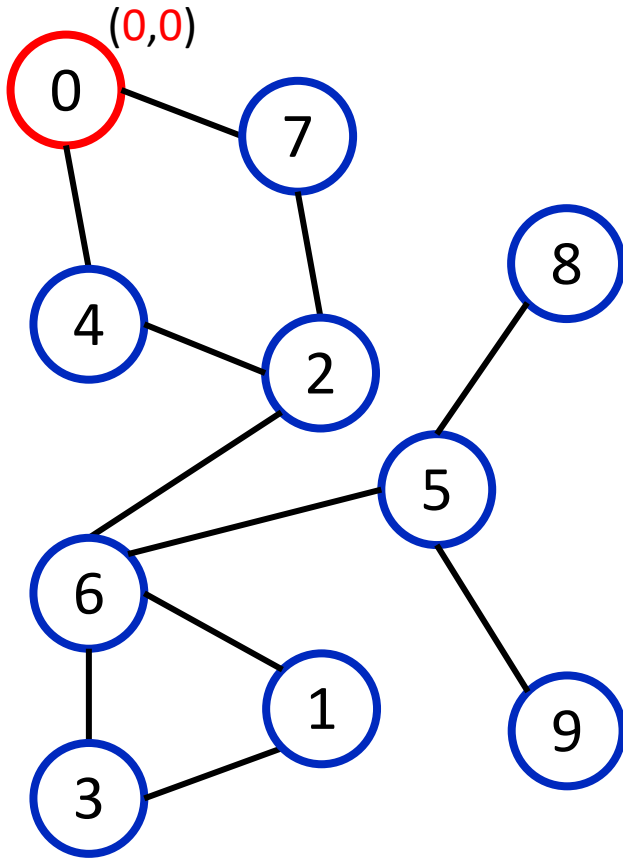


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {
2.     used[v] = true;
3.     tin[v] = (fup[v] = timer++);
4.     for (auto &to : g[v]) {
5.         if (to == p)
6.             continue;
7.         if (used[to])
8.             fup[v] = min(fup[v], tin[to]);
9.         else {
10.            dfs(to, v);
11.            fup[v] = min(fup[v], fup[to]);
12.            if (fup[to] > tin[v])
13.                isBridge(v, to);
14.        }
15.    }
16.}
```


Поиск мостов на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

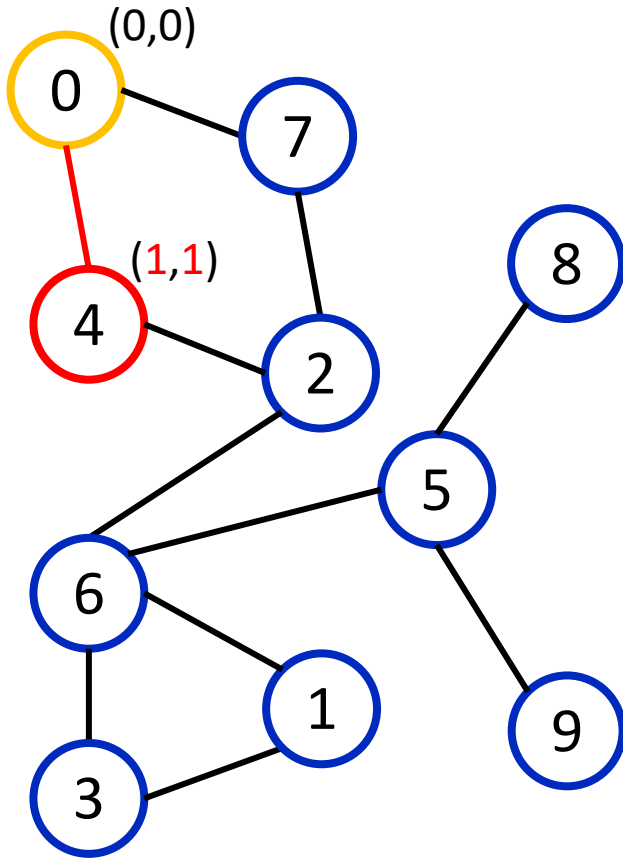


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {
2.     used[v] = true;
3.     tin[v] = (fup[v] = timer++);
4.     for (auto &to : g[v]) {
5.         if (to == p)
6.             continue;
7.         if (used[to])
8.             fup[v] = min(fup[v], tin[to]);
9.         else {
10.            dfs(to, v);
11.            fup[v] = min(fup[v], fup[to]);
12.            if (fup[to] > tin[v])
13.                isBridge(v, to);
14.        }
15.    }
16.}
```

Поиск мостов на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

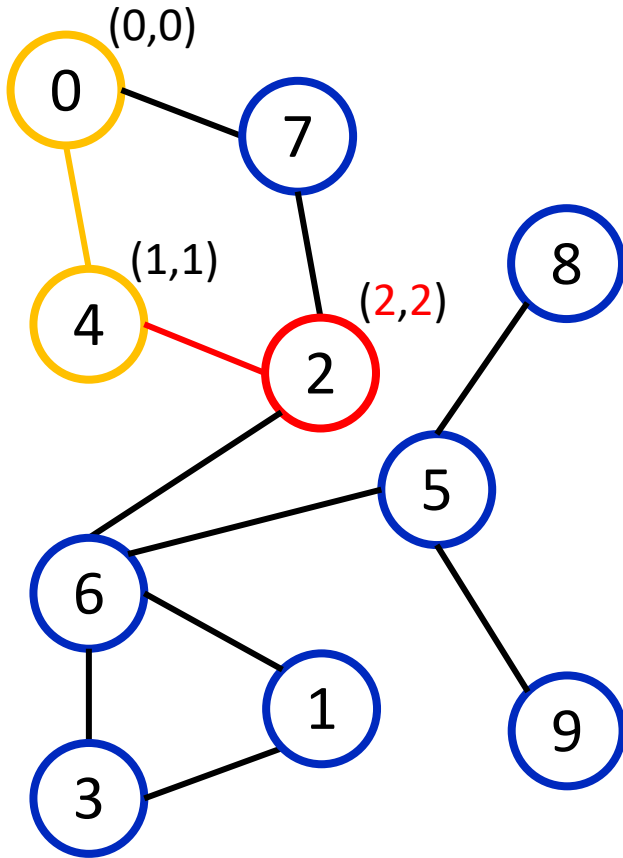


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {  
2.     used[v] = true;  
3.     tin[v] = (fup[v] = timer++);  
4.     for (auto &to : g[v]) {  
5.         if (to == p)  
6.             continue;  
7.         if (used[to])  
8.             fup[v] = min(fup[v], tin[to]);  
9.         else {  
10.            dfs(to, v);  
11.            fup[v] = min(fup[v], fup[to]);  
12.            if (fup[to] > tin[v])  
13.                isBridge(v, to);  
14.        }  
15.    }  
16.}
```

Поиск мостов на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

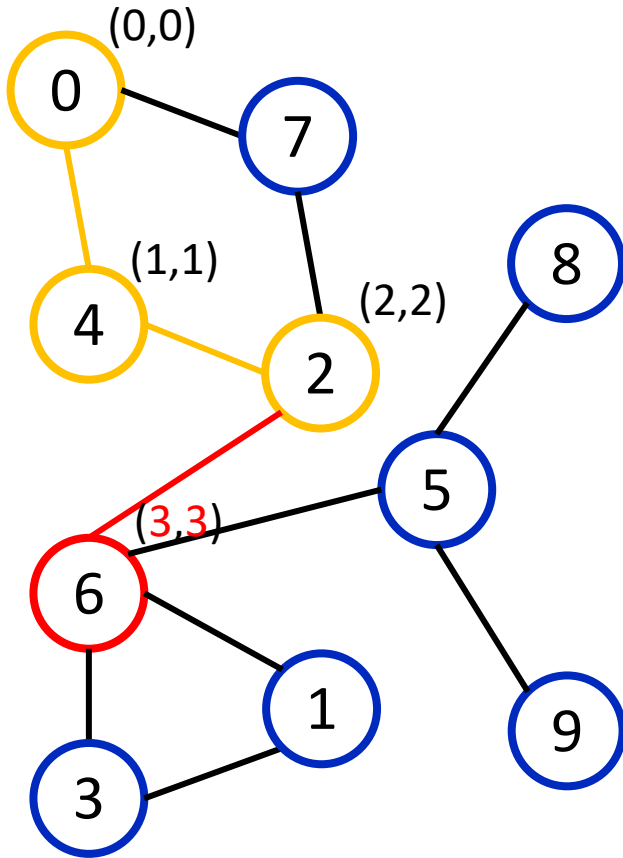


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {  
2.     used[v] = true;  
3.     tin[v] = (fup[v] = timer++);  
4.     for (auto &to : g[v]) {  
5.         if (to == p)  
6.             continue;  
7.         if (used[to])  
8.             fup[v] = min(fup[v], tin[to]);  
9.         else {  
10.            dfs(to, v);  
11.            fup[v] = min(fup[v], fup[to]);  
12.            if (fup[to] > tin[v])  
13.                isBridge(v, to);  
14.        }  
15.    }  
16.}
```

Поиск мостов на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

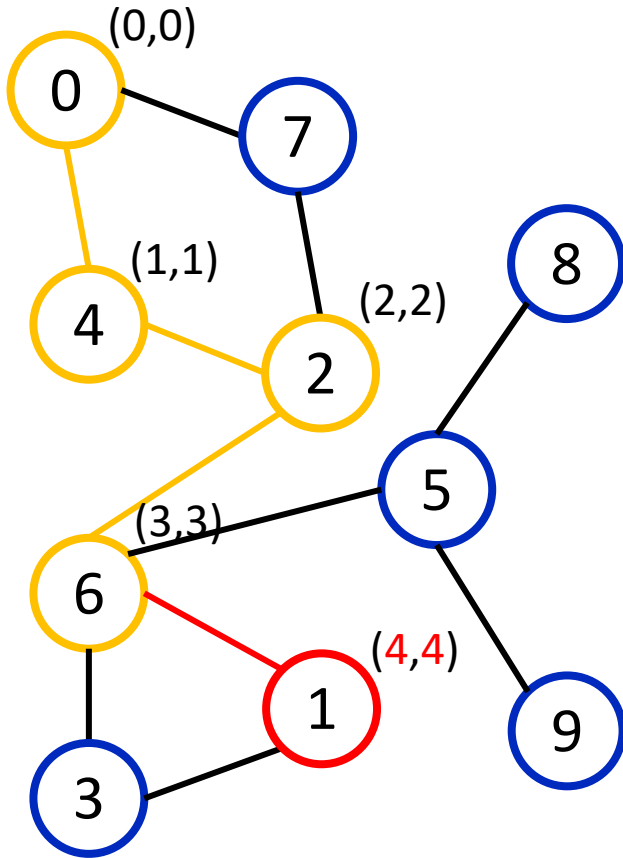


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {
2.     used[v] = true;
3.     tin[v] = (fup[v] = timer++);
4.     for (auto &to : g[v]) {
5.         if (to == p)
6.             continue;
7.         if (used[to])
8.             fup[v] = min(fup[v], tin[to]);
9.         else {
10.            dfs(to, v);
11.            fup[v] = min(fup[v], fup[to]);
12.            if (fup[to] > tin[v])
13.                isBridge(v, to);
14.        }
15.    }
16.}
```

Поиск мостов на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

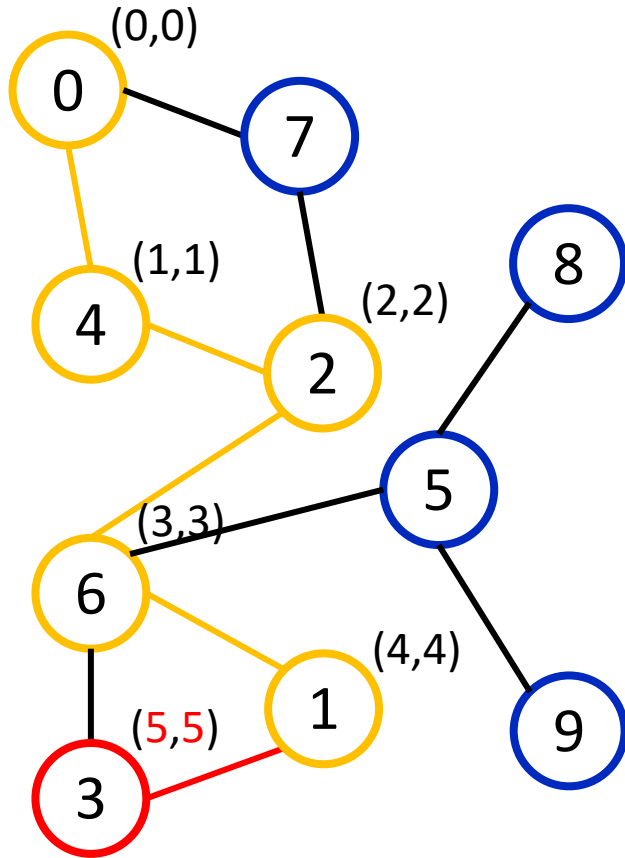


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {
2.     used[v] = true;
3.     tin[v] = (fup[v] = timer++);
4.     for (auto &to : g[v]) {
5.         if (to == p)
6.             continue;
7.         if (used[to])
8.             fup[v] = min(fup[v], tin[to]);
9.         else {
10.            dfs(to, v);
11.            fup[v] = min(fup[v], fup[to]);
12.            if (fup[to] > tin[v])
13.                isBridge(v, to);
14.        }
15.    }
16.}
```

Поиск мостов на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

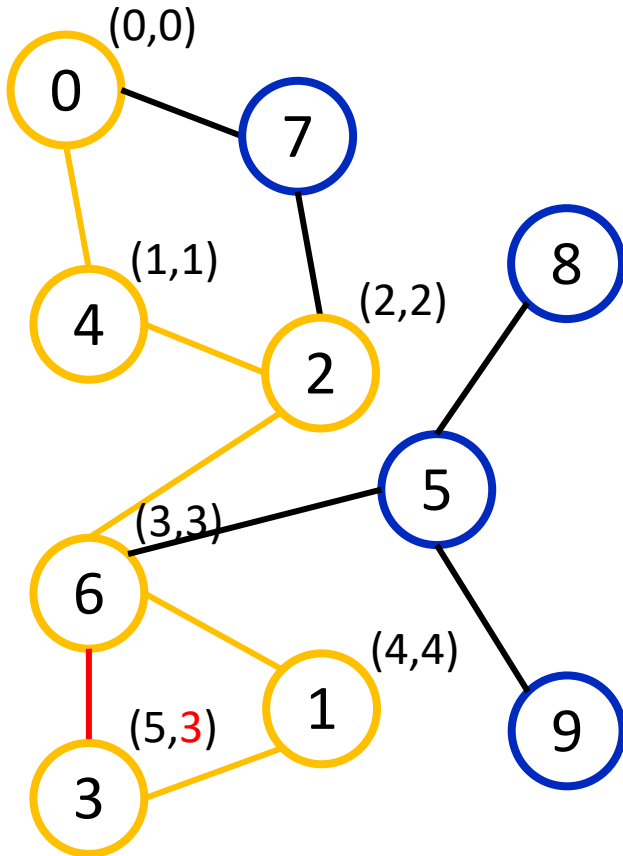


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {  
2.     used[v] = true;  
3.     tin[v] = (fup[v] = timer++);  
4.     for (auto &to : g[v]) {  
5.         if (to == p)  
6.             continue;  
7.         if (used[to])  
8.             → fup[v] = min(fup[v], tin[to]);  
9.         else {  
10.            dfs(to, v);  
11.            fup[v] = min(fup[v], fup[to]);  
12.            if (fup[to] > tin[v])  
13.                isBridge(v, to);  
14.        }  
15.    }  
16.}
```

Поиск мостов на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

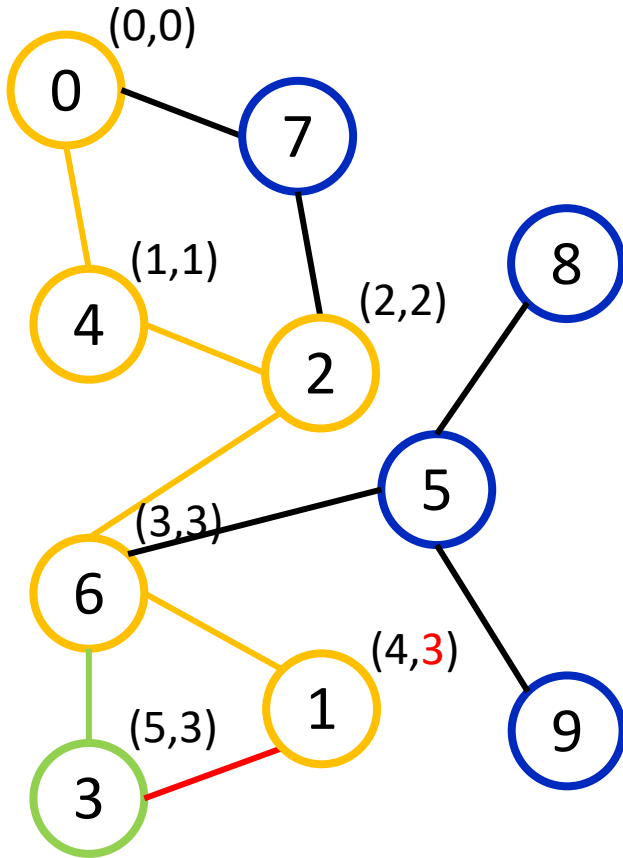


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {
2.     used[v] = true;
3.     tin[v] = (fup[v] = timer++);
4.     for (auto &to : g[v]) {
5.         if (to == p)
6.             continue;
7.         if (used[to])
8.             fup[v] = min(fup[v], tin[to]);
9.         else {
10.            dfs(to, v);
11.            fup[v] = min(fup[v], fup[to]);
12.            if (fup[to] > tin[v])
13.                isBridge(v, to);
14.        }
15.    }
16.}
```

Поиск мостов на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

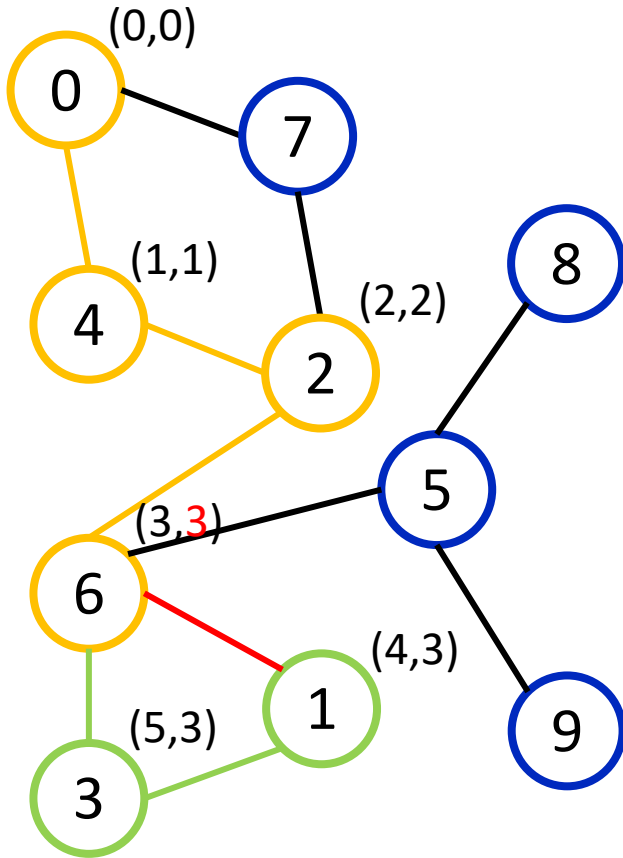


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {
2.     used[v] = true;
3.     tin[v] = (fup[v] = timer++);
4.     for (auto &to : g[v]) {
5.         if (to == p)
6.             continue;
7.         if (used[to])
8.             fup[v] = min(fup[v], tin[to]);
9.         else {
10.            dfs(to, v);
11.            fup[v] = min(fup[v], fup[to]);
12.            if (fup[to] > tin[v])
13.                isBridge(v, to);
14.        }
15.    }
16.}
```


Поиск мостов на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

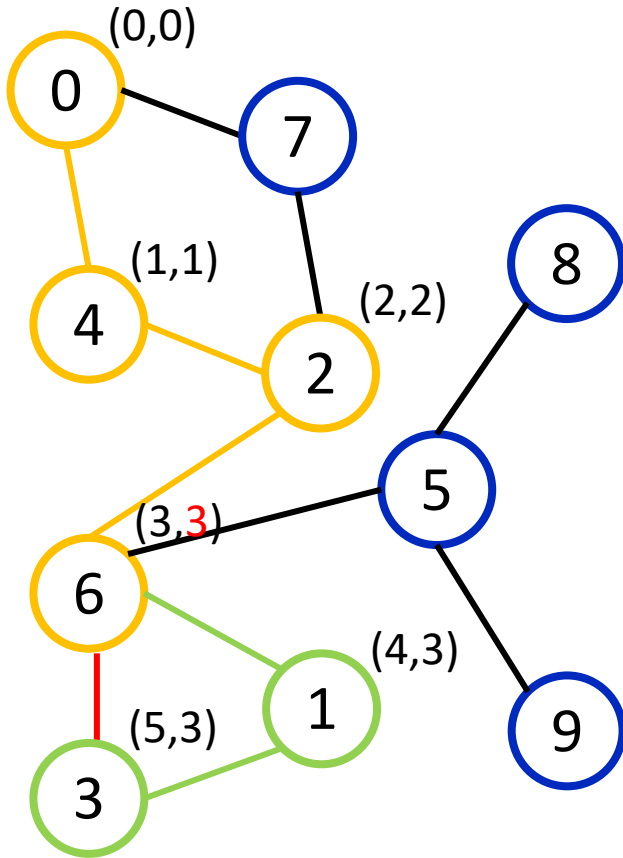


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {  
2.     used[v] = true;  
3.     tin[v] = (fup[v] = timer++);  
4.     for (auto &to : g[v]) {  
5.         if (to == p)  
6.             continue;  
7.         if (used[to])  
8.             fup[v] = min(fup[v], tin[to]);  
9.         else {  
10.            dfs(to, v);  
11.            fup[v] = min(fup[v], fup[to]);  
12.            if (fup[to] > tin[v])  
13.                isBridge(v, to);  
14.        }  
15.    }  
16.}
```

Поиск мостов на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

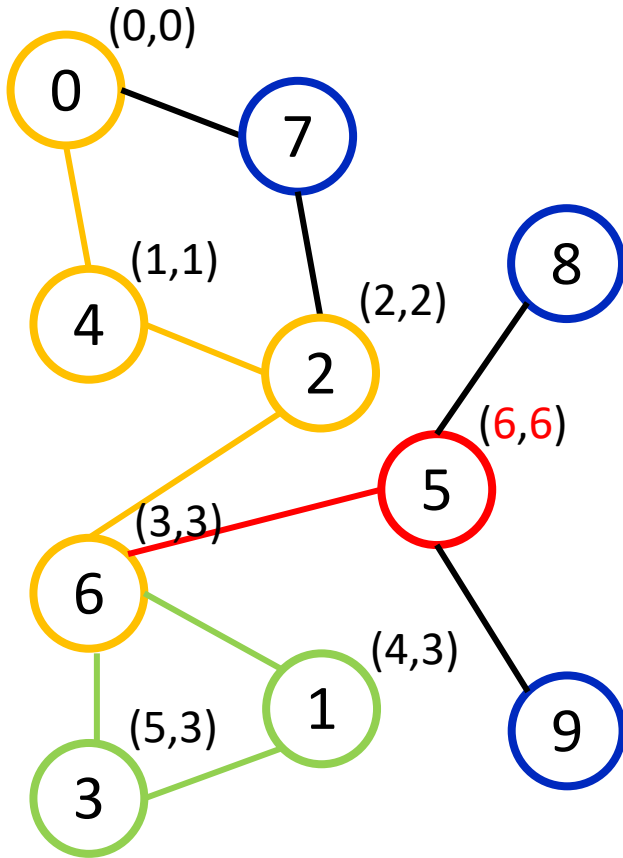


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {
2.     used[v] = true;
3.     tin[v] = (fup[v] = timer++);
4.     for (auto &to : g[v]) {
5.         if (to == p)
6.             continue;
7.         if (used[to])
8.             fup[v] = min(fup[v], tin[to]);
9.         else {
10.            dfs(to, v);
11.            fup[v] = min(fup[v], fup[to]);
12.            if (fup[to] > tin[v])
13.                isBridge(v, to);
14.        }
15.    }
16.}
```

Поиск мостов на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

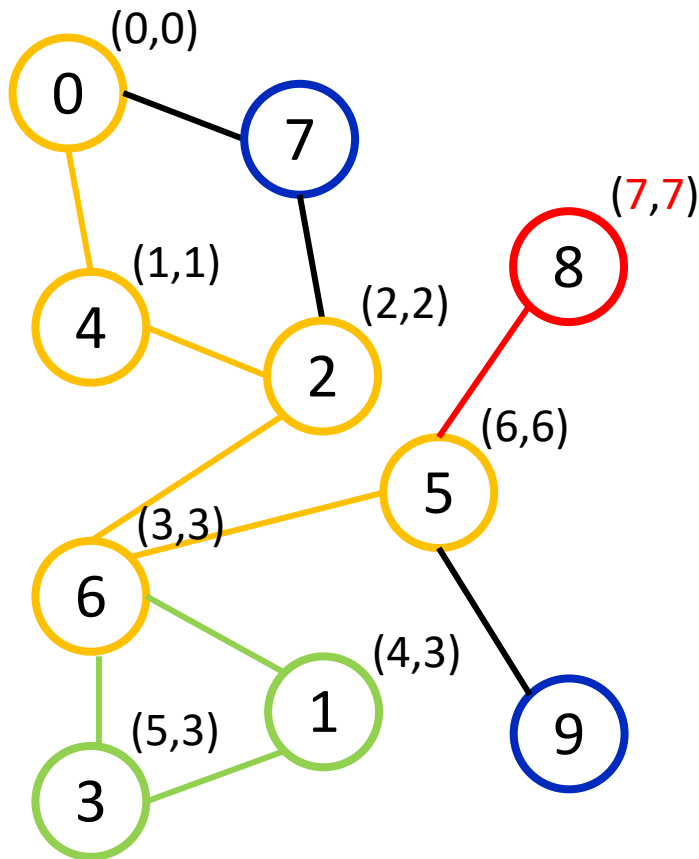


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {
2.     used[v] = true;
3.     tin[v] = (fup[v] = timer++);
4.     for (auto &to : g[v]) {
5.         if (to == p)
6.             continue;
7.         if (used[to])
8.             fup[v] = min(fup[v], tin[to]);
9.         else {
10.            dfs(to, v);
11.            fup[v] = min(fup[v], fup[to]);
12.            if (fup[to] > tin[v])
13.                isBridge(v, to);
14.        }
15.    }
16.}
```

Поиск мостов на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

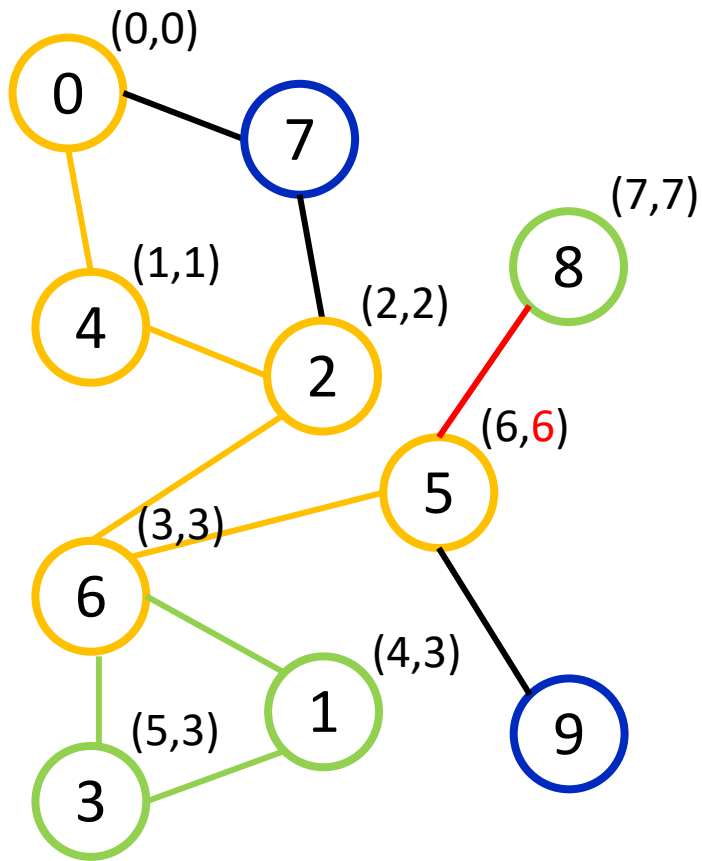


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {
2.     used[v] = true;
3.     tin[v] = (fup[v] = timer++);
4.     for (auto &to : g[v]) {
5.         if (to == p)
6.             continue;
7.         if (used[to])
8.             fup[v] = min(fup[v], tin[to]);
9.         else {
10.            dfs(to, v);
11.            fup[v] = min(fup[v], fup[to]);
12.            if (fup[to] > tin[v])
13.                isBridge(v, to);
14.        }
15.    }
16.}
```

Поиск мостов на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

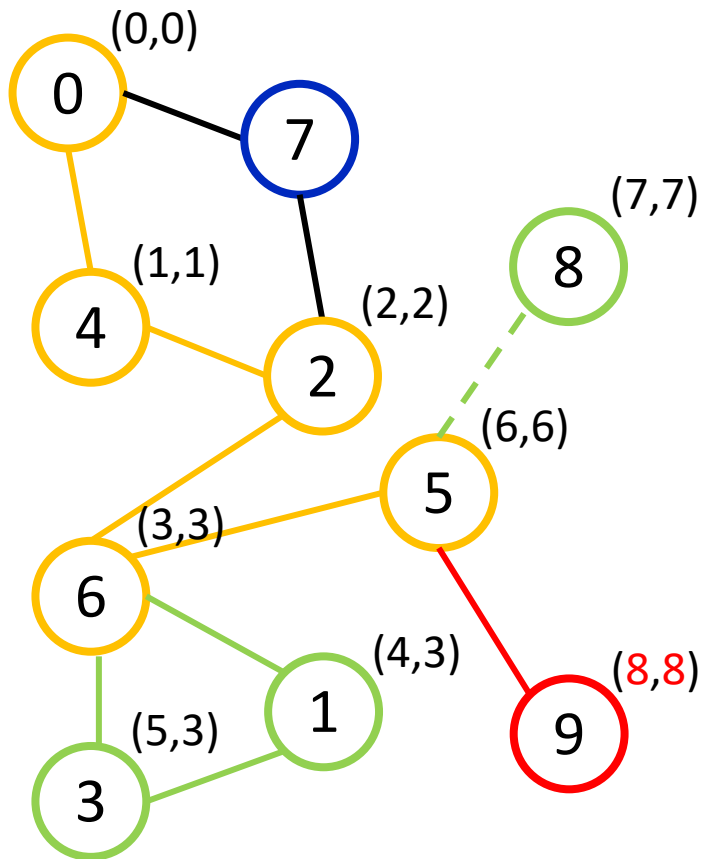


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {  
2.     used[v] = true;  
3.     tin[v] = (fup[v] = timer++);  
4.     for (auto &to : g[v]) {  
5.         if (to == p)  
6.             continue;  
7.         if (used[to])  
8.             fup[v] = min(fup[v], tin[to]);  
9.         else {  
10.            dfs(to, v);  
11.            fup[v] = min(fup[v], fup[to]);  
12.            if (fup[to] > tin[v])  
13.                isBridge(v, to);  
14.        }  
15.    }  
16.}
```

Поиск мостов на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

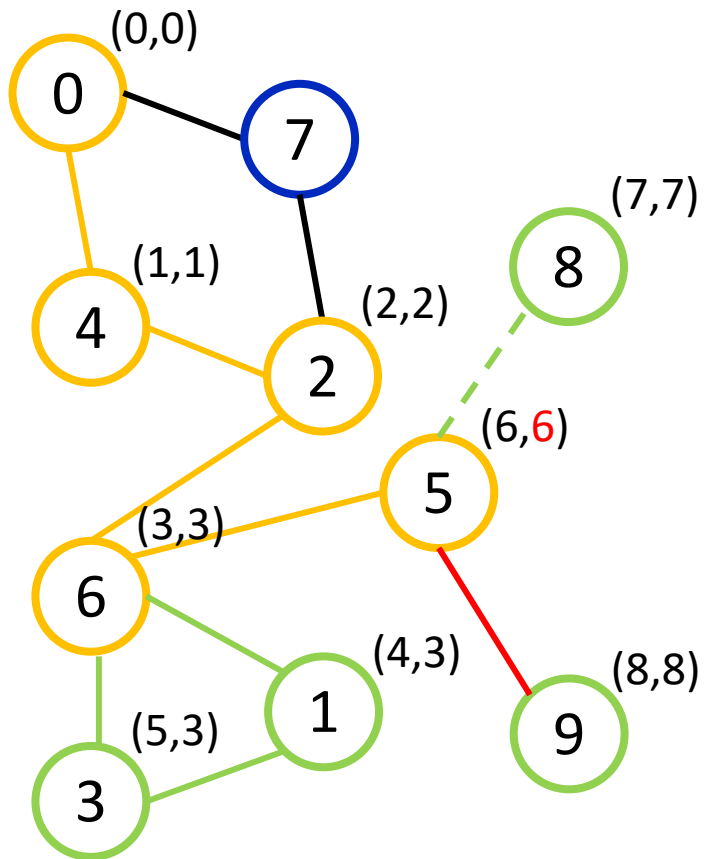


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {  
2.     used[v] = true;  
3.     tin[v] = (fup[v] = timer++);  
4.     for (auto &to : g[v]) {  
5.         if (to == p)  
6.             continue;  
7.         if (used[to])  
8.             fup[v] = min(fup[v], tin[to]);  
9.         else {  
10.            dfs(to, v);  
11.            fup[v] = min(fup[v], fup[to]);  
12.            if (fup[to] > tin[v])  
13.                isBridge(v, to);  
14.        }  
15.    }  
16.}
```

Поиск мостов на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

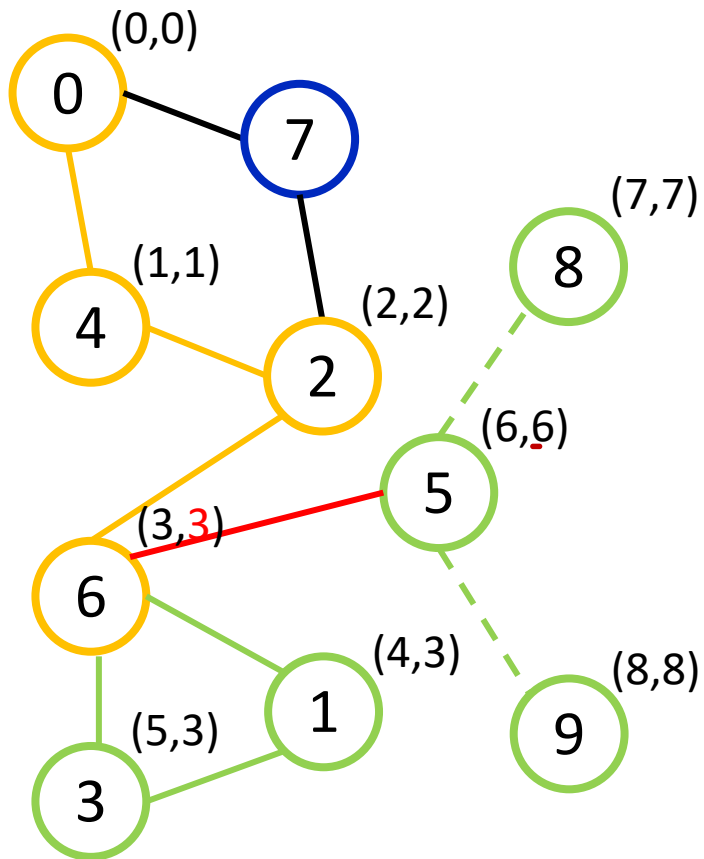


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {  
2.     used[v] = true;  
3.     tin[v] = (fup[v] = timer++);  
4.     for (auto &to : g[v]) {  
5.         if (to == p)  
6.             continue;  
7.         if (used[to])  
8.             fup[v] = min(fup[v], tin[to]);  
9.         else {  
10.            dfs(to, v);  
11.            fup[v] = min(fup[v], fup[to]);  
12.            if (fup[to] > tin[v])  
13.                isBridge(v, to);  
14.        }  
15.    }  
16.}
```

Поиск мостов на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

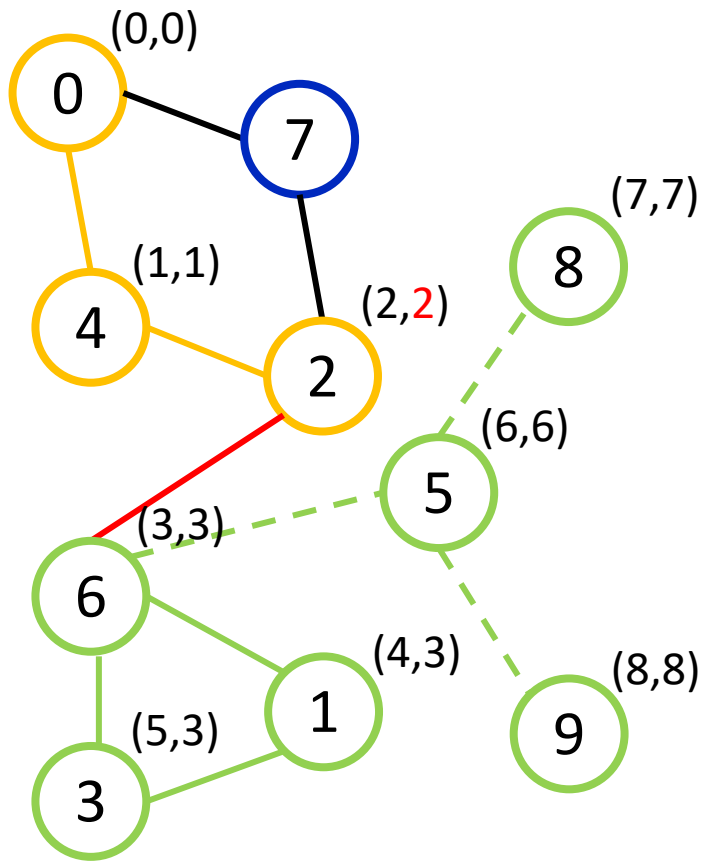


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {  
2.     used[v] = true;  
3.     tin[v] = (fup[v] = timer++);  
4.     for (auto &to : g[v]) {  
5.         if (to == p)  
6.             continue;  
7.         if (used[to])  
8.             fup[v] = min(fup[v], tin[to]);  
9.         else {  
10.            dfs(to, v);  
11.            fup[v] = min(fup[v], fup[to]);  
12.            if (fup[to] > tin[v])  
13.                isBridge(v, to);  
14.        }  
15.    }  
16.}
```


Поиск мостов на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

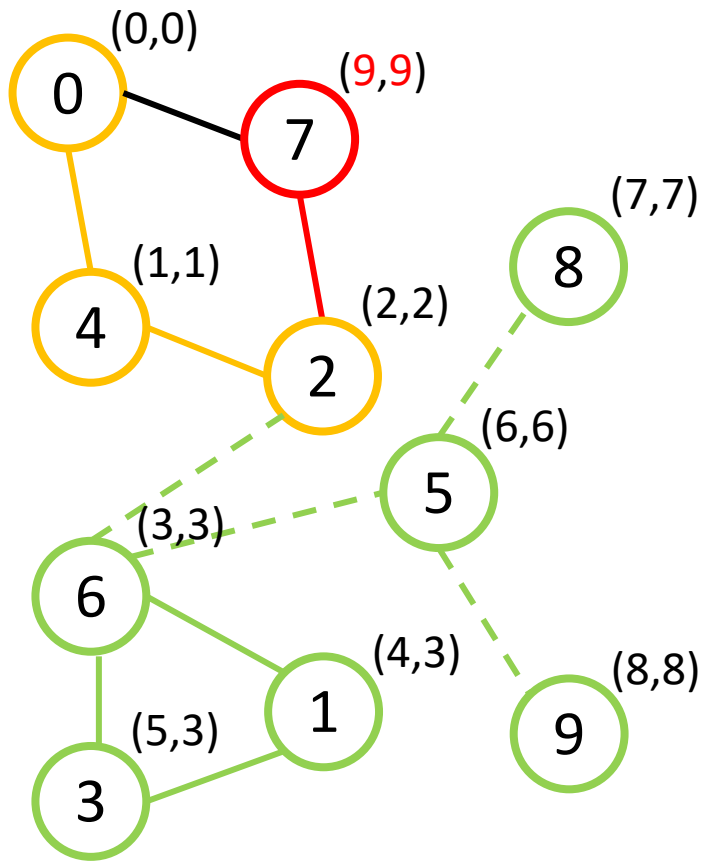


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {  
2.     used[v] = true;  
3.     tin[v] = (fup[v] = timer++);  
4.     for (auto &to : g[v]) {  
5.         if (to == p)  
6.             continue;  
7.         if (used[to])  
8.             fup[v] = min(fup[v], tin[to]);  
9.         else {  
10.            dfs(to, v);  
11.            fup[v] = min(fup[v], fup[to]);  
12.            if (fup[to] > tin[v])  
13.                isBridge(v, to);  
14.        }  
15.    }  
16.}
```

Поиск мостов на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

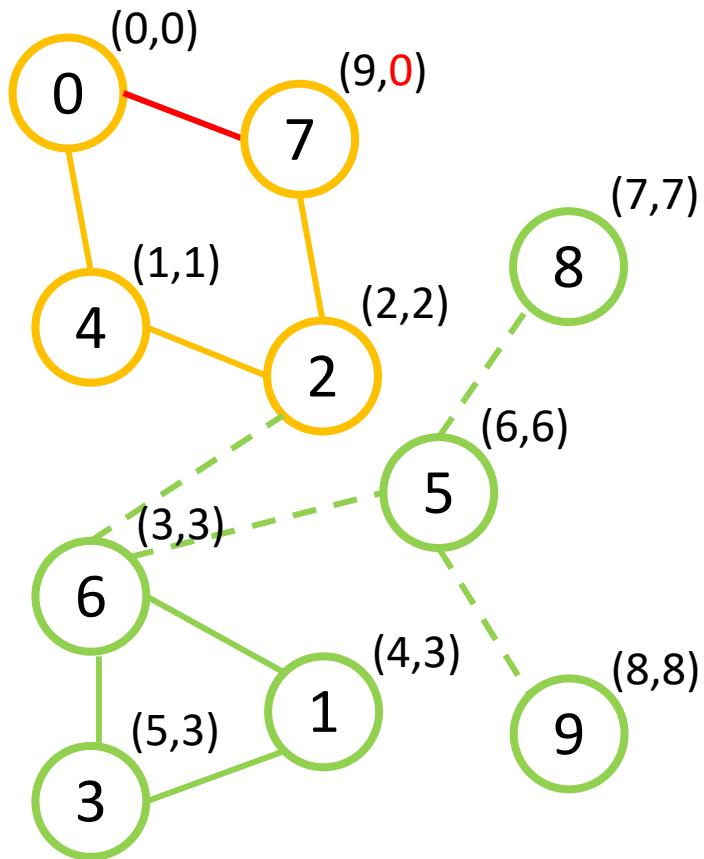


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {
2.     used[v] = true;
3.     tin[v] = (fup[v] = timer++);
4.     for (auto &to : g[v]) {
5.         if (to == p)
6.             continue;
7.         if (used[to])
8.             fup[v] = min(fup[v], tin[to]);
9.         else {
10.            dfs(to, v);
11.            fup[v] = min(fup[v], fup[to]);
12.            if (fup[to] > tin[v])
13.                isBridge(v, to);
14.        }
15.    }
16.}
```

Поиск мостов на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

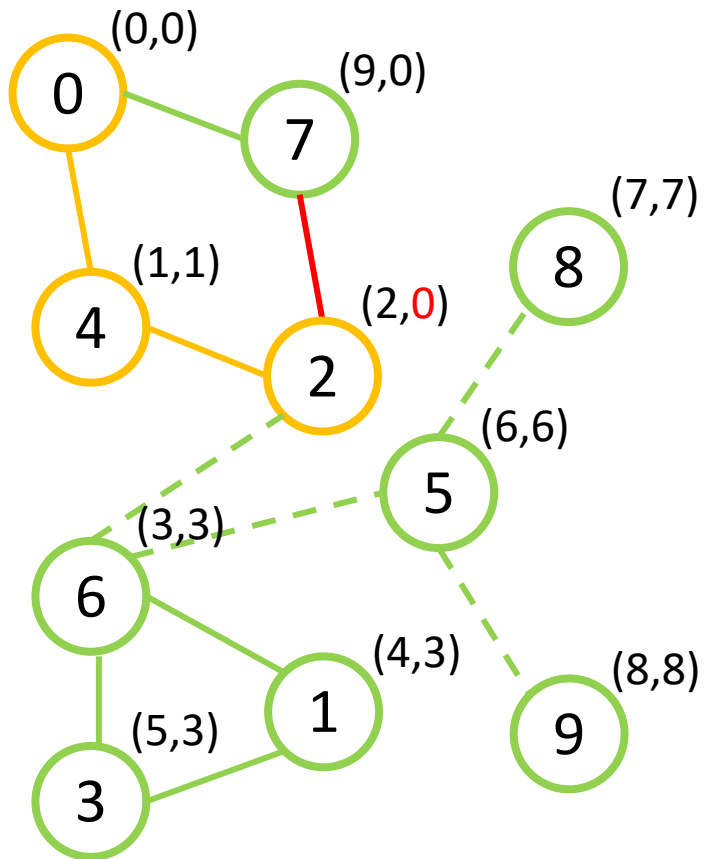


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {  
2.     used[v] = true;  
3.     tin[v] = (fup[v] = timer++);  
4.     for (auto &to : g[v]) {  
5.         if (to == p)  
6.             continue;  
7.         if (used[to])  
8.             fup[v] = min(fup[v], tin[to]);  
9.         else {  
10.            dfs(to, v);  
11.            fup[v] = min(fup[v], fup[to]);  
12.            if (fup[to] > tin[v])  
13.                isBridge(v, to);  
14.        }  
15.    }  
16.}
```

Поиск мостов на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

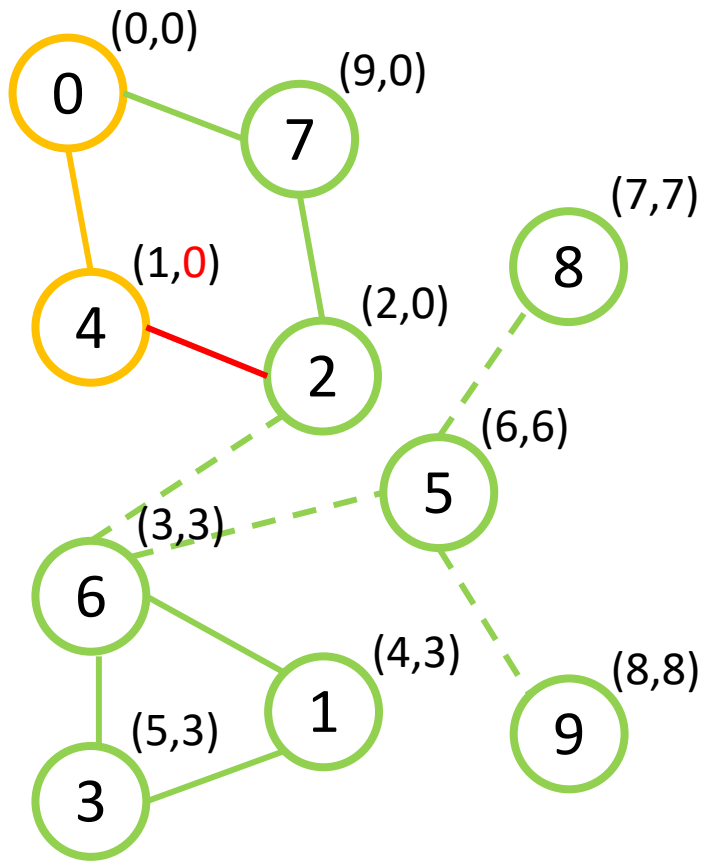


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {
2.     used[v] = true;
3.     tin[v] = (fup[v] = timer++);
4.     for (auto &to : g[v]) {
5.         if (to == p)
6.             continue;
7.         if (used[to])
8.             fup[v] = min(fup[v], tin[to]);
9.         else {
10.            dfs(to, v);
11.            fup[v] = min(fup[v], fup[to]);
12.            if (fup[to] > tin[v])
13.                isBridge(v, to);
14.        }
15.    }
16.}
```

Поиск мостов на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

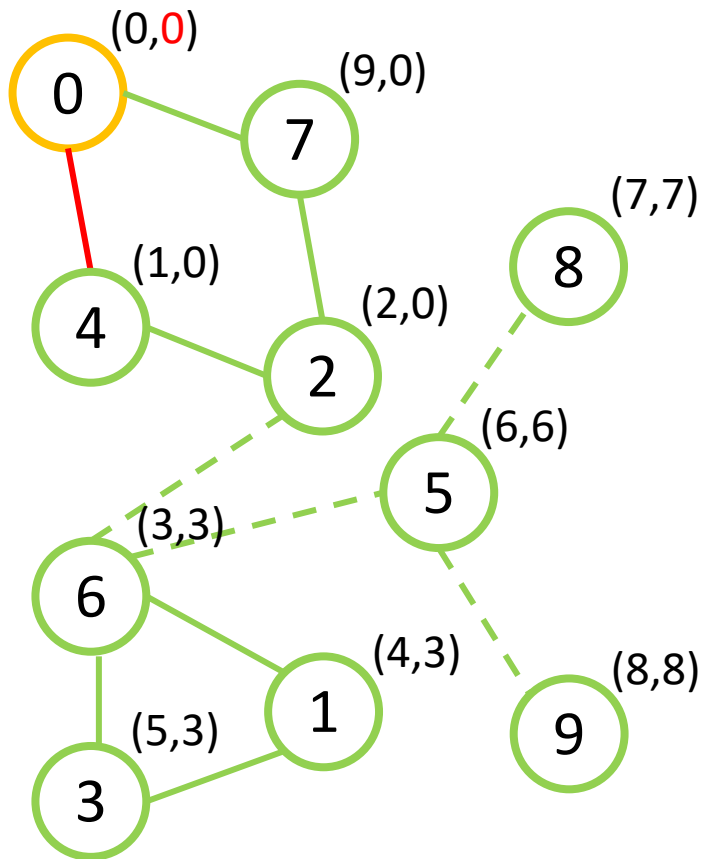


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {
2.     used[v] = true;
3.     tin[v] = (fup[v] = timer++);
4.     for (auto &to : g[v]) {
5.         if (to == p)
6.             continue;
7.         if (used[to])
8.             fup[v] = min(fup[v], tin[to]);
9.         else {
10.            dfs(to, v);
11.            fup[v] = min(fup[v], fup[to]);
12.            if (fup[to] > tin[v])
13.                isBridge(v, to);
14.        }
15.    }
16.}
```

Поиск мостов на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

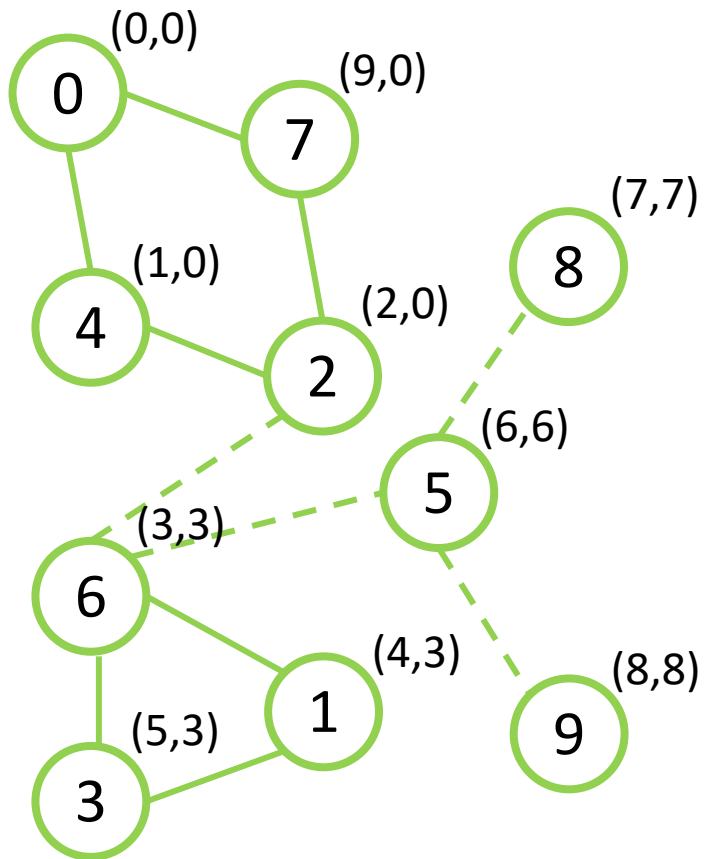


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {  
2.     used[v] = true;  
3.     tin[v] = (fup[v] = timer++);  
4.     for (auto &to : g[v]) {  
5.         if (to == p)  
6.             continue;  
7.         if (used[to])  
8.             fup[v] = min(fup[v], tin[to]);  
9.         else {  
10.            dfs(to, v);  
11.            fup[v] = min(fup[v], fup[to]);  
12.            if (fup[to] > tin[v])  
13.                isBridge(v, to);  
14.        }  
15.    }  
16.}
```

Поиск мостов на основе DFS – $O(V+E)$. Пример

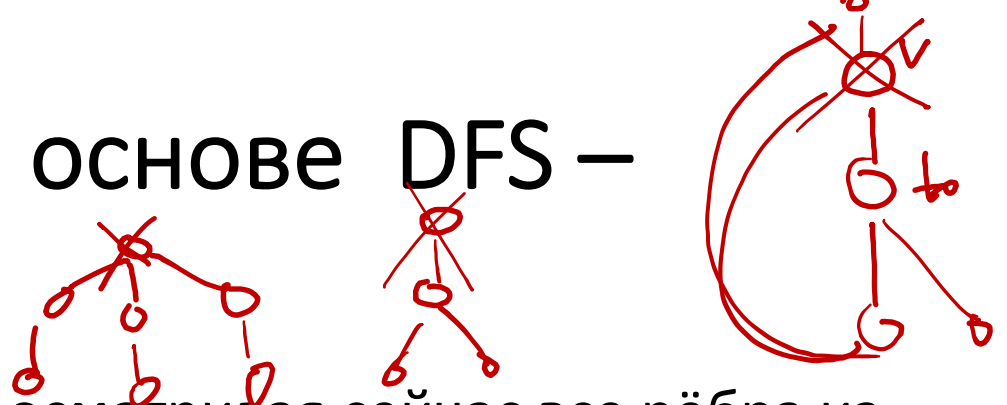
Неориентированный граф G



Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {
2.     used[v] = true;
3.     tin[v] = (fup[v] = timer++);
4.     for (auto &to : g[v]) {
5.         if (to == p)
6.             continue;
7.         if (used[to])
8.             fup[v] = min(fup[v], tin[to]);
9.         else {
10.            dfs(to, v);
11.            fup[v] = min(fup[v], fup[to]);
12.            if (fup[to] > tin[v])
13.                isBridge(v, to);
14.        }
15.    }
16.}
```

Поиск точек сочленения на основе DFS – $O(V+E)$



- Пусть мы находимся в обходе в глубину, просматривая сейчас все рёбра из вершины $v \neq root$. Тогда, если текущее ребро (v, to) таково, что из вершины to и из любого её потомка в дереве обхода в глубину нет обратного ребра в какого-либо предка вершины v , то вершина v является точкой сочленения. В противном случае, т.е. если обход в глубину просмотрел все рёбра из вершины v , и не нашёл удовлетворяющего вышеописанным условиям ребра, то вершина v не является точкой сочленения.
- В самом деле, мы этим условием проверяем, нет ли другого пути из v в to .
- Рассмотрим теперь оставшийся случай: $v = root$. Тогда эта вершина является точкой сочленения тогда и только тогда, когда эта вершина имеет более одного потомка в дереве обхода в глубину.
- В самом деле, это означает, что, пройдя из $root$ по произвольному ребру, мы не смогли обойти весь граф, откуда сразу следует, что $root$ — точка сочленения.

Поиск точек сочленения на основе DFS – $O(V+E)$

- Итак, пусть $tin[v]$ — это время захода поиска в глубину в вершину v . Теперь введём массив $fup[v]$, который и позволит нам отвечать на вышеописанные запросы. Время $fup[v]$ равно минимуму из времени захода в саму вершину $tin[v]$, времён захода в каждую вершину p , являющуюся концом некоторого обратного ребра (v, p) , а также из всех значений $fup[to]$ для каждой вершины to , являющейся непосредственным потомком v в дереве поиска:

$$fup[v] = \min \left\{ \begin{array}{l} \overline{tin[v]}, \\ \overline{tin[p], (v, p)} - \text{обратное ребро} \\ \overline{fup[to], (v, to)} - \text{древесное ребро} \end{array} \right.$$

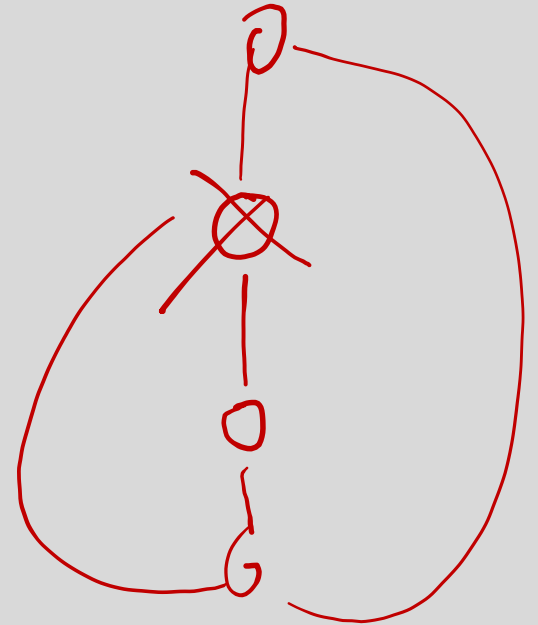
Поиск точек сочленения на основе DFS – $O(V+E)$. Алгоритм

- Если говорить о самой реализации, то здесь нам нужно уметь различать три случая: когда мы идём по ребру дерева поиска в глубину, когда идём по обратному ребру, и когда пытаемся пойти по ребру дерева в обратную сторону. Это, соответственно, случаи:
 - $used[to] = false$ — критерий ребра дерева поиска;
 - $used[to] = true \ \&\& \ to \neq parent$ — критерий обратного ребра;
 - $to = parent$ — критерий прохода по ребру дерева поиска в обратную сторону.

```
1.  const int kMaxNum = ...;
2.  vector<vector<int>> g(kMaxNum);
3.  bool used[kMaxNum];
4.  int timer = 0;
5.  int tin[kMaxNum];
6.  int fup[kMaxNum];

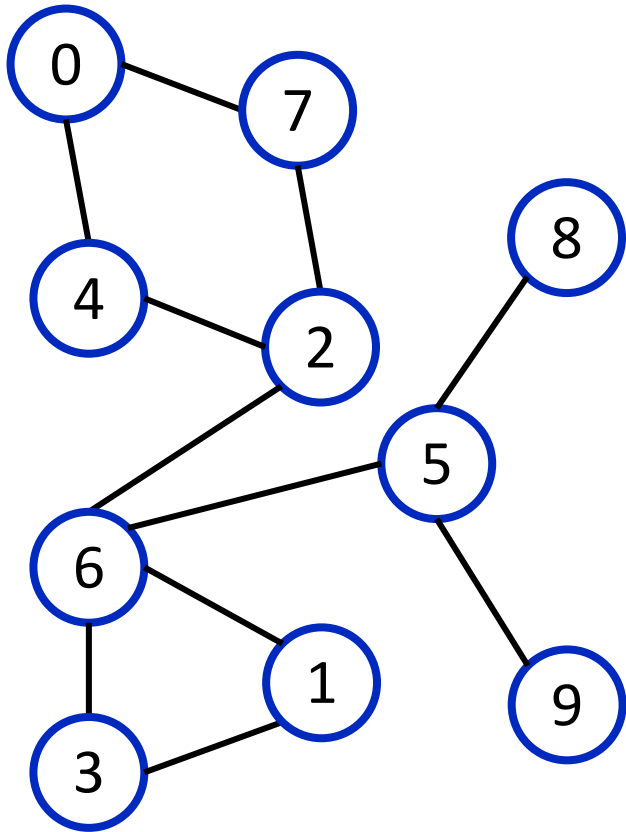
7.  void dfs(int v, int p = -1) {
8.      | used[v] = true;
9.      | tin[v] = (fup[v] = timer++);
10.     int children = 0;
11.     for (auto &to : g[v]) {
12.         if (to == p)
13.             continue;
14.         if (used[to])
15.             fup[v] = min(fup[v], tin[to]);
16.         else {
17.             dfs(to, v);
18.             fup[v] = min(fup[v], fup[to]);
19.             ++children;
20.             if (fup[to] >= tin[v] && p != -1)
21.                 isCutpoint(v);
22.         }
23.     }
24.     if (p == -1 && children > 1)
25.         isCutpoint(v);
26. }

27. int main() {
28.     int n;
29.     // Чтение g.
30.     timer = 0;
31.     for (int i = 0; i < kMaxNum; ++i)
32.         used[i] = false;
33.     dfs(0);
34.     . . . }
```



Поиск точек сочленения на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

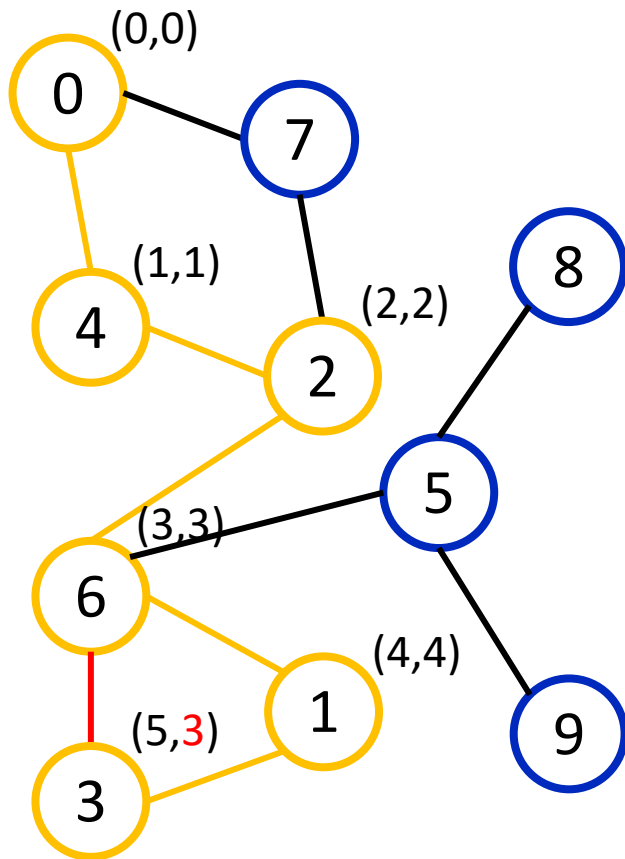


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {
2.     used[v] = true;
3.     tin[v] = (fup[v] = timer++);
4.     int children = 0;
5.     for (auto &to : g[v]) {
6.         if (to == p)
7.             continue;
8.         if (used[to])
9.             fup[v] = min(fup[v], tin[to]);
10.        else {
11.            dfs(to, v);
12.            fup[v] = min(fup[v], fup[to]);
13.            ++children;
14.            if (fup[to] >= tin[v] && p != -1)
15.                isCutpoint(v);
16.        }
17.    }
18.    if (p == -1 && children > 1)
19.        isCutpoint(v);
20. }
```

Поиск точек сочленения на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

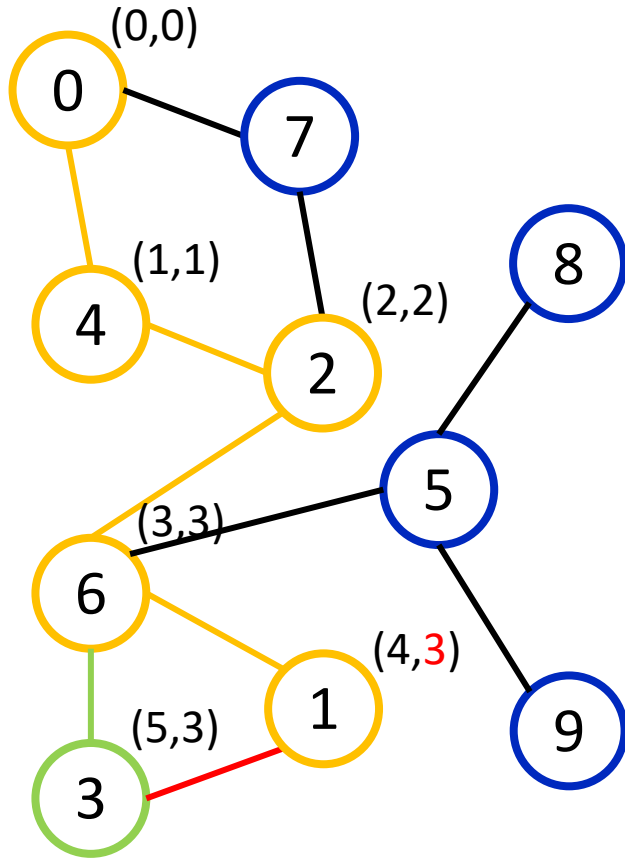


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {
2.     used[v] = true;
3.     tin[v] = (fup[v] = timer++);
4.     int children = 0;
5.     for (auto &to : g[v]) {
6.         if (to == p)
7.             continue;
8.         if (used[to])
9.             fup[v] = min(fup[v], tin[to]);
10.        else {
11.            dfs(to, v);
12.            fup[v] = min(fup[v], fup[to]);
13.            ++children;
14.            if (fup[to] >= tin[v] && p != -1)
15.                isCutpoint(v);
16.        }
17.    }
18.    if (p == -1 && children > 1)
19.        isCutpoint(v);
20. }
```

Поиск точек сочленения на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

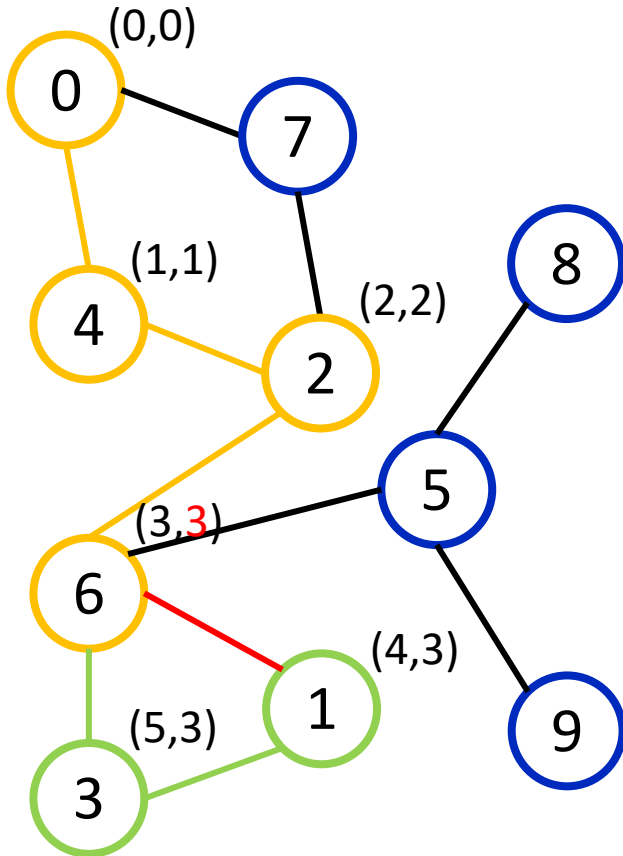


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {
2.     used[v] = true;
3.     tin[v] = (fup[v] = timer++);
4.     int children = 0;
5.     for (auto &to : g[v]) {
6.         if (to == p)
7.             continue;
8.         if (used[to])
9.             fup[v] = min(fup[v], tin[to]);
10.        else {
11.            dfs(to, v);
12.            fup[v] = min(fup[v], fup[to]);
13.            ++children;
14.            if (fup[to] >= tin[v] && p != -1)
15.                isCutpoint(v);
16.        }
17.    }
18.    if (p == -1 && children > 1)
19.        isCutpoint(v);
20. }
```

Поиск точек сочленения на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

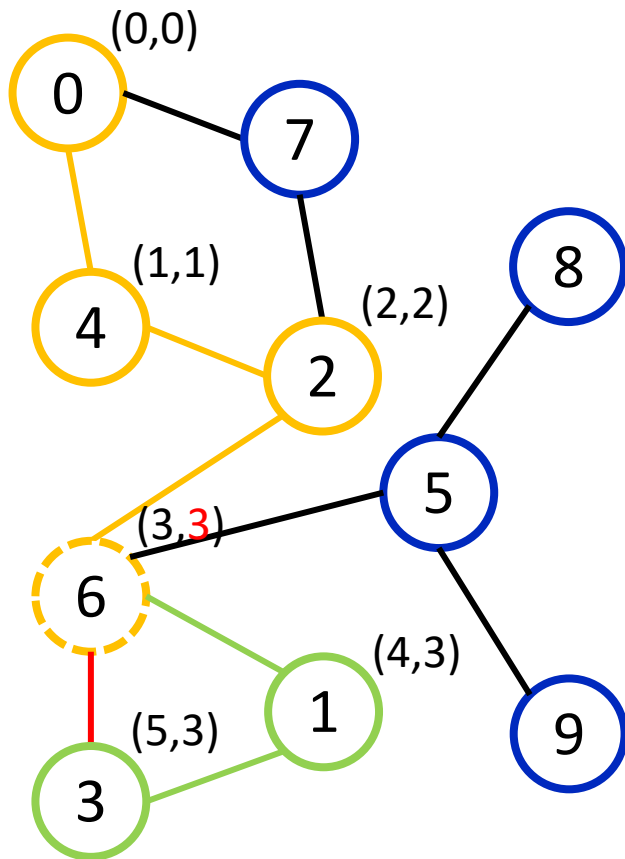


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {
2.     used[v] = true;
3.     tin[v] = (fup[v] = timer++);
4.     int children = 0;
5.     for (auto &to : g[v]) {
6.         if (to == p)
7.             continue;
8.         if (used[to])
9.             fup[v] = min(fup[v], tin[to]);
10.        else {
11.            dfs(to, v);
12.            fup[v] = min(fup[v], fup[to]);
13.            ++children;
14.            if (fup[to] >= tin[v] && p != -1)
15.                isCutpoint(v);
16.        }
17.    }
18.    if (p == -1 && children > 1)
19.        isCutpoint(v);
20. }
```

Поиск точек сочленения на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

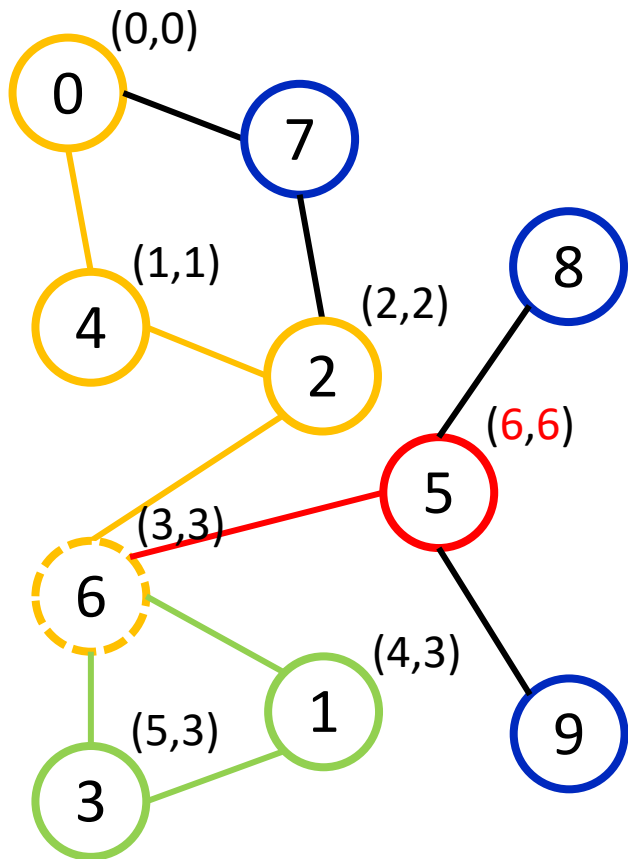


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {  
2.     used[v] = true;  
3.     tin[v] = (fup[v] = timer++);  
4.     int children = 0;  
5.     for (auto &to : g[v]) {  
6.         if (to == p)  
7.             continue;  
8.         if (used[to])  
9.             fup[v] = min(fup[v], tin[to]);  
10.        else {  
11.            dfs(to, v);  
12.            fup[v] = min(fup[v], fup[to]);  
13.            ++children;  
14.            if (fup[to] >= tin[v] && p != -1)  
15.                isCutpoint(v);  
16.        }  
17.    }  
18.    if (p == -1 && children > 1)  
19.        isCutpoint(v);  
20. }
```

Поиск точек сочленения на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

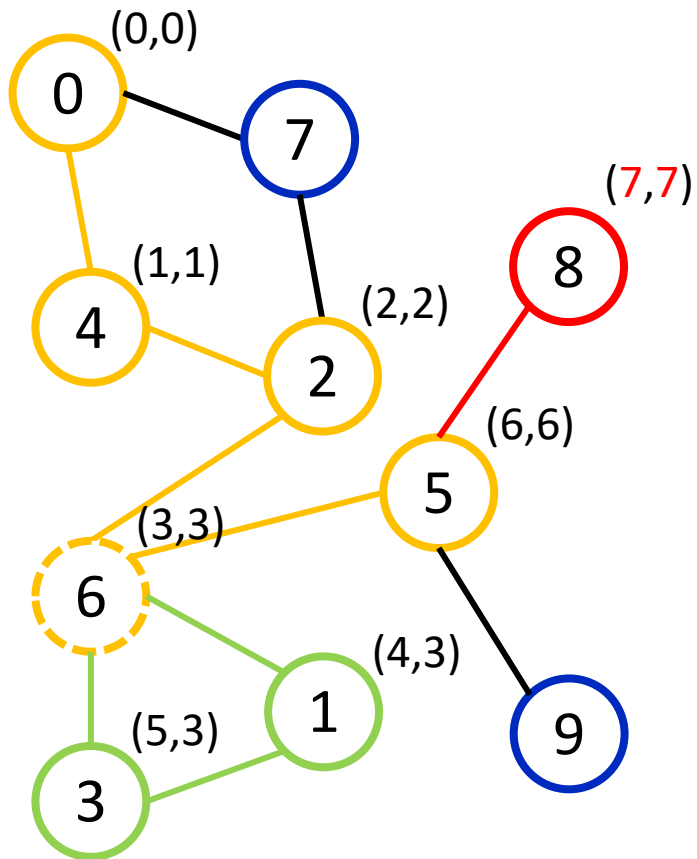


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {  
2.     used[v] = true;  
3.     tin[v] = (fup[v] = timer++);  
4.     int children = 0;  
5.     for (auto &to : g[v]) {  
6.         if (to == p)  
7.             continue;  
8.         if (used[to])  
9.             fup[v] = min(fup[v], tin[to]);  
10.        else {  
11.            dfs(to, v);  
12.            fup[v] = min(fup[v], fup[to]);  
13.            ++children;  
14.            if (fup[to] >= tin[v] && p != -1)  
15.                isCutpoint(v);  
16.        }  
17.    }  
18.    if (p == -1 && children > 1)  
19.        isCutpoint(v);  
20. }
```


Поиск точек сочленения на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

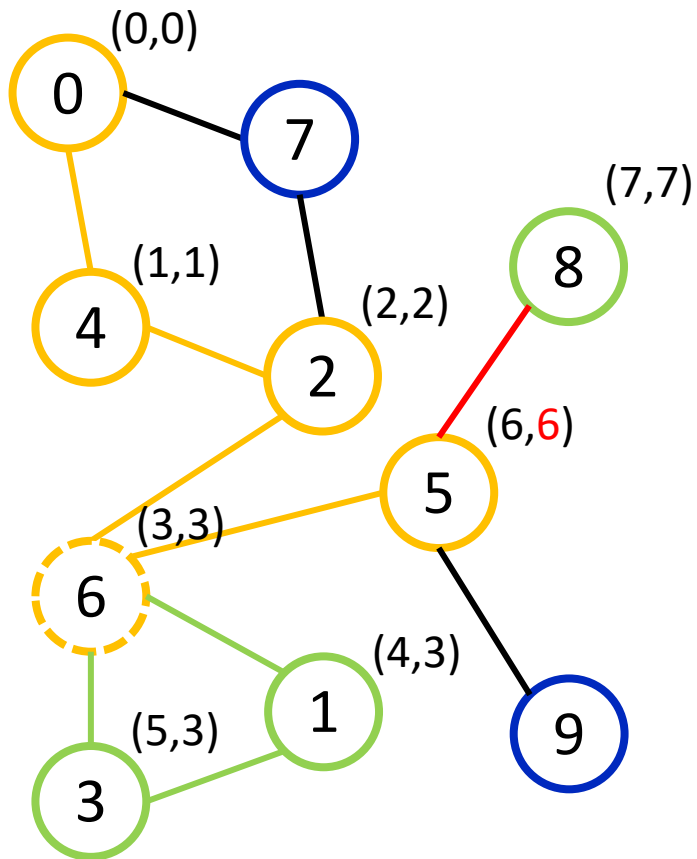


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {
2.     used[v] = true;
3.     tin[v] = (fup[v] = timer++);
4.     int children = 0;
5.     for (auto &to : g[v]) {
6.         if (to == p)
7.             continue;
8.         if (used[to])
9.             fup[v] = min(fup[v], tin[to]);
10.        else {
11.            dfs(to, v);
12.            fup[v] = min(fup[v], fup[to]);
13.            ++children;
14.            if (fup[to] >= tin[v] && p != -1)
15.                isCutpoint(v);
16.        }
17.    }
18.    if (p == -1 && children > 1)
19.        isCutpoint(v);
20. }
```

Поиск точек сочленения на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

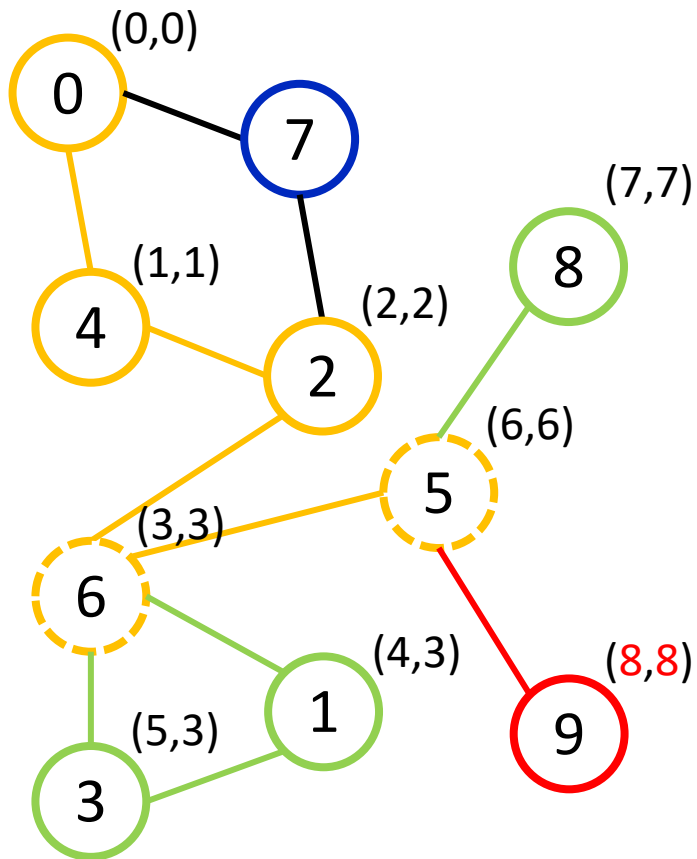


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {  
2.     used[v] = true;  
3.     tin[v] = (fup[v] = timer++);  
4.     int children = 0;  
5.     for (auto &to : g[v]) {  
6.         if (to == p)  
7.             continue;  
8.         if (used[to])  
9.             fup[v] = min(fup[v], tin[to]);  
10.        else {  
11.            dfs(to, v);  
12.            fup[v] = min(fup[v], fup[to]);  
13.            ++children;  
14.            if (fup[to] >= tin[v] && p != -1)  
15.                isCutpoint(v);  
16.        }  
17.    }  
18.    if (p == -1 && children > 1)  
19.        isCutpoint(v);  
20. }
```

Поиск точек сочленения на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

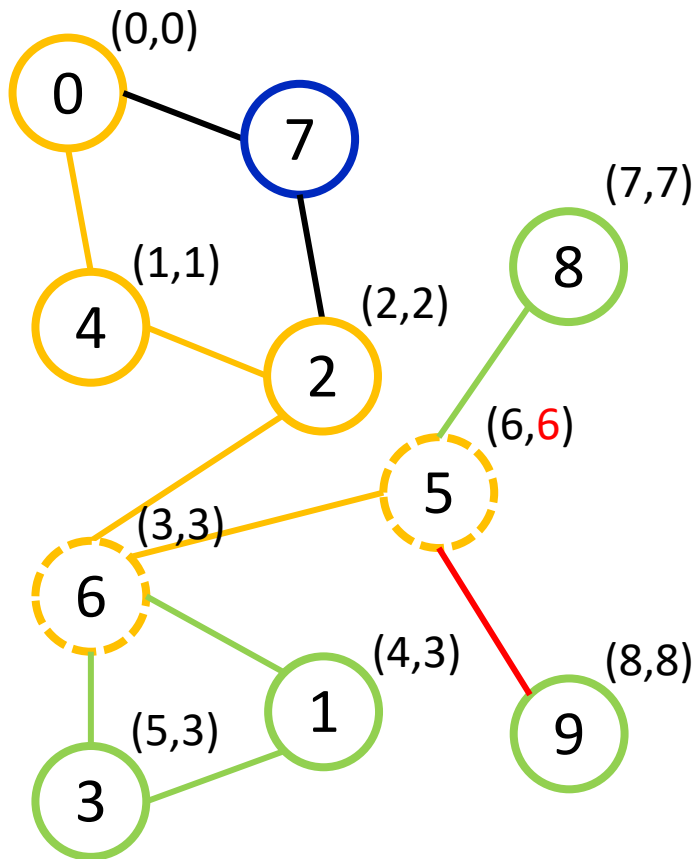


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {
2.     used[v] = true;
3.     tin[v] = (fup[v] = timer++);
4.     int children = 0;
5.     for (auto &to : g[v]) {
6.         if (to == p)
7.             continue;
8.         if (used[to])
9.             fup[v] = min(fup[v], tin[to]);
10.        else {
11.            dfs(to, v);
12.            fup[v] = min(fup[v], fup[to]);
13.            ++children;
14.            if (fup[to] >= tin[v] && p != -1)
15.                isCutpoint(v);
16.        }
17.    }
18.    if (p == -1 && children > 1)
19.        isCutpoint(v);
20. }
```

Поиск точек сочленения на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

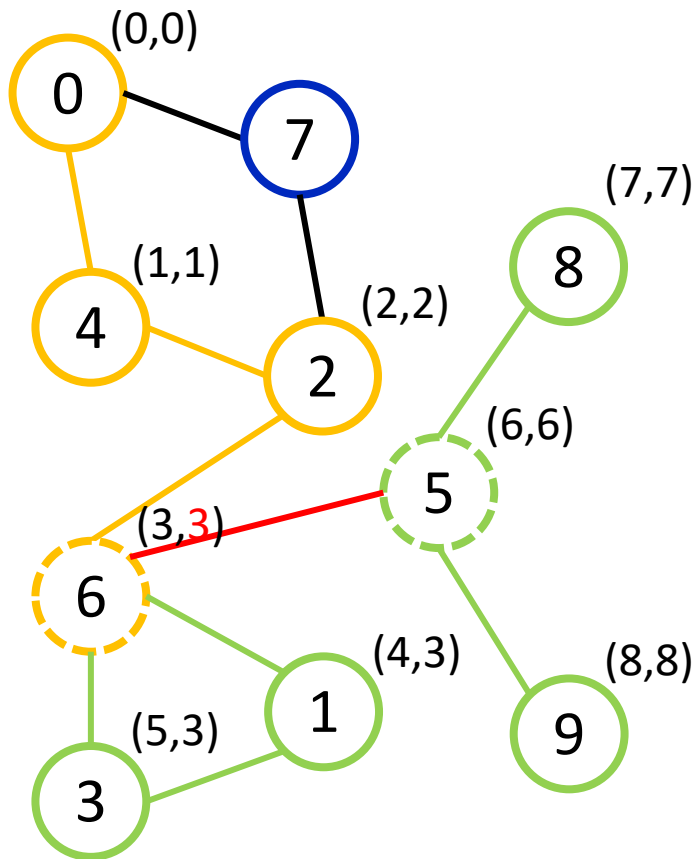


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {  
2.     used[v] = true;  
3.     tin[v] = (fup[v] = timer++);  
4.     int children = 0;  
5.     for (auto &to : g[v]) {  
6.         if (to == p)  
7.             continue;  
8.         if (used[to])  
9.             fup[v] = min(fup[v], tin[to]);  
10.        else {  
11.            dfs(to, v);  
12.            fup[v] = min(fup[v], fup[to]);  
13.            ++children;  
14.            if (fup[to] >= tin[v] && p != -1)  
15.                isCutpoint(v);  
16.        }  
17.    }  
18.    if (p == -1 && children > 1)  
19.        isCutpoint(v);  
20. }
```

Поиск точек сочленения на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

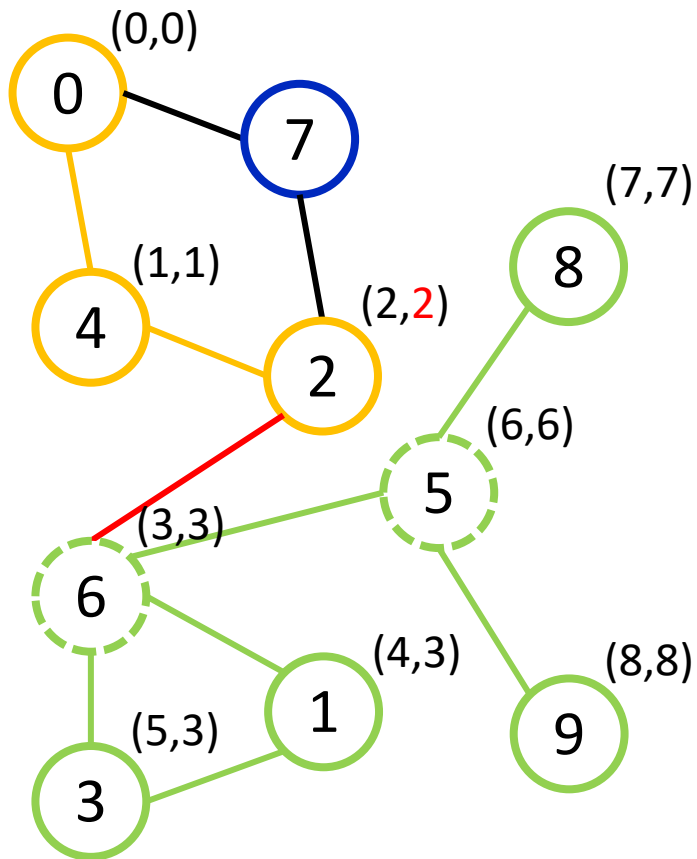


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {
2.     used[v] = true;
3.     tin[v] = (fup[v] = timer++);
4.     int children = 0;
5.     for (auto &to : g[v]) {
6.         if (to == p)
7.             continue;
8.         if (used[to])
9.             fup[v] = min(fup[v], tin[to]);
10.        else {
11.            dfs(to, v);
12.            fup[v] = min(fup[v], fup[to]);
13.            ++children;
14.            if (fup[to] >= tin[v] && p != -1)
15.                isCutpoint(v);
16.        }
17.    }
18.    if (p == -1 && children > 1)
19.        isCutpoint(v);
20. }
```

Поиск точек сочленения на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

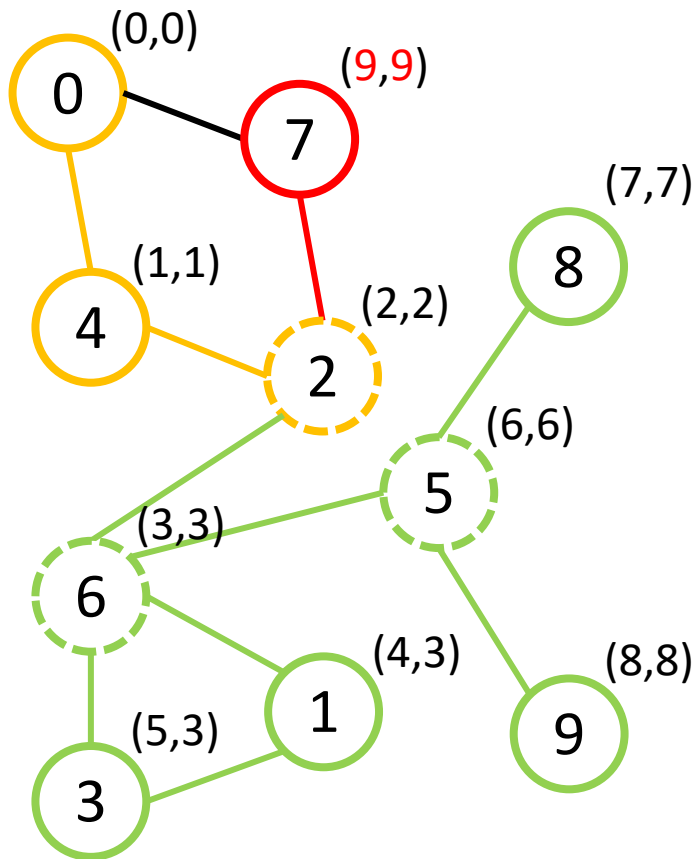


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {
2.     used[v] = true;
3.     tin[v] = (fup[v] = timer++);
4.     int children = 0;
5.     for (auto &to : g[v]) {
6.         if (to == p)
7.             continue;
8.         if (used[to])
9.             fup[v] = min(fup[v], tin[to]);
10.        else {
11.            dfs(to, v);
12.            fup[v] = min(fup[v], fup[to]);
13.            ++children;
14.            if (fup[to] >= tin[v] && p != -1)
15.                isCutpoint(v);
16.        }
17.    }
18.    if (p == -1 && children > 1)
19.        isCutpoint(v);
20. }
```

Поиск точек сочленения на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

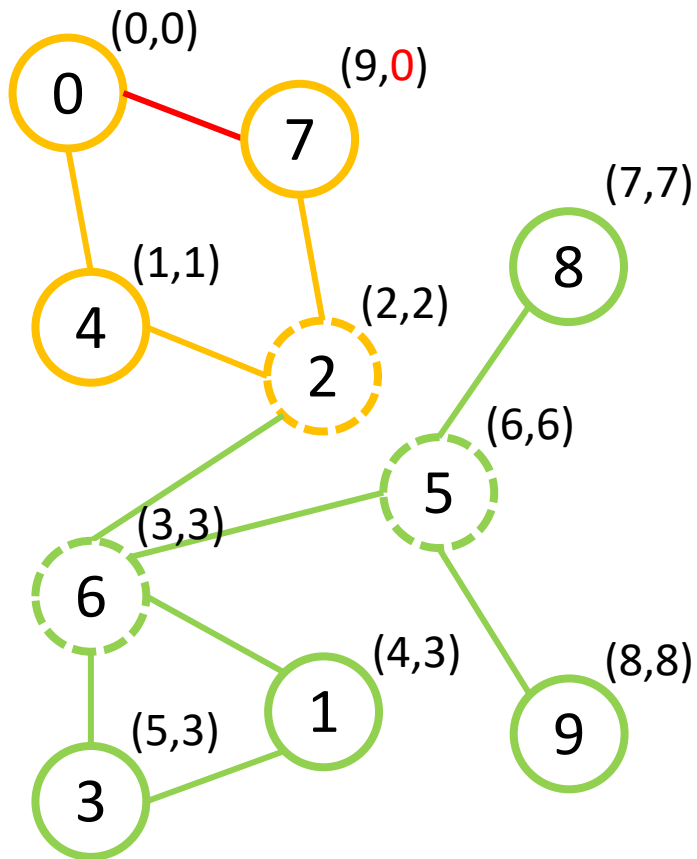


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {
2.     used[v] = true;
3.     tin[v] = (fup[v] = timer++);
4.     int children = 0;
5.     for (auto &to : g[v]) {
6.         if (to == p)
7.             continue;
8.         if (used[to])
9.             fup[v] = min(fup[v], tin[to]);
10.        else {
11.            dfs(to, v);
12.            fup[v] = min(fup[v], fup[to]);
13.            ++children;
14.            if (fup[to] >= tin[v] && p != -1)
15.                isCutpoint(v);
16.        }
17.    }
18.    if (p == -1 && children > 1)
19.        isCutpoint(v);
20. }
```

Поиск точек сочленения на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

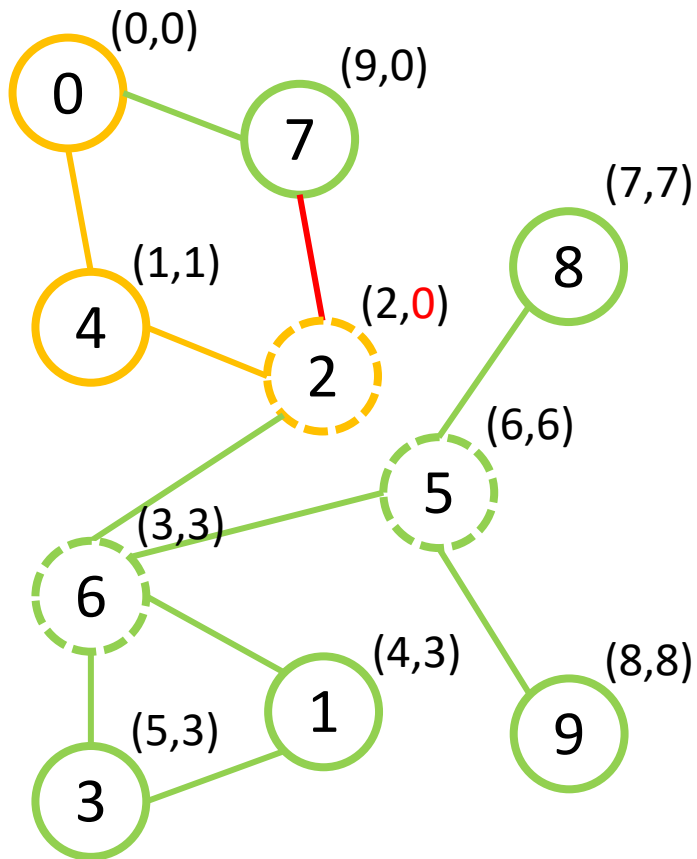


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {
2.     used[v] = true;
3.     tin[v] = (fup[v] = timer++);
4.     int children = 0;
5.     for (auto &to : g[v]) {
6.         if (to == p)
7.             continue;
8.         if (used[to])
9.             fup[v] = min(fup[v], tin[to]);
10.        else {
11.            dfs(to, v);
12.            fup[v] = min(fup[v], fup[to]);
13.            ++children;
14.            if (fup[to] >= tin[v] && p != -1)
15.                isCutpoint(v);
16.        }
17.    }
18.    if (p == -1 && children > 1)
19.        isCutpoint(v);
20. }
```


Поиск точек сочленения на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

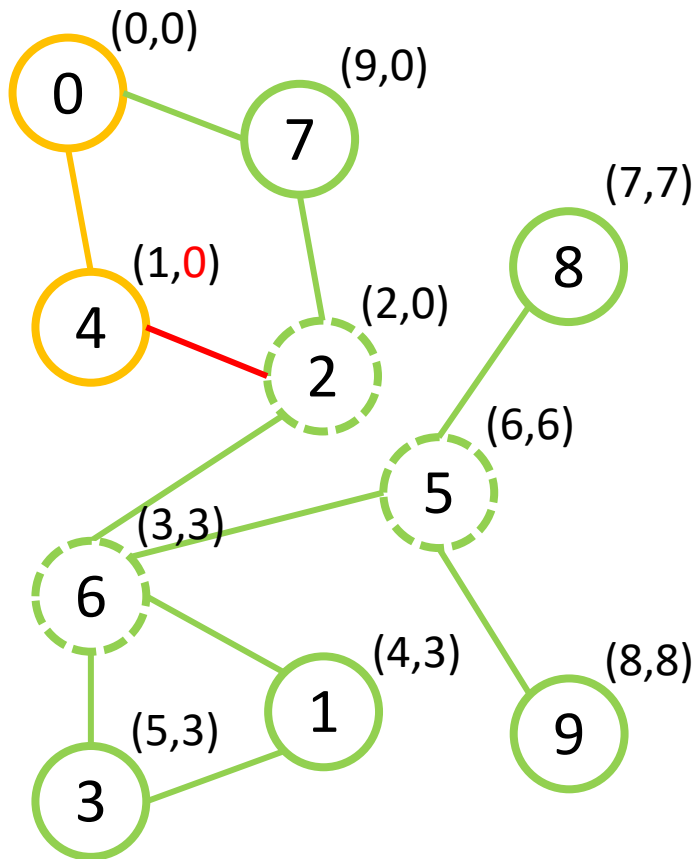


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {
2.     used[v] = true;
3.     tin[v] = (fup[v] = timer++);
4.     int children = 0;
5.     for (auto &to : g[v]) {
6.         if (to == p)
7.             continue;
8.         if (used[to])
9.             fup[v] = min(fup[v], tin[to]);
10.        else {
11.            dfs(to, v);
12.            fup[v] = min(fup[v], fup[to]);
13.            ++children;
14.            if (fup[to] >= tin[v] && p != -1)
15.                isCutpoint(v);
16.        }
17.    }
18.    if (p == -1 && children > 1)
19.        isCutpoint(v);
20. }
```

Поиск точек сочленения на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

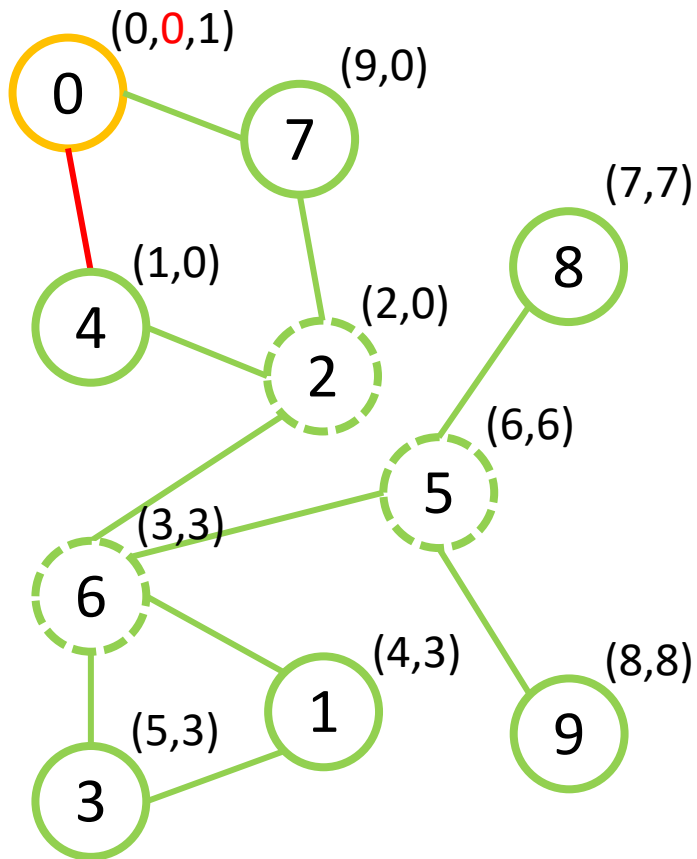


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {
2.     used[v] = true;
3.     tin[v] = (fup[v] = timer++);
4.     int children = 0;
5.     for (auto &to : g[v]) {
6.         if (to == p)
7.             continue;
8.         if (used[to])
9.             fup[v] = min(fup[v], tin[to]);
10.        else {
11.            dfs(to, v);
12.            fup[v] = min(fup[v], fup[to]);
13.            ++children;
14.            if (fup[to] >= tin[v] && p != -1)
15.                isCutpoint(v);
16.        }
17.    }
18.    if (p == -1 && children > 1)
19.        isCutpoint(v);
20. }
```

Поиск точек сочленения на основе DFS – $O(V+E)$. Пример

Неориентированный граф G

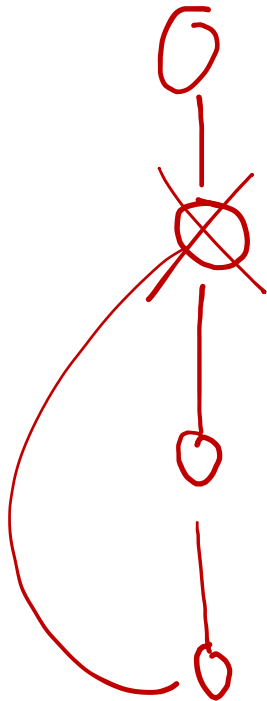
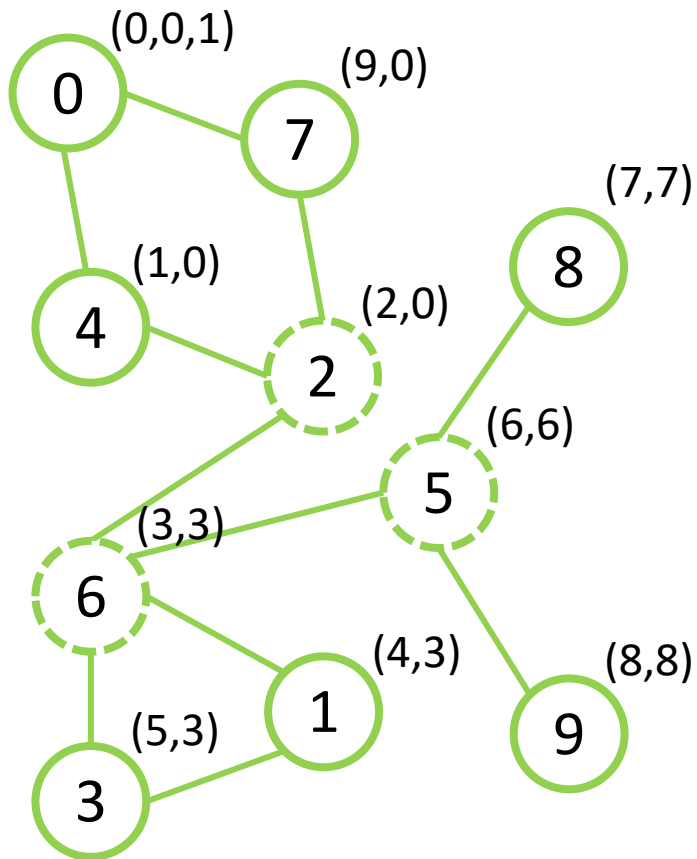


Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {
2.     used[v] = true;
3.     tin[v] = (fup[v] = timer++);
4.     int children = 0;
5.     for (auto &to : g[v]) {
6.         if (to == p)
7.             continue;
8.         if (used[to])
9.             fup[v] = min(fup[v], tin[to]);
10.        else {
11.            dfs(to, v);
12.            fup[v] = min(fup[v], fup[to]);
13.            ++children;
14.            if (fup[to] >= tin[v] && p != -1)
15.                isCutpoint(v);
16.        }
17.    }
18.    if (p == -1 && children > 1)
19.        isCutpoint(v);
20. }
```

Поиск точек сочленения на основе DFS – $O(V+E)$. Пример

Неориентированный граф G



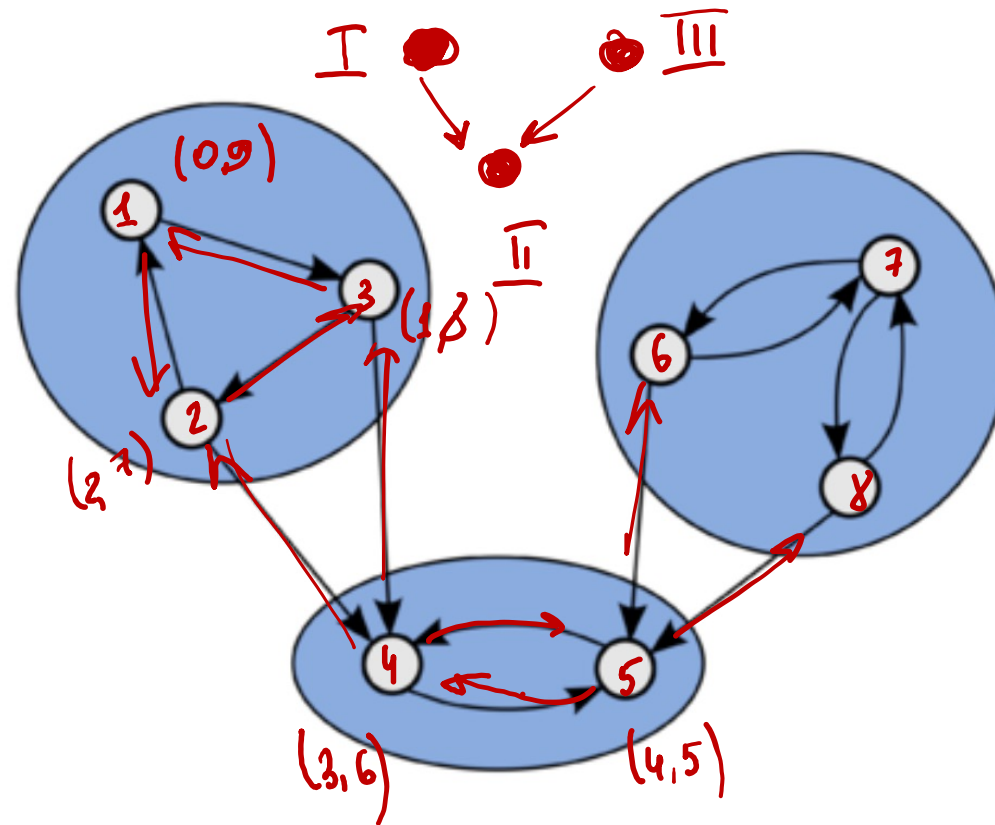
Функция обхода графа в глубину DFS

```
1. void dfs(int v, int p = -1) {
2.     used[v] = true;
3.     tin[v] = (fup[v] = timer++);
4.     int children = 0;
5.     for (auto &to : g[v]) {
6.         if (to == p)
7.             continue;
8.         if (used[to])
9.             fup[v] = min(fup[v], tin[to]);
10.        else {
11.            dfs(to, v);
12.            fup[v] = min(fup[v], fup[to]);
13.            ++children;
14.            if (fup[to] >= tin[v] && p != -1)
15.                isCutpoint(v);
16.        }
17.    }
18.    if (p == -1 && children > 1)
19.        isCutpoint(v);
20. }
```

Алгоритм Косарайю поиска компонент сильной связности

Теорема. Пусть C и C' – две различные компоненты сильной связности, и пусть в графе конденсации между ними есть дуга (C, C') , тогда $\text{tout}[C] > \text{tout}[C']$.

Компонентой сильной связности (strongly connected component) называется такое (максимальное по включению) подмножество вершин, что любые две вершины этого подмножества достижимы друг из друга.



Конденсация - графа, получаемого сжатием каждой компоненты сильной связности в одну вершину. Каждой вершине графа конденсации соответствует компонента сильной связности графа, а ориентированное ребро между двумя вершинами графа конденсации проводится, если найдётся пара вершин из сжатых компонент, между которыми существовало ребро в исходном графе.

Алгоритм Косарайю поиска компонент сильной связности

1. Запустить серию обходов в глубину графа G , которая возвращает вершины в порядке увеличения времени завершения обхода $tout$, то есть некоторый список $order$.
 2. Построить транспонированный граф G^T .
 3. Запустить серию обходов в глубину/ширину этого графа в порядке, определяемом списком $order$ (в обратном порядке, то есть в порядке уменьшения времени выхода). Каждое множество вершин, достигнутое в результате очередного запуска, будет определять отдельную компоненту сильной связности.
- **Сложность** алгоритма будет ограничиваться $O(|V| + |E|)$, так как алгоритм использует два обхода в глубину/ширину.

Алгоритм Косарайю поиска компонент сильной связности. Алгоритм

```
1.  int main() {
2.    int from;
3.    int to;
4.    for (int i = 0; i < kMaxNum; ++i) {
5.        // Считывание ребра (a, b).
6.        g[a].push_back(b);
7.        gr[b].push_back(a);
8.        used[i] = false;
9.    }

10.   for (int i = 0; i < kMaxNum; ++i)
11.       if (!used[i])
12.           dfs(i, false);

13.   for (int i = 0; i < kMaxNum; ++i)
14.       used[i] = false;

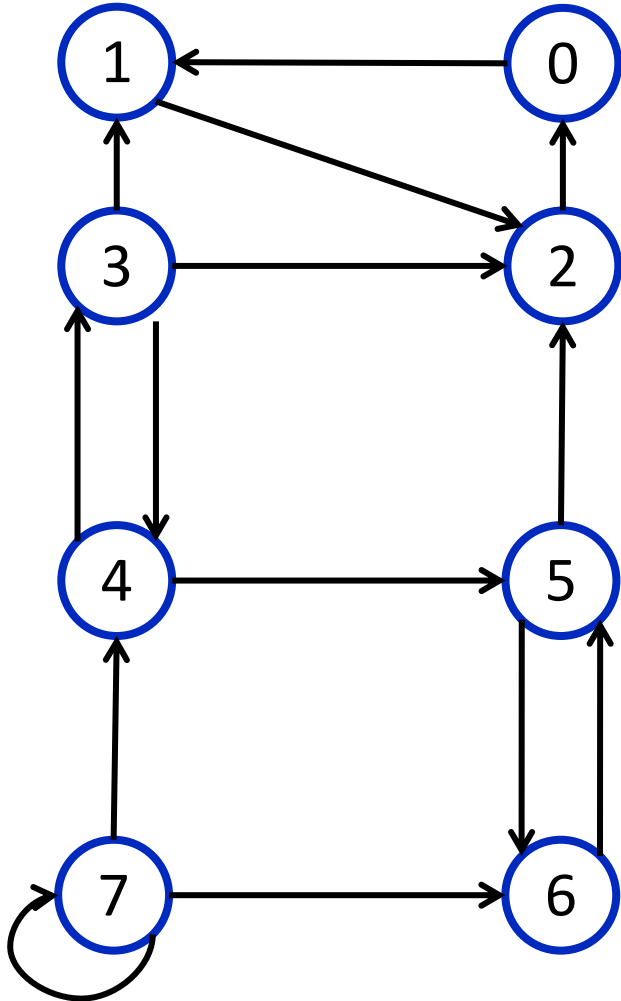
15.   for (int i = 0; i < kMaxNum; ++i) {
16.       int v = order[kMaxNum - 1 - i];
17.       if (!used[i]) {
18.           dfs(v, true);
19.           // Вывод полученной КСС.
20.           component.clear();
21.       }
22.   }
23. }
```

```
1.  const int kMaxNum = ...;
2.  vector<vector<int>> g(kMaxNum);
3.  vector<vector<int>> gr(kMaxNum);
4.  vector<int> order(kMaxNum);
5.  vector<int> component(kMaxNum);
6.  bool used[kMaxNum];

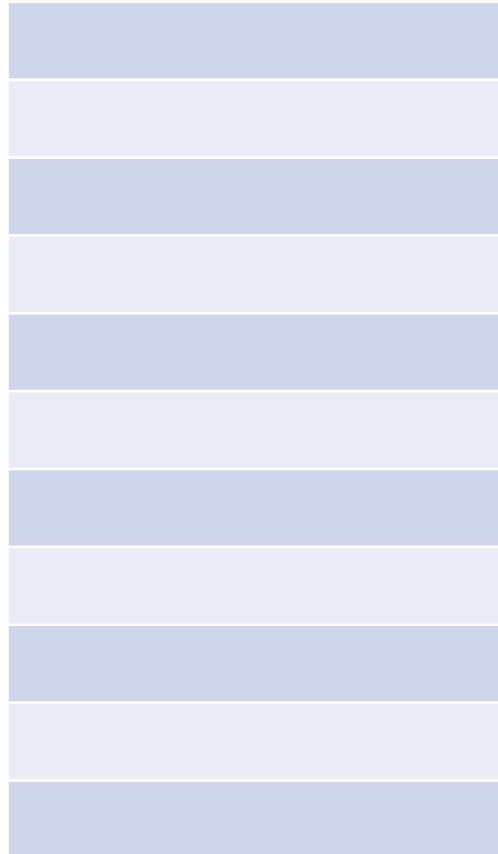
7.  void dfs(int v, bool for_reversed) {
8.      used[v] = true;
9.      if (for_reversed) {
10.         component.push_back(v);
11.         for (auto &to : gr[v]) {
12.             if (!used[to])
13.                 dfs(to, for_reversed);
14.         }
15.     } else {
16.         for (auto &to : g[v]) {
17.             if (!used[to])
18.                 dfs(to, for_reversed);
19.         }
20.         order.push_back(v);
21.     }
22. }
```

Алгоритм Косарайю поиска компонент сильной связности. Пример

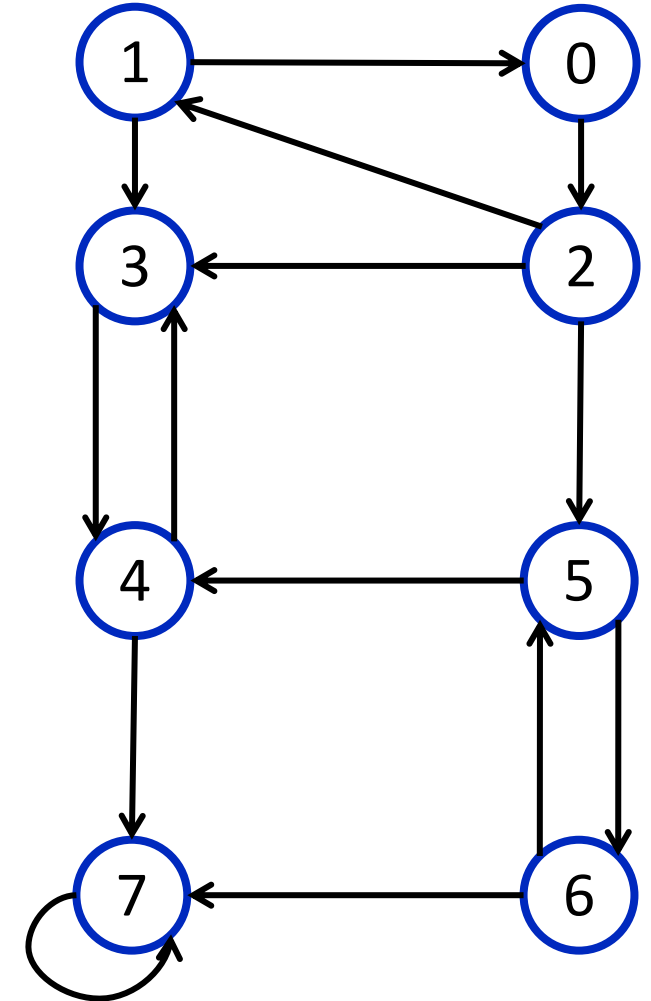
Ориентированный граф G



Список обхода

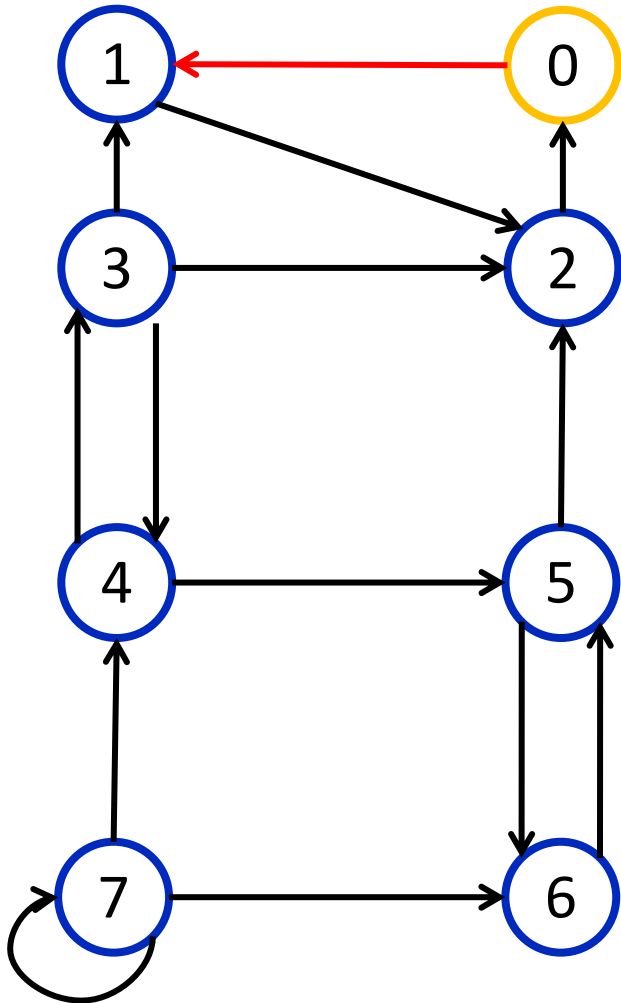


Ориентированный граф G^T

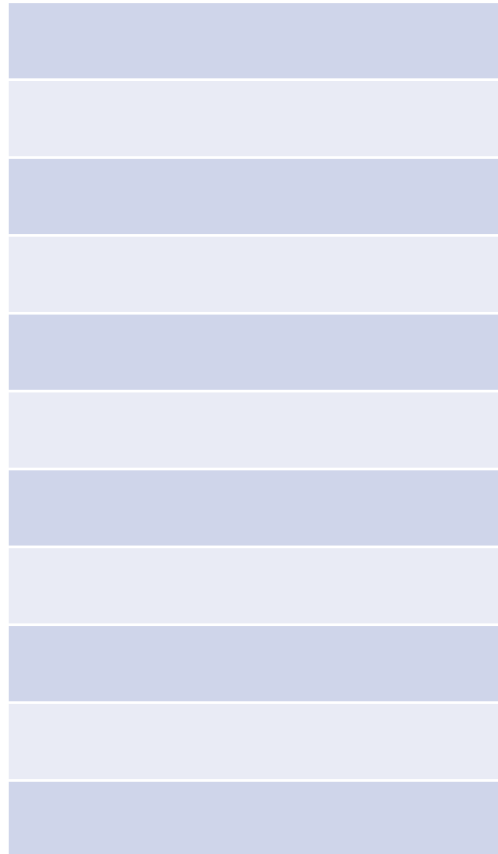


Алгоритм Косарайю поиска компонент сильной связности. Пример

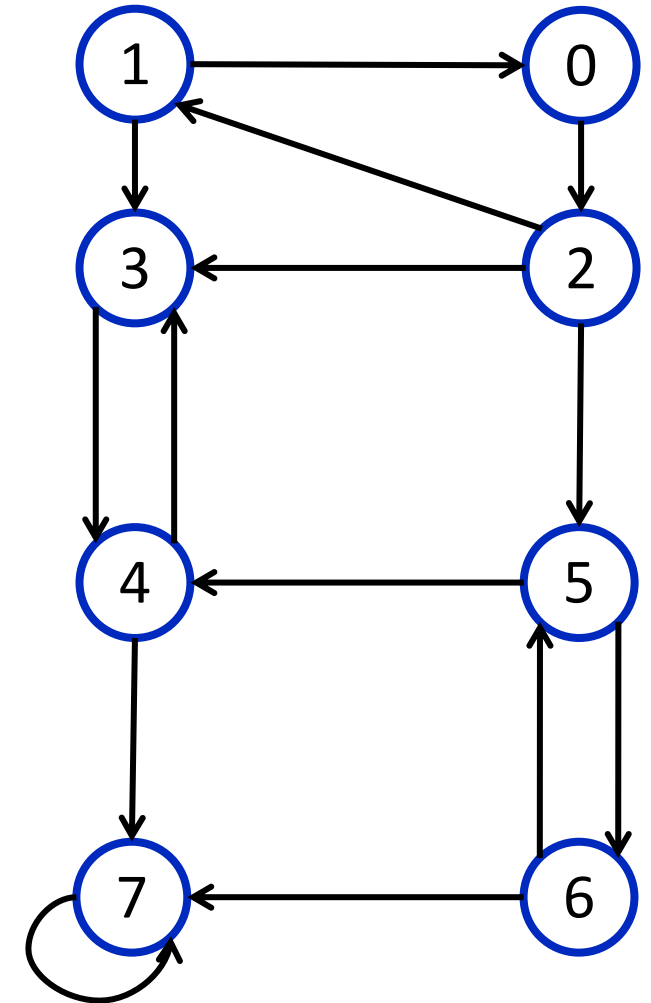
Ориентированный граф G



Список обхода

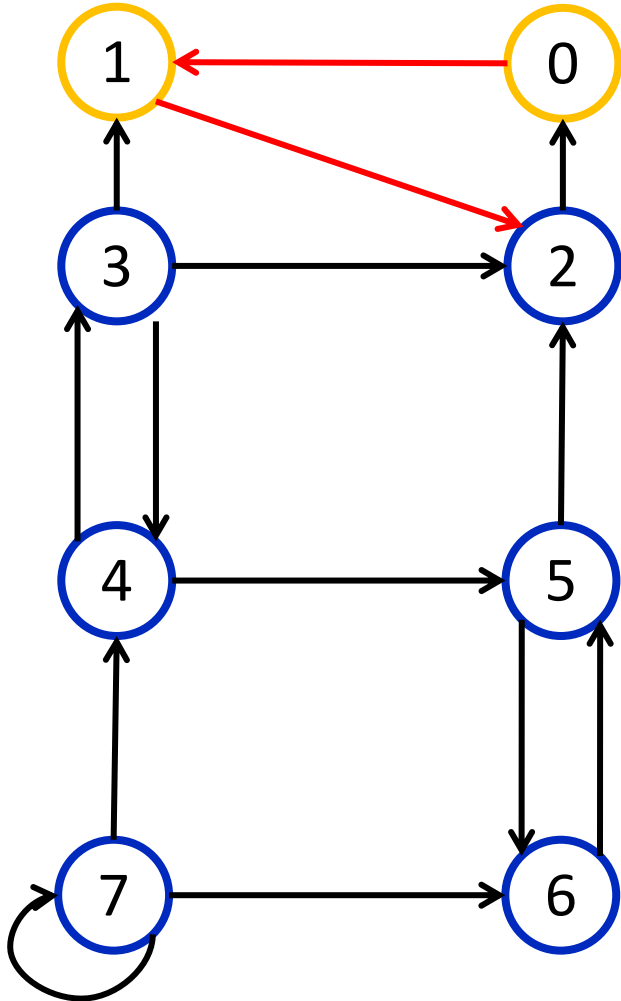


Ориентированный граф G^T

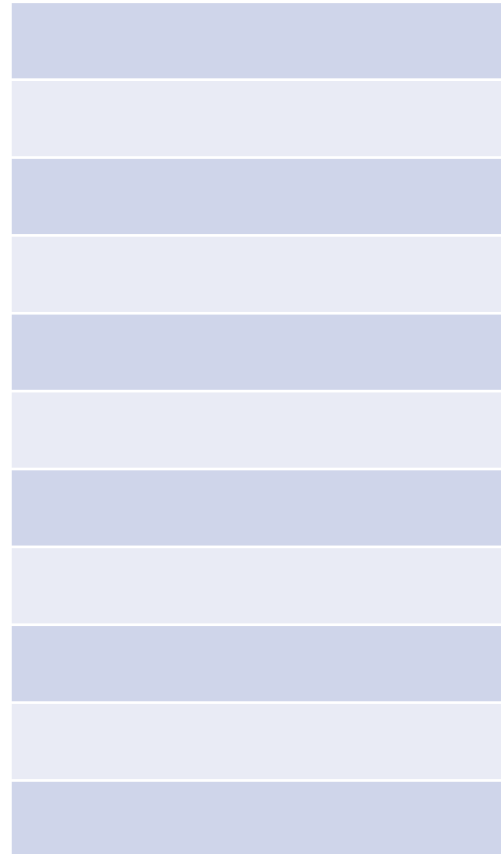


Алгоритм Косарайю поиска компонент сильной связности. Пример

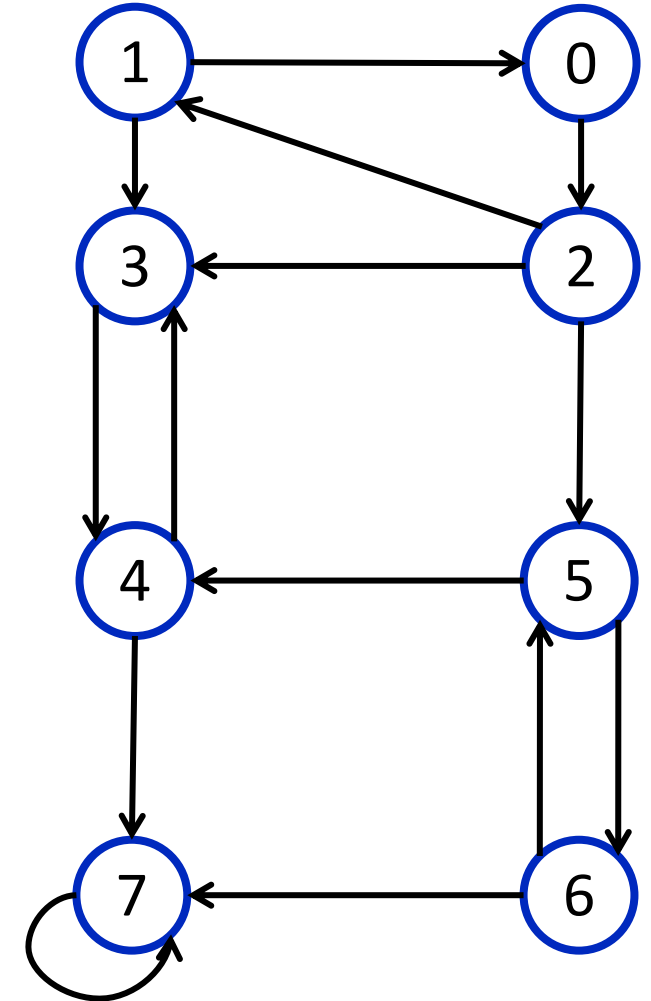
Ориентированный граф G



Список обхода

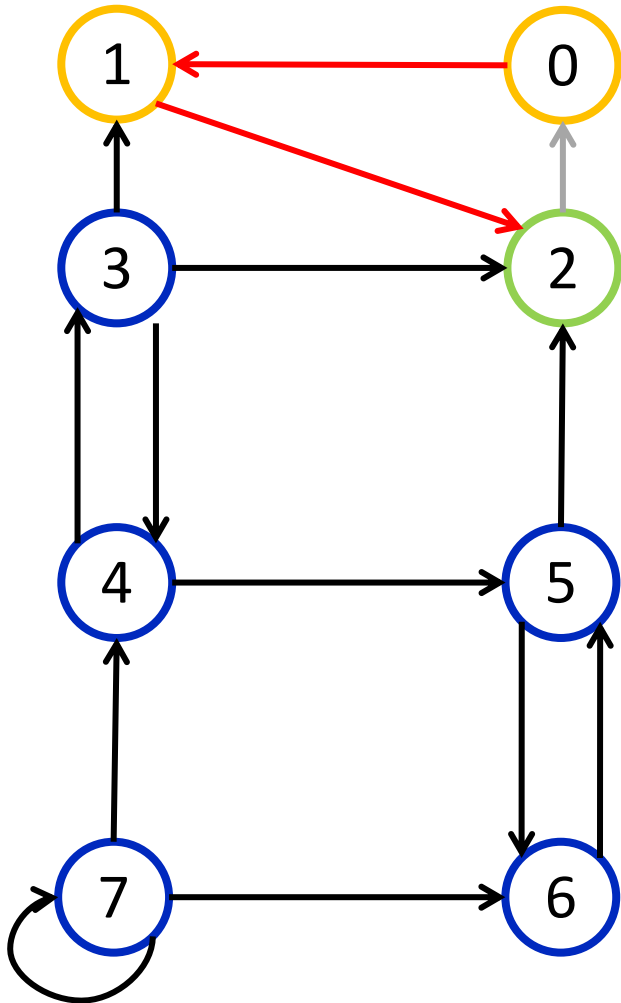


Ориентированный граф G^T

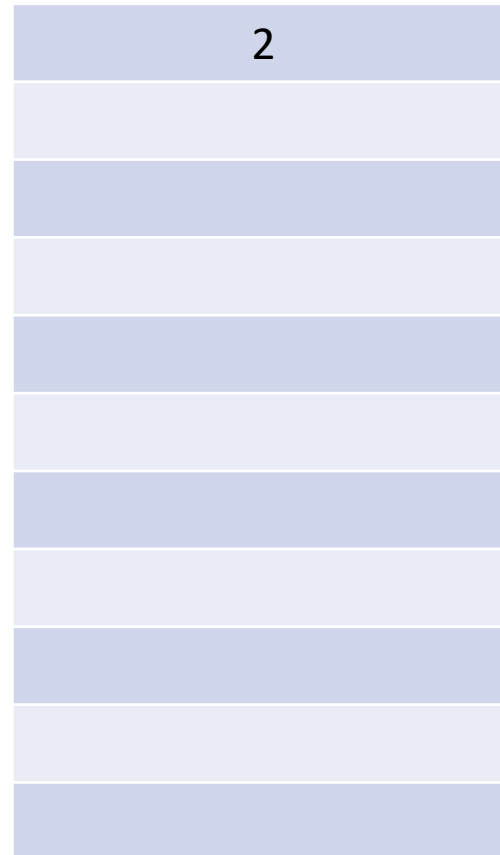


Алгоритм Косарайю поиска компонент сильной связности. Пример

Ориентированный граф G

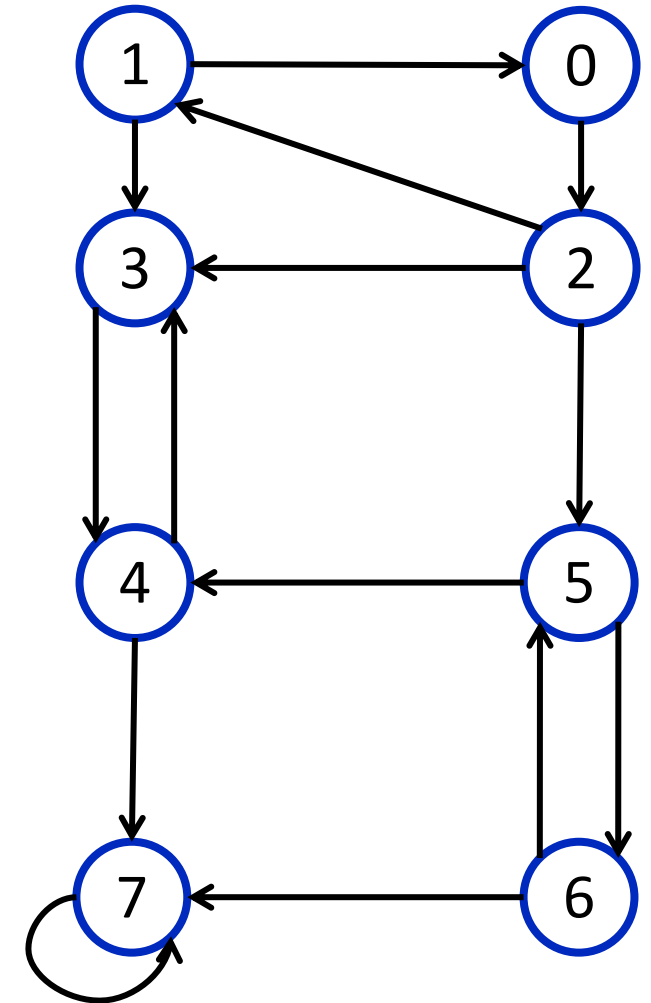


Список обхода



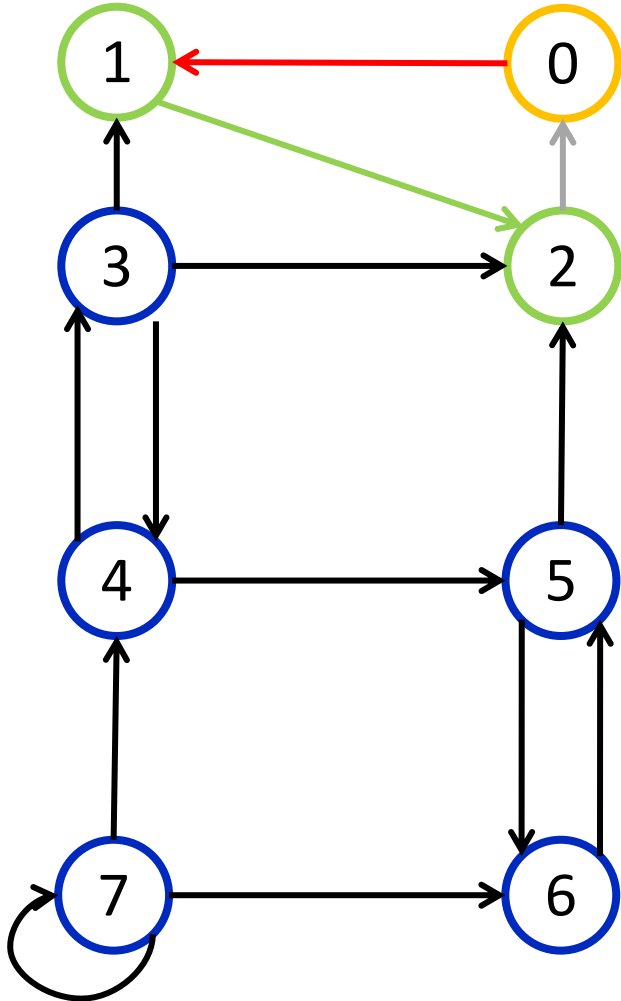
© Горденко М.К., 2022, CC BY

Ориентированный граф G^T

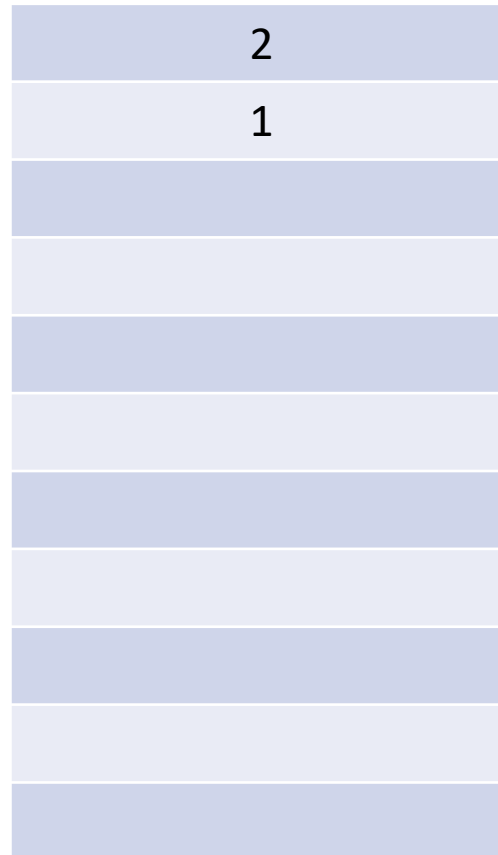


Алгоритм Косарайю поиска компонент сильной связности. Пример

Ориентированный граф G

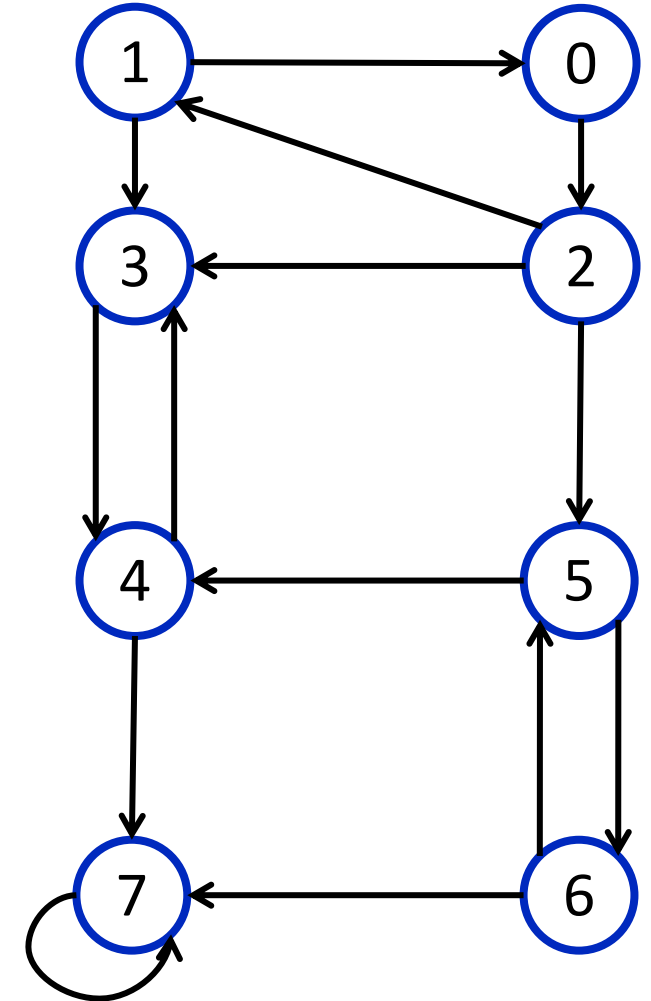


Список обхода



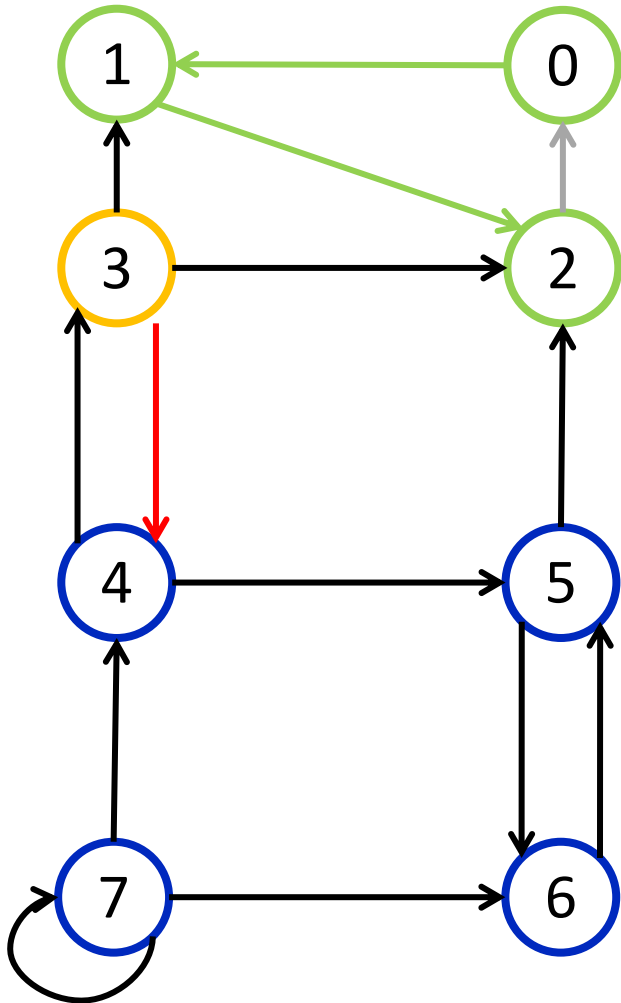
© Горденко М.К., 2022, CC BY

Ориентированный граф G^T



Алгоритм Косарайю поиска компонент сильной связности. Пример

Ориентированный граф G

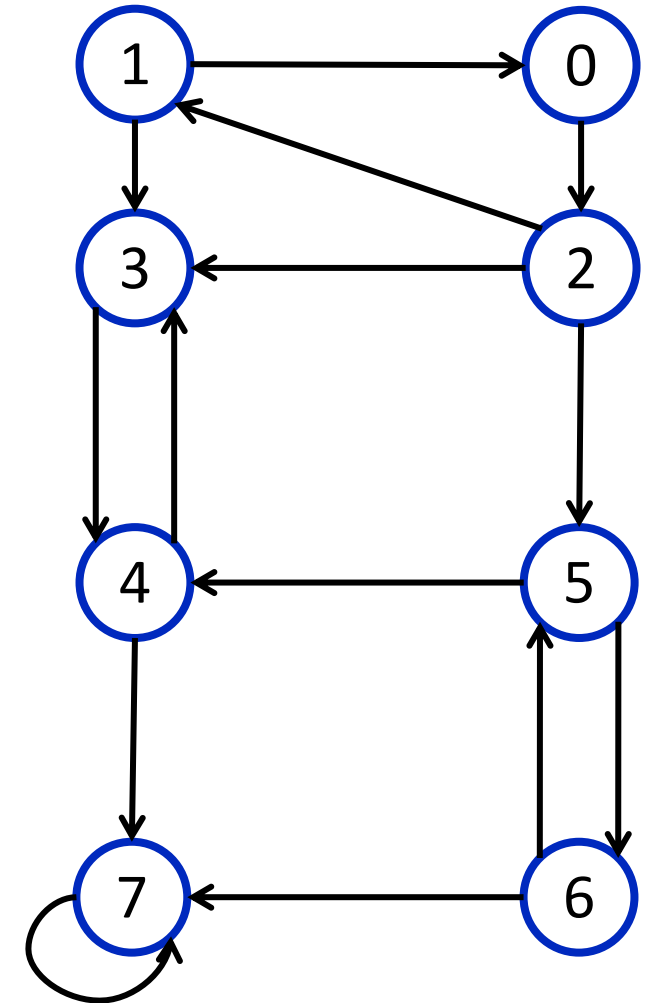


Список обхода

2
1
0

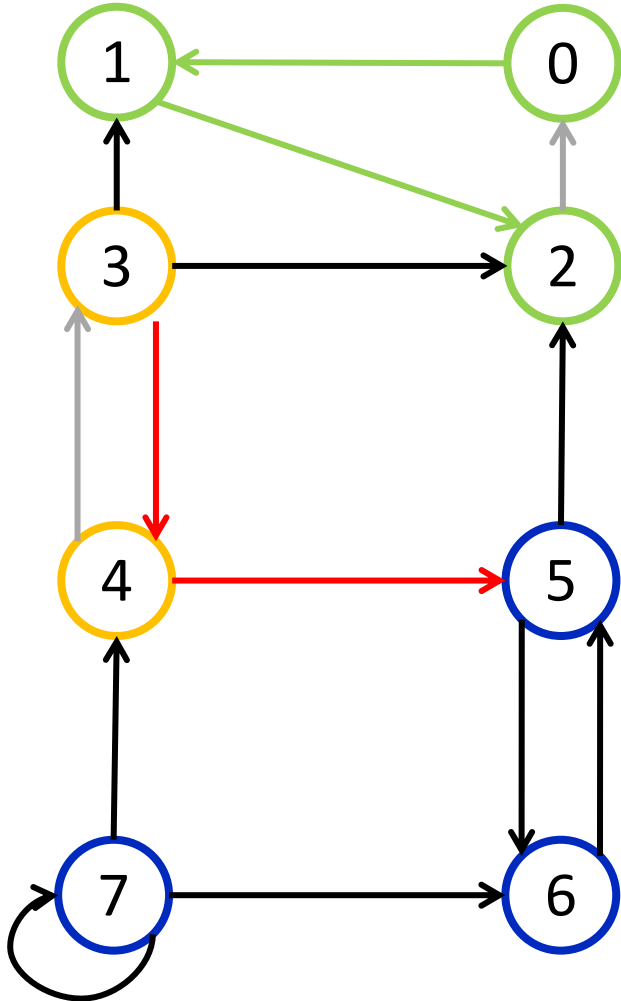
© Горденко М.К., 2022, CC BY

Ориентированный граф G^T



Алгоритм Косарайю поиска компонент сильной связности. Пример

Ориентированный граф G

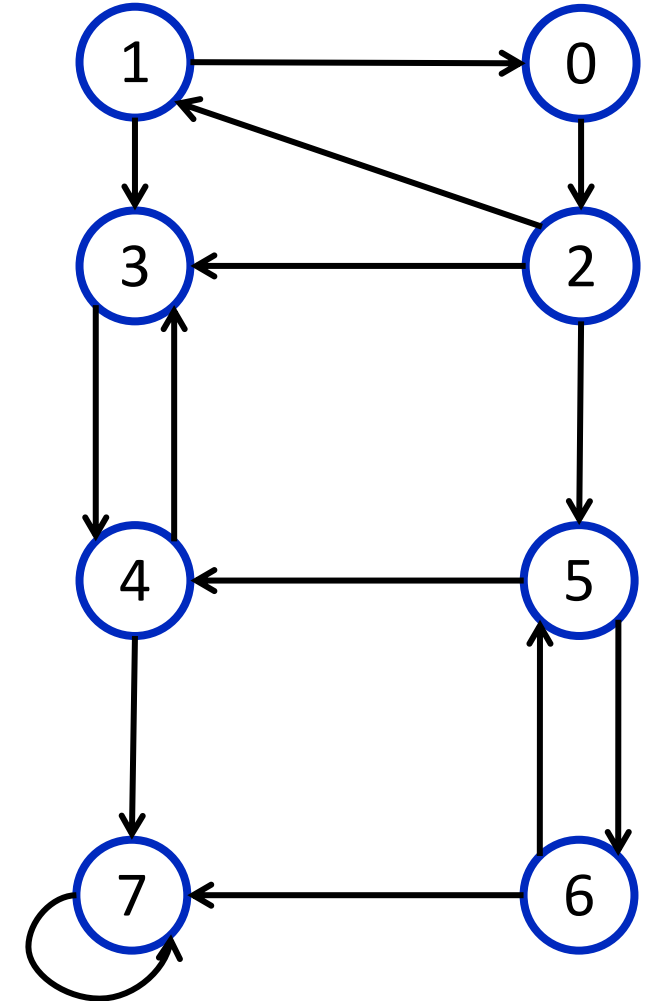


Список обхода

2
1
0

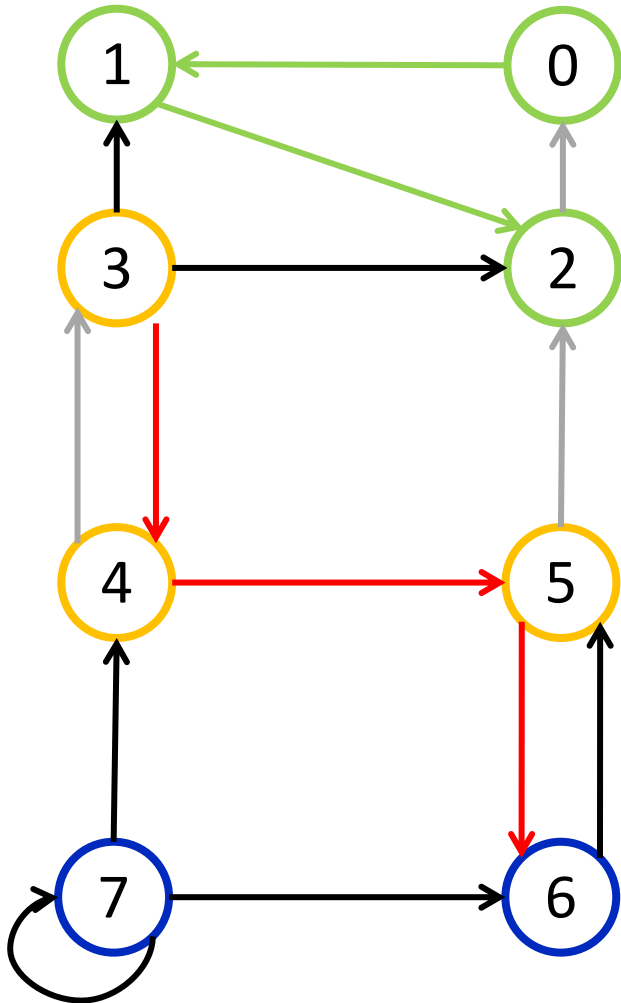
© Горденко М.К., 2022, CC BY

Ориентированный граф G^T



Алгоритм Косарайю поиска компонент сильной связности. Пример

Ориентированный граф G

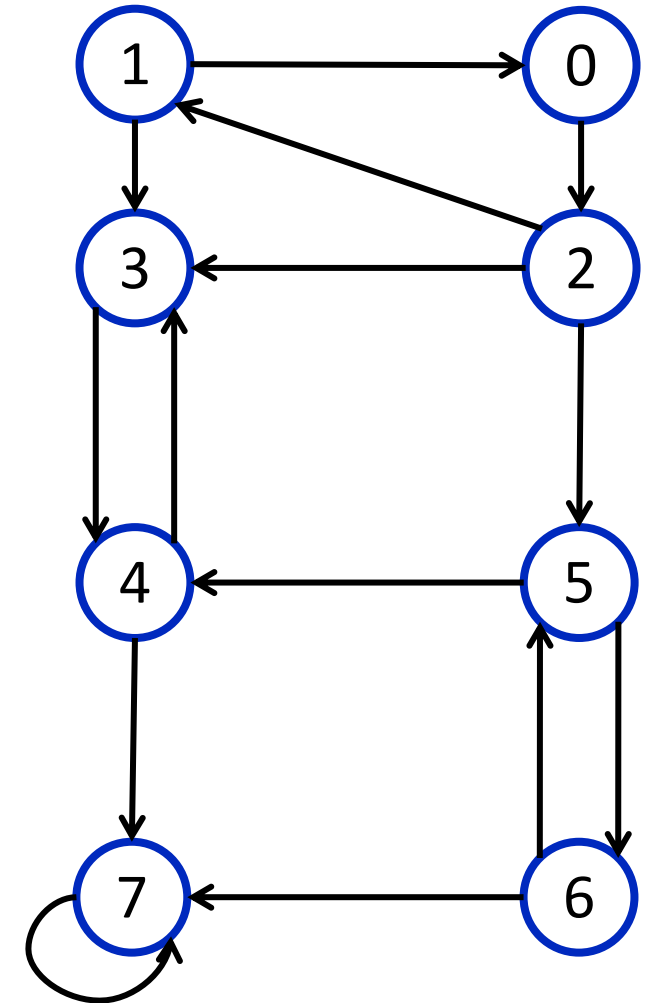


Список обхода

2
1
0

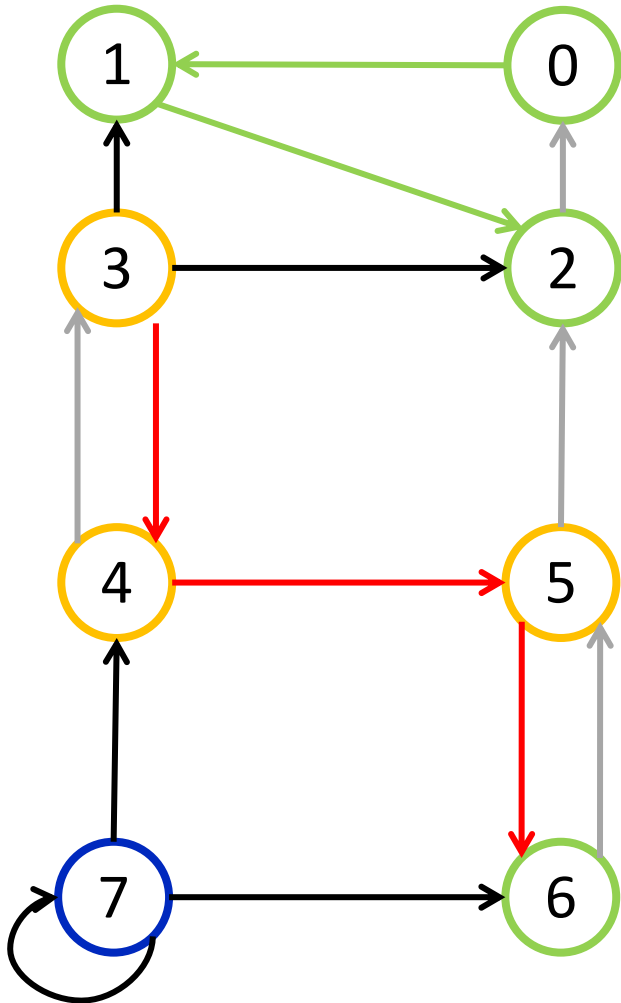
© Горденко М.К., 2022, CC BY

Ориентированный граф G^T



Алгоритм Косарайю поиска компонент сильной связности. Пример

Ориентированный граф G

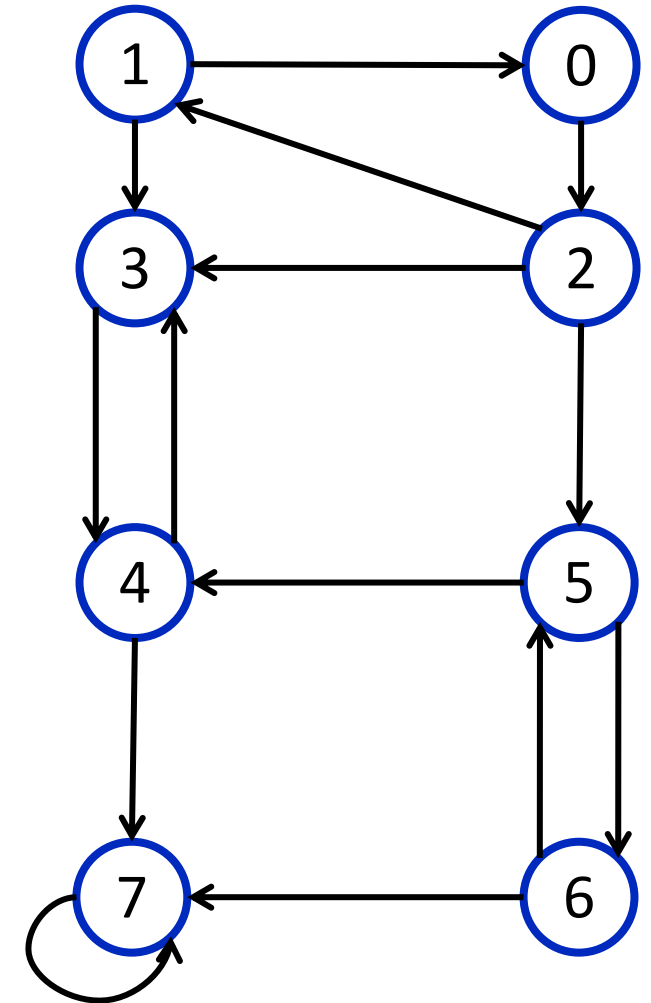


Список обхода

2
1
0
6

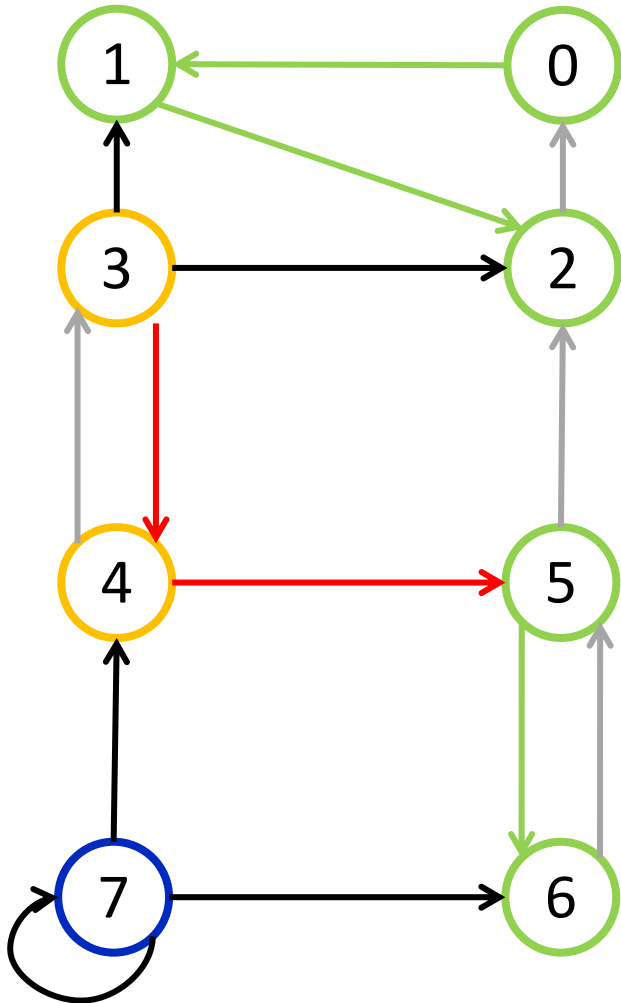
© Горденко М.К., 2022, CC BY

Ориентированный граф G^T



Алгоритм Косарайю поиска компонент сильной связности. Пример

Ориентированный граф G

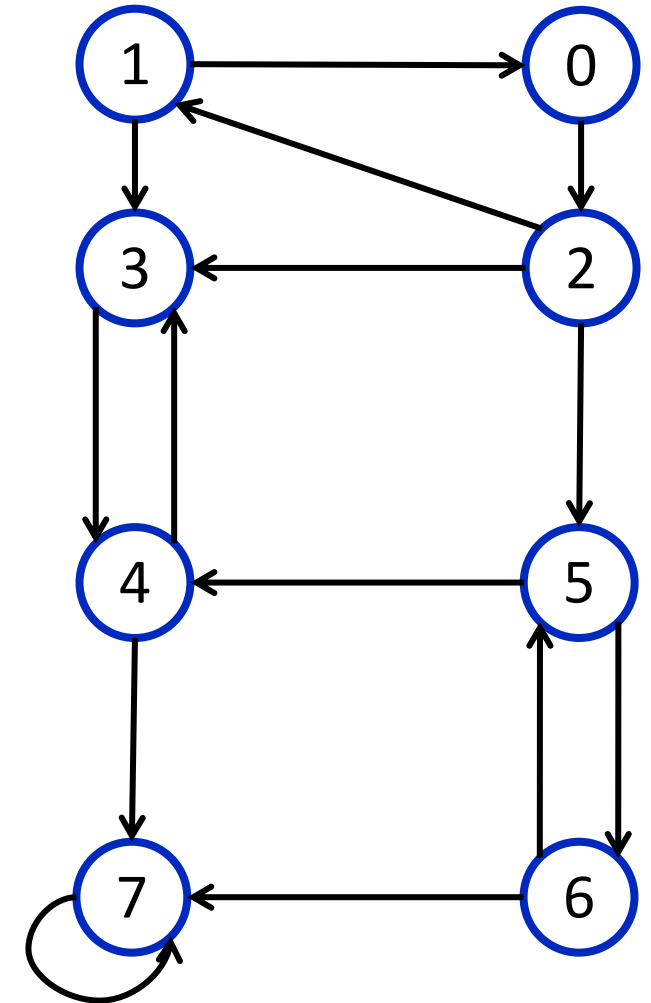


Список обхода

2
1
0
6
5

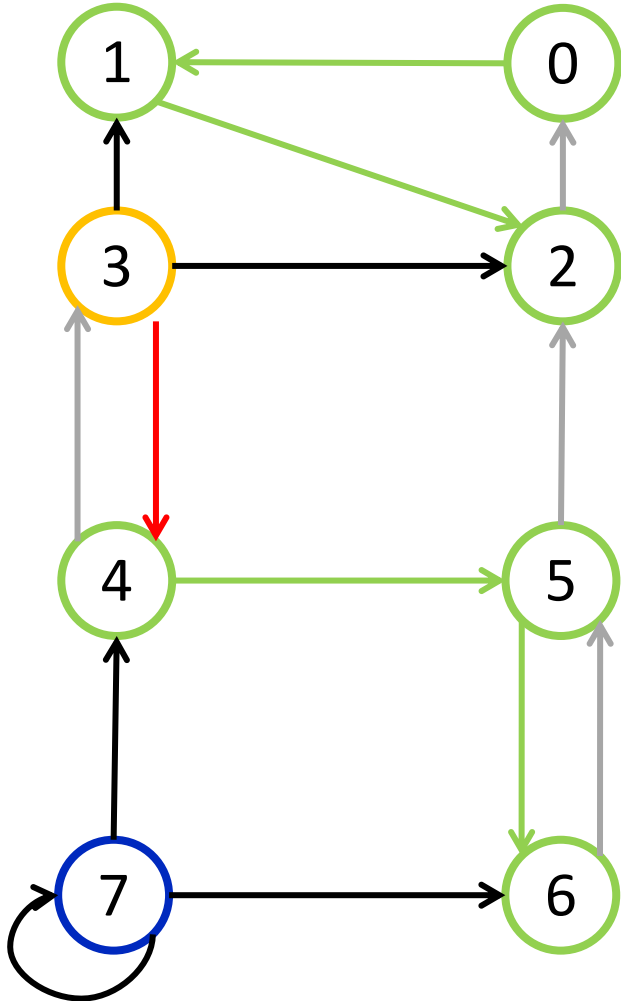
© Горденко М.К., 2022, CC BY

Ориентированный граф G^T



Алгоритм Косарайю поиска компонент сильной связности. Пример

Ориентированный граф G

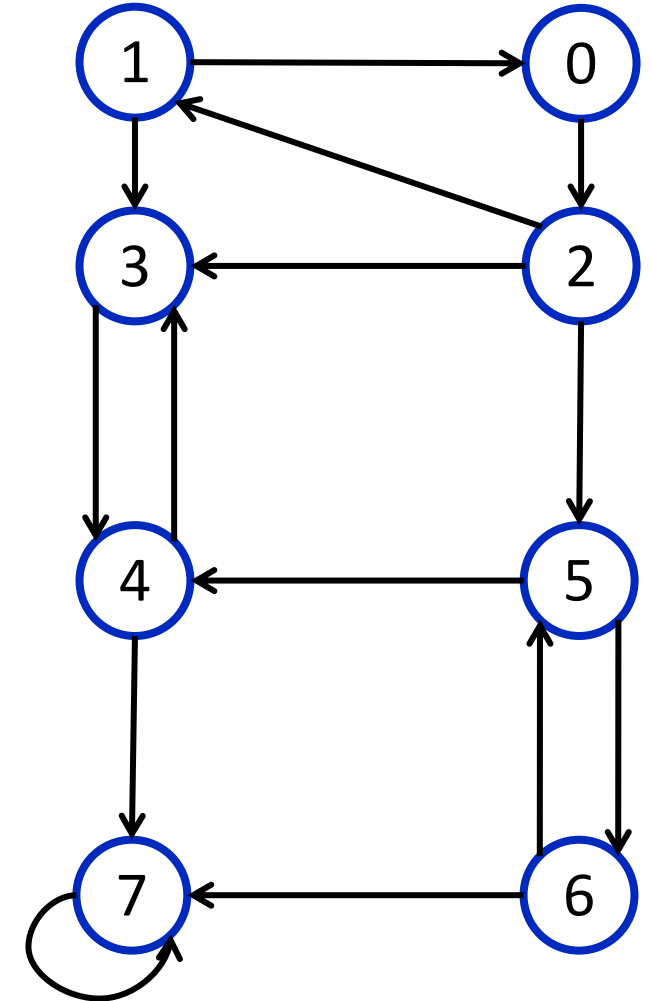


Список обхода

2
1
0
6
5
4

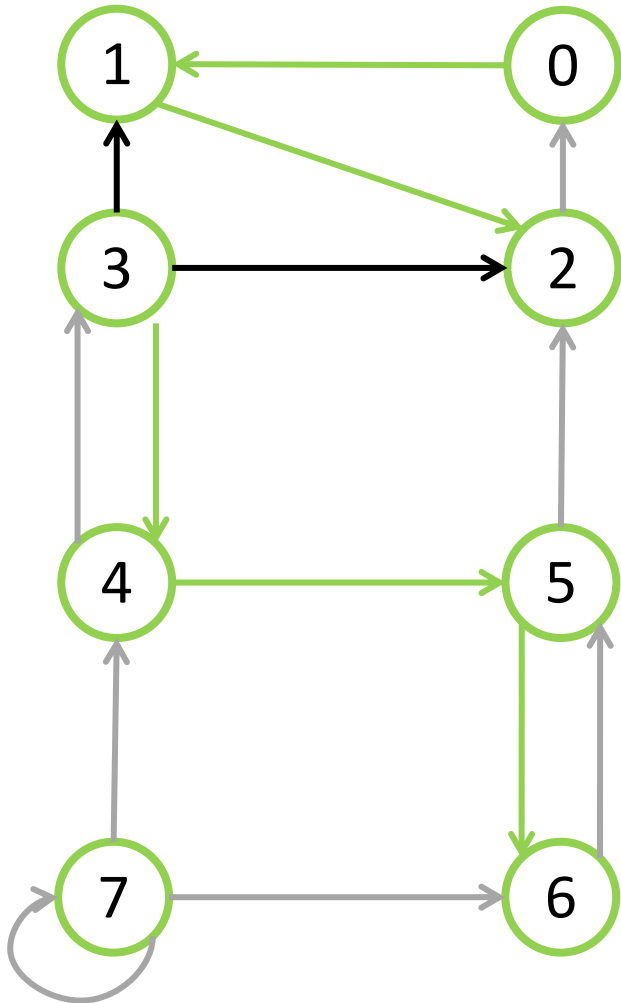
© Горденко М.К., 2022, CC BY

Ориентированный граф G^T



Алгоритм Косарайю поиска компонент сильной связности. Пример

Ориентированный граф G

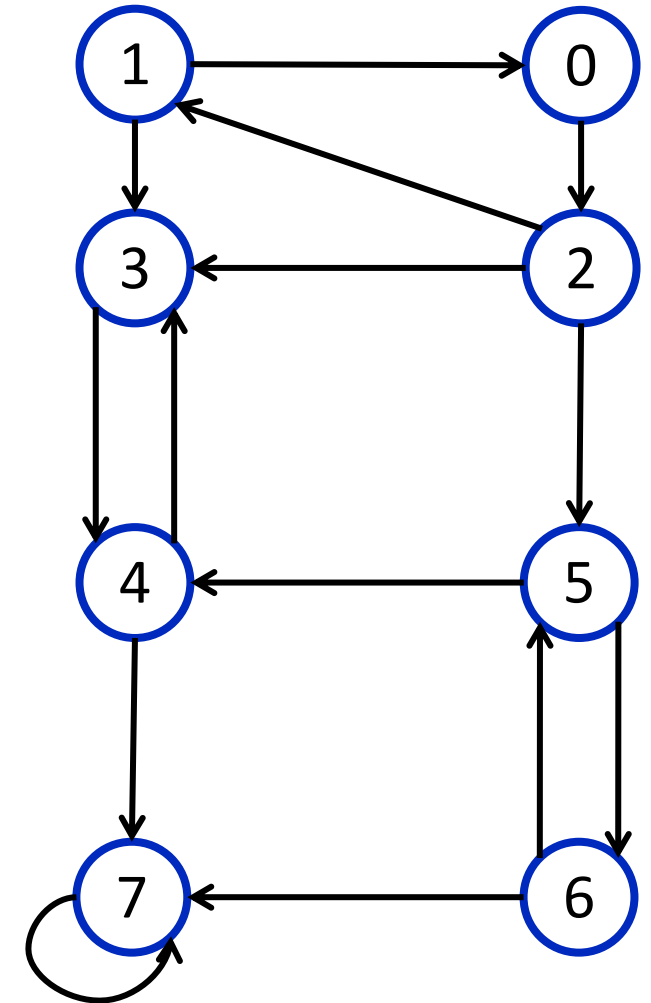


Список обхода

2
1
0
6
5
4
3
7

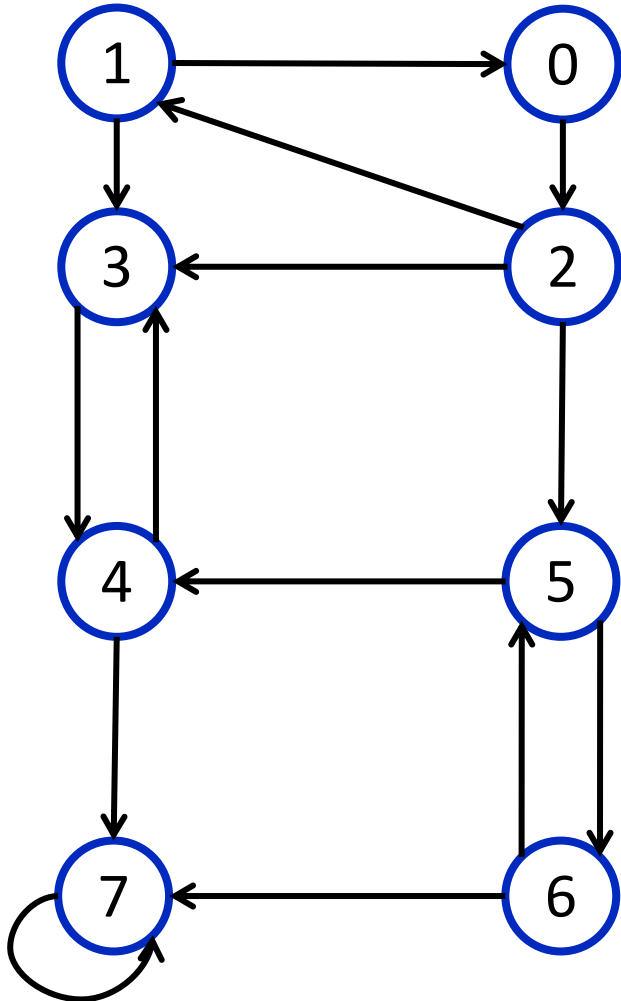
© Горденко М.К., 2022, CC BY

Ориентированный граф G^T



Алгоритм Косарайю поиска компонент сильной связности. Пример

Ориентированный граф G^T



Инв. список
обхода

7
3
4
5
6
0
1
2

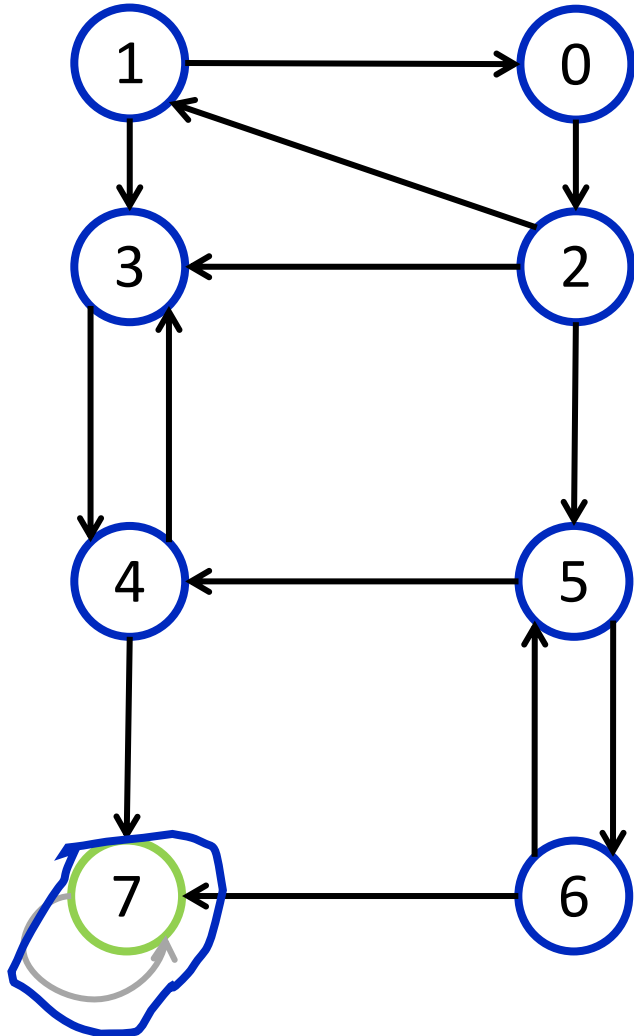
Функция обхода графа в глубину DFS

```
1. const int kMaxNum = ...;
2. vector<vector<int>> g(kMaxNum);
3. vector<vector<int>> gr(kMaxNum);
4. vector<int> order(kMaxNum);
5. vector<int> component(kMaxNum);
6. bool used[kMaxNum];

7. void dfs(int v, bool for_reversed) {
8.     used[v] = true;
9.     if (for_reversed) {
10.         component.push_back(v);
11.         for (auto &to : gr[v]) {
12.             if (!used[to])
13.                 dfs(to, for_reversed);
14.         }
15.     } else {
16.         for (auto &to : g[v]) {
17.             if (!used[to])
18.                 dfs(to, for_reversed);
19.         }
20.         order.push_back(v);
21.     }
22. }
```

Алгоритм Косарайю поиска компонент сильной связности. Пример

Ориентированный граф G^T



Инв. список
обхода

7
3
4
5
6
0
1
2

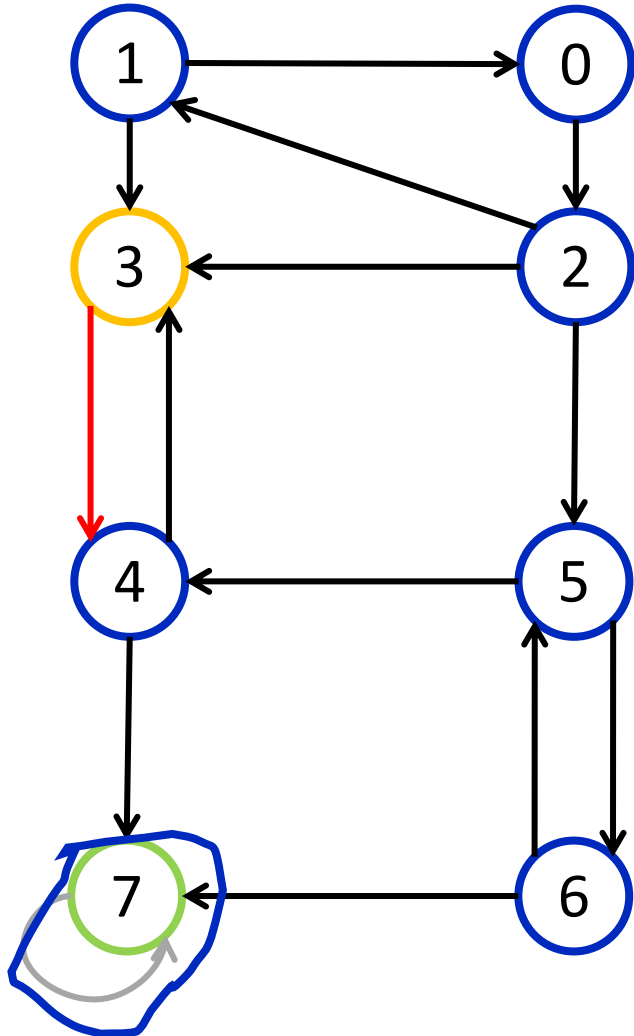
Функция обхода графа в глубину DFS

```
1. const int kMaxNum = ...;
2. vector<vector<int>> g(kMaxNum);
3. vector<vector<int>> gr(kMaxNum);
4. vector<int> order(kMaxNum);
5. vector<int> component(kMaxNum);
6. bool used[kMaxNum];

7. void dfs(int v, bool for_reversed) {
8.     used[v] = true;
9.     if (for_reversed) {
10.        component.push_back(v);
11.        for (auto &to : gr[v]) {
12.            if (!used[to])
13.                dfs(to, for_reversed);
14.        }
15.    } else {
16.        for (auto &to : g[v]) {
17.            if (!used[to])
18.                dfs(to, for_reversed);
19.        }
20.        order.push_back(v);
21.    }
22. }
```

Алгоритм Косарайю поиска компонент сильной связности. Пример

Ориентированный граф G^T



Инв. список
обхода

7
3
4
5
6
0
1
2

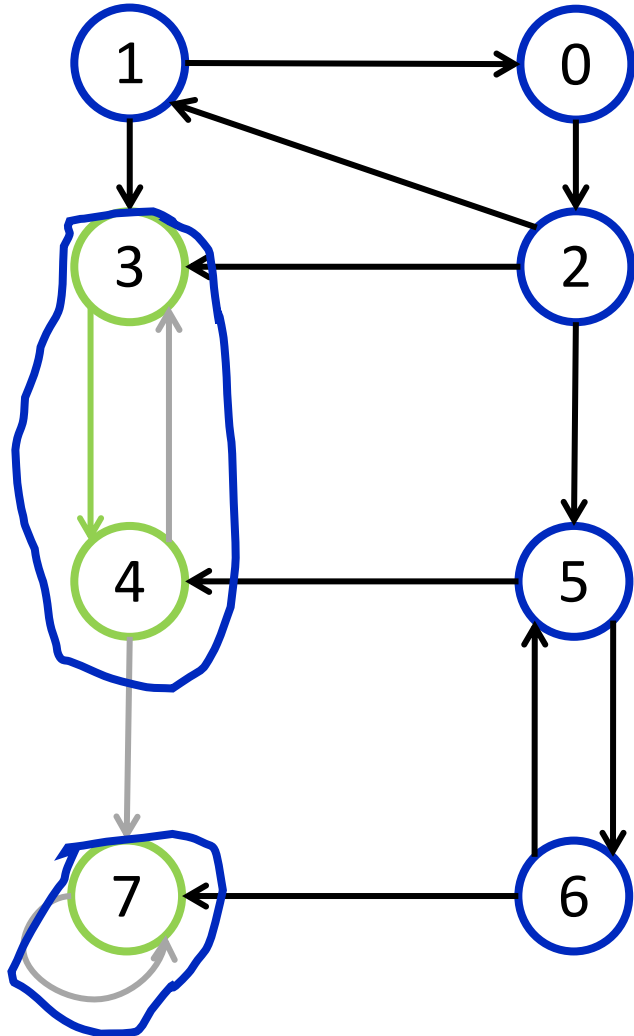
Функция обхода графа в глубину DFS

```
1. const int kMaxNum = ...;
2. vector<vector<int>> g(kMaxNum);
3. vector<vector<int>> gr(kMaxNum);
4. vector<int> order(kMaxNum);
5. vector<int> component(kMaxNum);
6. bool used[kMaxNum];

7. void dfs(int v, bool for_reversed) {
8.     used[v] = true;
9.     if (for_reversed) {
10.        component.push_back(v);
11.        for (auto &to : gr[v]) {
12.            if (!used[to])
13.                dfs(to, for_reversed);
14.        }
15.    } else {
16.        for (auto &to : g[v]) {
17.            if (!used[to])
18.                dfs(to, for_reversed);
19.        }
20.        order.push_back(v);
21.    }
22. }
```

Алгоритм Косарайю поиска компонент сильной связности. Пример

Ориентированный граф G^T



Инв. список
обхода

7
3
4
5
6
0
1
2

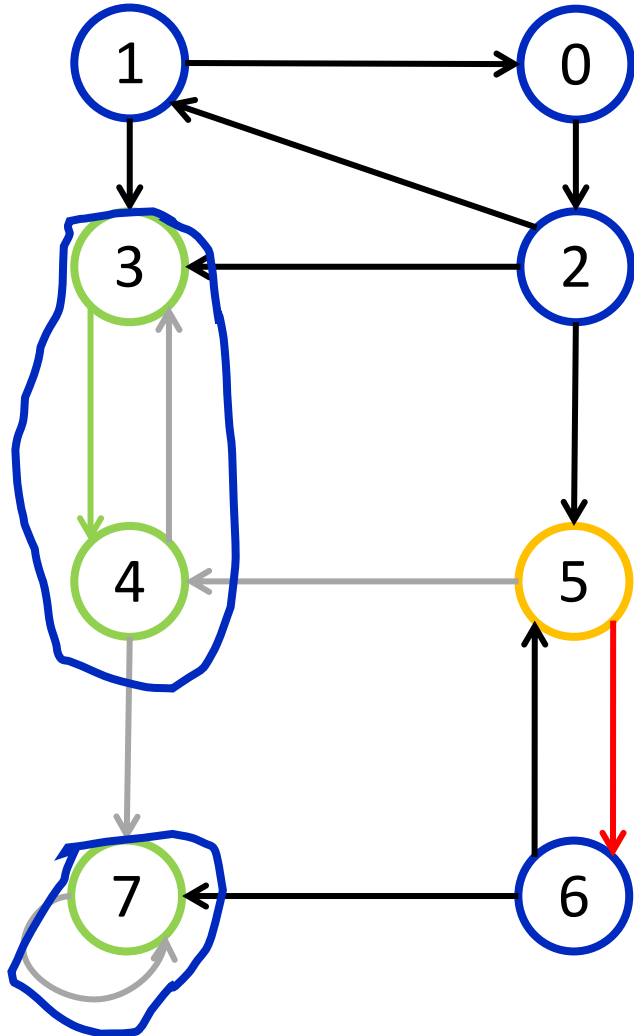
Функция обхода графа в глубину DFS

```
1. const int kMaxNum = ...;
2. vector<vector<int>> g(kMaxNum);
3. vector<vector<int>> gr(kMaxNum);
4. vector<int> order(kMaxNum);
5. vector<int> component(kMaxNum);
6. bool used[kMaxNum];

7. void dfs(int v, bool for_reversed) {
8.     used[v] = true;
9.     if (for_reversed) {
10.        component.push_back(v);
11.        for (auto &to : gr[v]) {
12.            if (!used[to])
13.                dfs(to, for_reversed);
14.        }
15.    } else {
16.        for (auto &to : g[v]) {
17.            if (!used[to])
18.                dfs(to, for_reversed);
19.        }
20.        order.push_back(v);
21.    }
22. }
```

Алгоритм Косарайю поиска компонент сильной связности. Пример

Ориентированный граф G^T



Инв. список
обхода

7
3
4
5
6
0
1
2

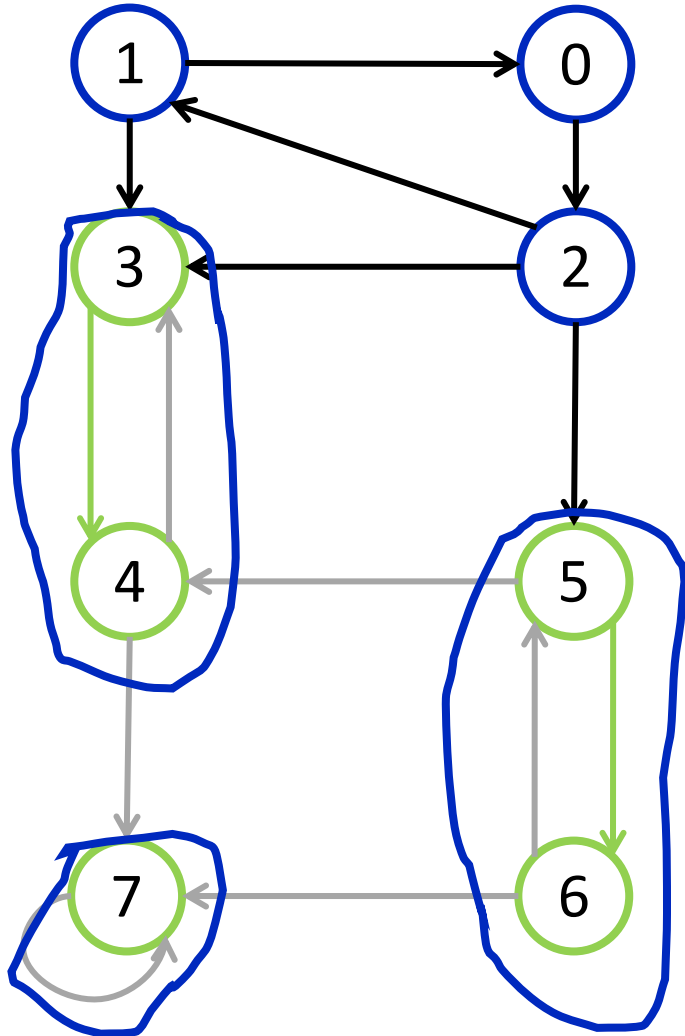
Функция обхода графа в глубину DFS

```
1. const int kMaxNum = ...;
2. vector<vector<int>> g(kMaxNum);
3. vector<vector<int>> gr(kMaxNum);
4. vector<int> order(kMaxNum);
5. vector<int> component(kMaxNum);
6. bool used[kMaxNum];

7. void dfs(int v, bool for_reversed) {
8.     used[v] = true;
9.     if (for_reversed) {
10.        component.push_back(v);
11.        for (auto &to : gr[v]) {
12.            if (!used[to])
13.                dfs(to, for_reversed);
14.        }
15.    } else {
16.        for (auto &to : g[v]) {
17.            if (!used[to])
18.                dfs(to, for_reversed);
19.        }
20.        order.push_back(v);
21.    }
22. }
```


Алгоритм Косарайю поиска компонент сильной связности. Пример

Ориентированный граф G^T



Инв. список
обхода

7
3
4
5
6
0
1
2

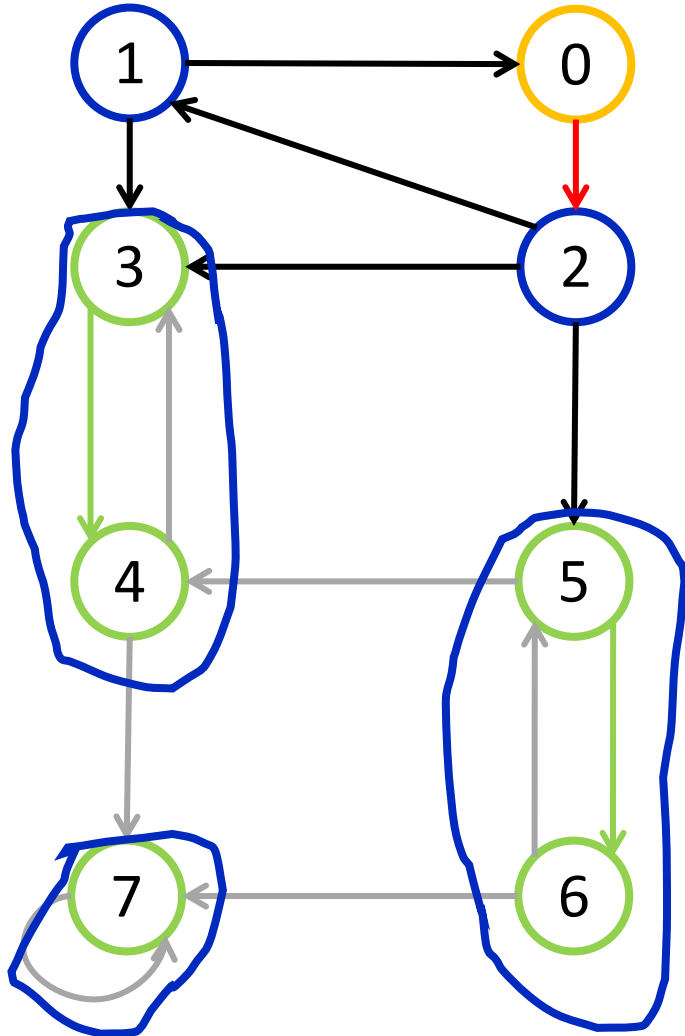
Функция обхода графа в глубину DFS

```
1. const int kMaxNum = ...;
2. vector<vector<int>> g(kMaxNum);
3. vector<vector<int>> gr(kMaxNum);
4. vector<int> order(kMaxNum);
5. vector<int> component(kMaxNum);
6. bool used[kMaxNum];

7. void dfs(int v, bool for_reversed) {
8.     used[v] = true;
9.     if (for_reversed) {
10.        component.push_back(v);
11.        for (auto &to : gr[v]) {
12.            if (!used[to])
13.                dfs(to, for_reversed);
14.        }
15.    } else {
16.        for (auto &to : g[v]) {
17.            if (!used[to])
18.                dfs(to, for_reversed);
19.        }
20.        order.push_back(v);
21.    }
22. }
```

Алгоритм Косарайю поиска компонент сильной связности. Пример

Ориентированный граф G^T



Инв. список
обхода

7
3
4
5
6
0
1
2

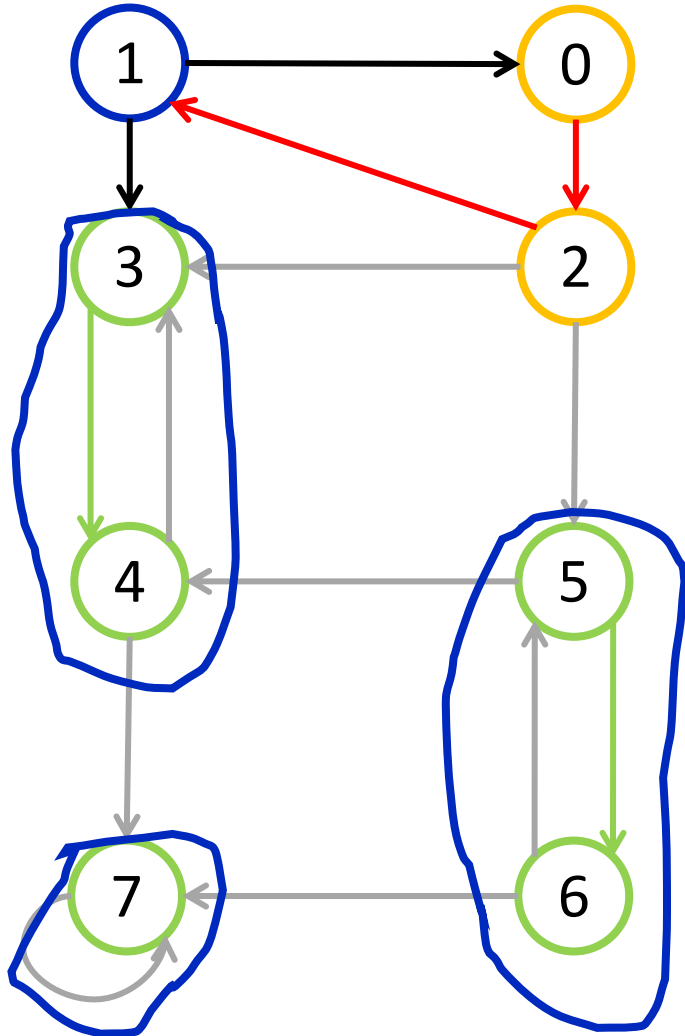
Функция обхода графа в глубину DFS

```
1. const int kMaxNum = ...;
2. vector<vector<int>> g(kMaxNum);
3. vector<vector<int>> gr(kMaxNum);
4. vector<int> order(kMaxNum);
5. vector<int> component(kMaxNum);
6. bool used[kMaxNum];

7. void dfs(int v, bool for_reversed) {
8.     used[v] = true;
9.     if (for_reversed) {
10.        component.push_back(v);
11.        for (auto &to : gr[v]) {
12.            if (!used[to])
13.                dfs(to, for_reversed);
14.        }
15.    } else {
16.        for (auto &to : g[v]) {
17.            if (!used[to])
18.                dfs(to, for_reversed);
19.        }
20.        order.push_back(v);
21.    }
22. }
```

Алгоритм Косарайю поиска компонент сильной связности. Пример

Ориентированный граф G^T



Инв. список
обхода

7
3
4
5
6
0
1
2

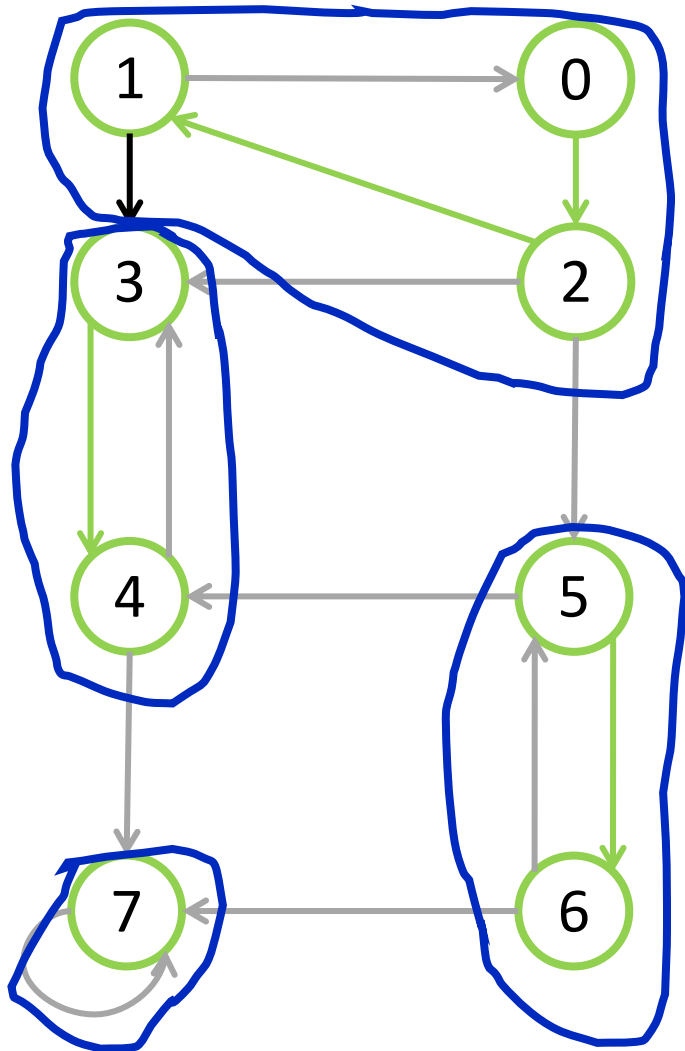
Функция обхода графа в глубину DFS

```
1. const int kMaxNum = ...;
2. vector<vector<int>> g(kMaxNum);
3. vector<vector<int>> gr(kMaxNum);
4. vector<int> order(kMaxNum);
5. vector<int> component(kMaxNum);
6. bool used[kMaxNum];

7. void dfs(int v, bool for_reversed) {
8.     used[v] = true;
9.     if (for_reversed) {
10.        component.push_back(v);
11.        for (auto &to : gr[v]) {
12.            if (!used[to])
13.                dfs(to, for_reversed);
14.        }
15.    } else {
16.        for (auto &to : g[v]) {
17.            if (!used[to])
18.                dfs(to, for_reversed);
19.        }
20.        order.push_back(v);
21.    }
22. }
```

Алгоритм Косарайю поиска компонент сильной связности. Пример

Ориентированный граф G^T



Инв. список
обхода

	7
	3
	4
	5
	6
	0
	1
	2

Функция обхода графа в глубину DFS

```
1. const int kMaxNum = ...;
2. vector<vector<int>> g(kMaxNum);
3. vector<vector<int>> gr(kMaxNum);
4. vector<int> order(kMaxNum);
5. vector<int> component(kMaxNum);
6. bool used[kMaxNum];

7. void dfs(int v, bool for_reversed) {
8.     used[v] = true;
9.     if (for_reversed) {
10.        component.push_back(v);
11.        for (auto &to : gr[v]) {
12.            if (!used[to])
13.                dfs(to, for_reversed);
14.        }
15.    } else {
16.        for (auto &to : g[v]) {
17.            if (!used[to])
18.                dfs(to, for_reversed);
19.        }
20.        order.push_back(v);
21.    }
22. }
```

Алгоритм Тарьяна поиска компонент сильной связности

1. Выполняется обход в глубину и каждая новая рассматриваемая вершина заносится на стек и для нее определяется временная метка начала рассмотрения вершины
2. Если для какой-либо рассматриваемой вершины найдется дуга, ведущее в уже находящуюся на стеке вершину, тогда во вспомогательный контейнер в ячейку для рассматриваемой вершины записывается минимальная из временных меток рассматриваемой вершины и той, что находится на стеке
3. Как только все выходящие из рассматриваемой вершины дуги будут изучены, проверяется равенство исходной временной метки вершины ее текущему значению (так как при анализе дуг это значение могло уменьшиться) – если значения равны, следовательно, рассматриваемая вершина будет «корневой» для новой КСС, и все вершины с эквивалентными минимальными временными метками снимаются со стека и помещаются в новую КСС

Алгоритм Тарьяна поиска компонент сильной связности

- Имеет ту же временную сложность, что и алгоритм Косарайю - $O(|V| + |E|)$,
однако емкостная сложность ограничивается $O(N)$ с чуть большим
константным множителем
- Требуется небольших изменений в алгоритме обхода в глубину
- Использует только один обход в глубину
- Не требует транспонирования (обращения) графа

Алгоритм Тарьяна поиска компонент сильной связности. Алгоритм

- `vector<int> tin` – контейнер исходных временных меток вершин
- `vector<int> trn` – контейнер минимальных временных меток вершин (назовем их «числами Тарьяна»)
- `stack<int> component` – стек рассматриваемых вершин
- `unordered_set<int> used_in_cmp` – множество находящихся на стеке (для проверки за константное время)

```
1.  const int kMaxNum = ...;
2.  vector<vector<int>> g(kMaxNum);
3.  vector<int> tin(kMaxNum);
4.  vector<int> trn(kMaxNum);
5.  stack<int> component;
6.  unordered_set<int> used_in_cmp;
7.  bool used[kMaxNum];
8.  int timer = 0;

9.  void dfs(int v) {...}

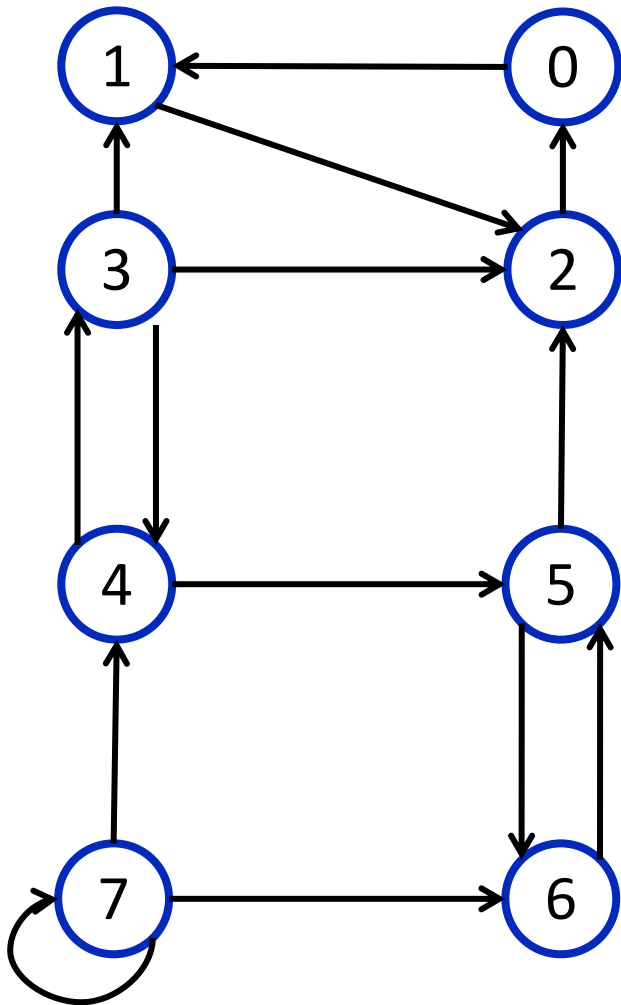
10. int main() {
11.     int from;
12.     int to;
13.     for (int i = 0; i < kMaxNum; ++i) {
14.         // Считывание ребра (a, b).
15.         g[a].push_back(b);
16.         used[i] = false;
17.     }

18.     for (int i = 0; i < kMaxNum; ++i)
19.         if (!used[i])
20.             dfs(i);
21. }
```

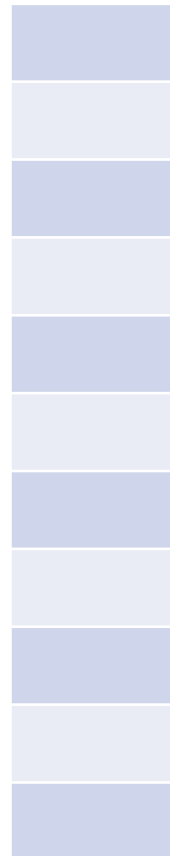
```
1.  void dfs(int v) {
2.      used[v] = true;
3.      trn[v] = (tin[v] = timer++);
4.      component.insert(v);
5.      for (auto &to : gr[v]) {
6.          if (!used[to]) {
7.              dfs(to);
8.              trn[v] = min(trn[v], trn[to]);
9.          } else if (used_in_cmp.find(to) != used_in_cmp.end()) {
10.             trn[v] = min(trn[v], tin[to]);
11.          }
12.      }
13.      if (tin[v] == trn[v]) {
14.          // Создание новой КСС.
15.          int curr;
16.          do {
17.              curr = component.top();
18.              // Добавление «curr» в новую КСС.
19.              component.pop();
20.              used_in_cmp.erase(curr);
21.          } while (trn[v] == trn[component.top()]);
22.          // Вывод полученной КСС.
23.      }
24. }
```

Алгоритм Тарьяна поиска компонент сильной связности. Пример

Ориентированный граф G



Стек

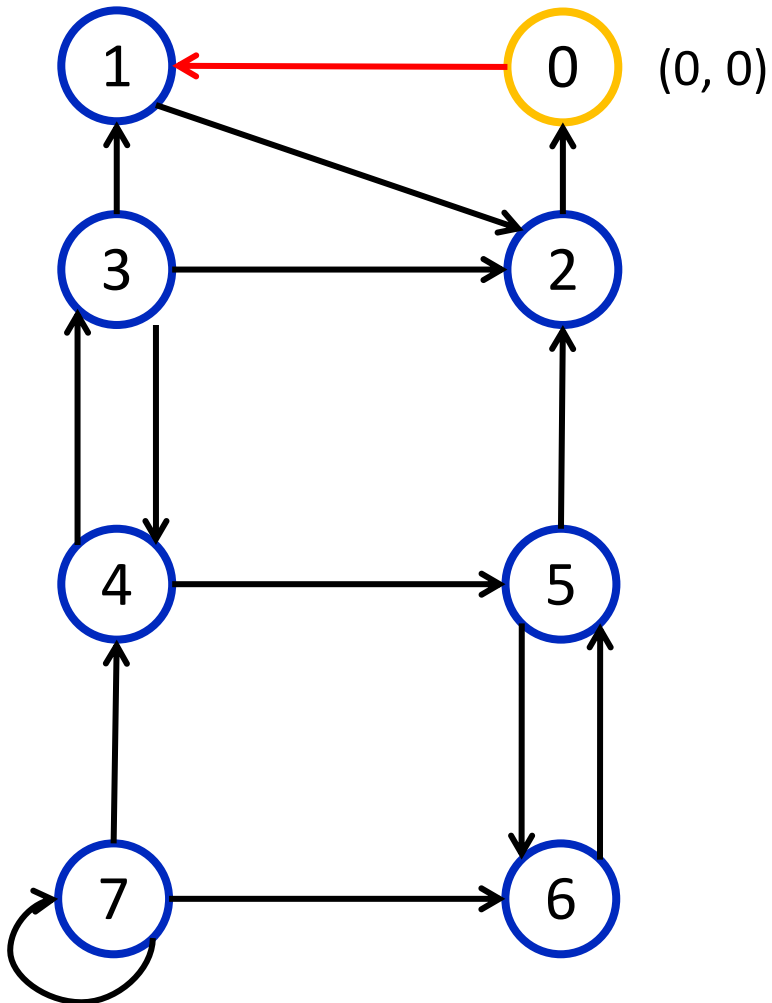


Функция обхода графа в глубину DFS

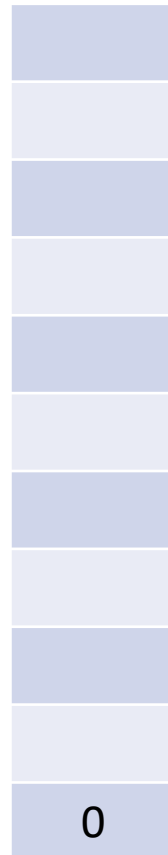
```
1. void dfs(int v) {
2.     used[v] = true;
3.     trn[v] = (tin[v] = timer++);
4.     component.insert(v);
5.     for (auto &to : gr[v]) {
6.         if (!used[to]) {
7.             dfs(to);
8.             trn[v] = min(trn[v], trn[to]);
9.         } else if (used_in_cmp.find(to) != used_in_cmp.end()) {
10.            trn[v] = min(trn[v], tin[to]);
11.        }
12.    }
13.    if (tin[v] == trn[v]) {
14.        // Создание новой KCC.
15.        int curr;
16.        do {
17.            curr = component.top();
18.            // Добавление «curr» в новую KCC.
19.            component.pop();
20.            used_in_cmp.erase(curr);
21.        } while (trn[v] == trn[component.top()]);
22.        // Вывод полученной KCC.
23.    }
24. }
```


Алгоритм Тарьяна поиска компонент сильной связности. Пример

Ориентированный граф G



Стек

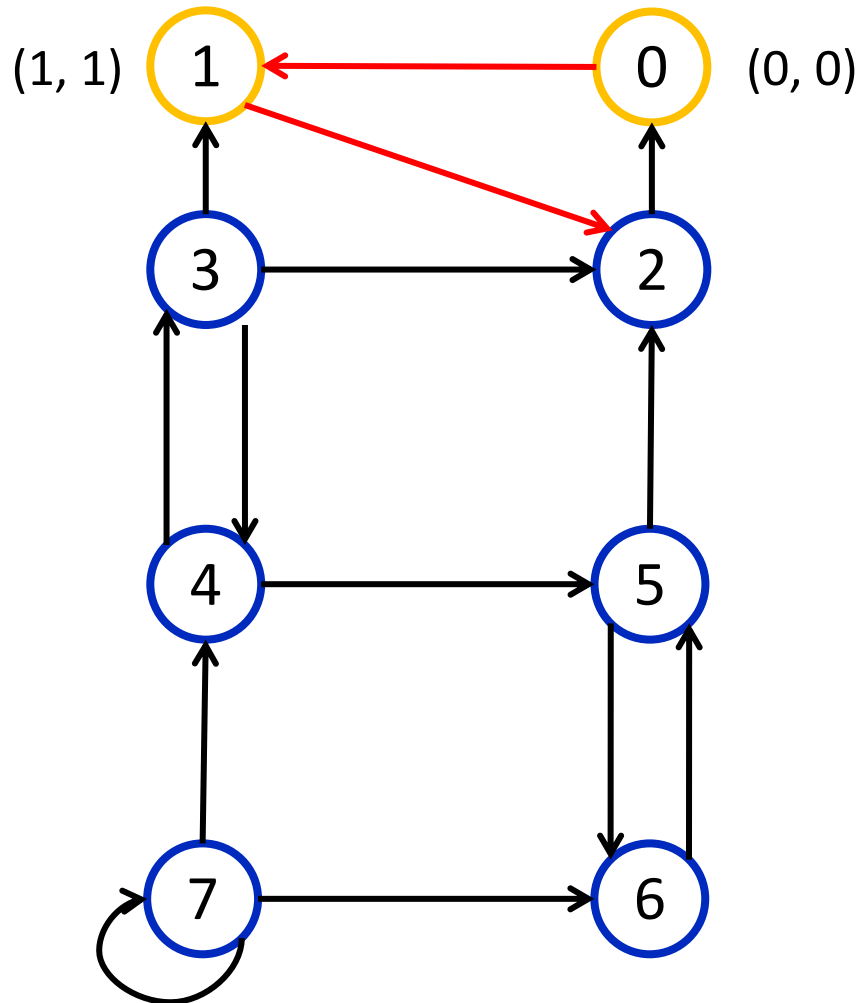


Функция обхода графа в глубину DFS

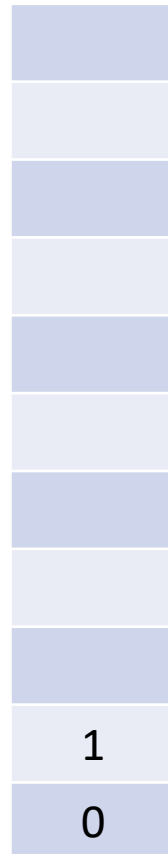
```
1. void dfs(int v) {
2.   used[v] = true;
3.   trn[v] = (tin[v] = timer++);
4.   component.insert(v);
5.   for (auto &to : gr[v]) {
6.     if (!used[to]) {
7.       dfs(to);
8.       trn[v] = min(trn[v], trn[to]);
9.     } else if (used_in_cmp.find(to) != used_in_cmp.end()) {
10.      trn[v] = min(trn[v], tin[to]);
11.    }
12.  }
13.  if (tin[v] == trn[v]) {
14.    // Создание новой KCC.
15.    int curr;
16.    do {
17.      curr = component.top();
18.      // Добавление «curr» в новую KCC.
19.      component.pop();
20.      used_in_cmp.erase(curr);
21.    } while (trn[v] == trn[component.top()]);
22.    // Вывод полученной KCC.
23.  }
24. }
```

Алгоритм Тарьяна поиска компонент сильной связности. Пример

Ориентированный граф G



Стек

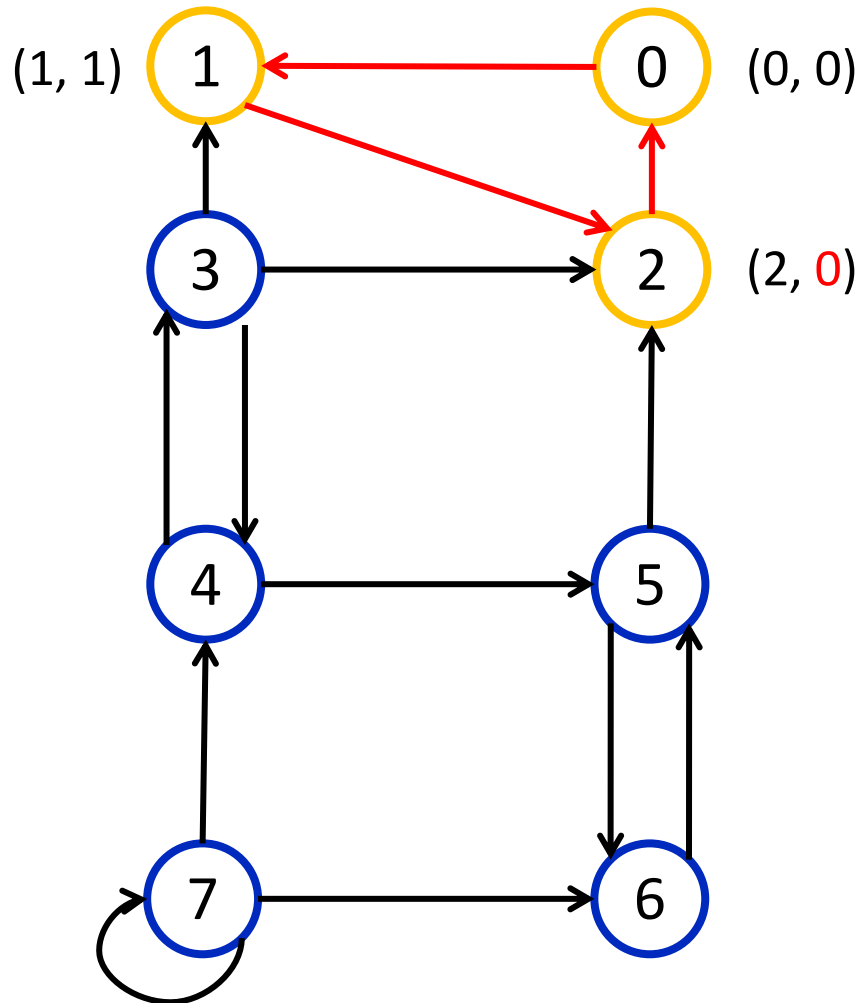


Функция обхода графа в глубину DFS

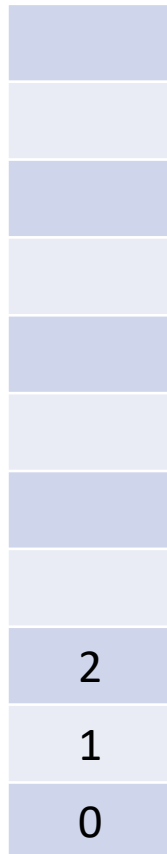
```
1. void dfs(int v) {
2.     used[v] = true;
3.     trn[v] = (tin[v] = timer++);
4.     component.insert(v);
5.     for (auto &to : gr[v]) {
6.         if (!used[to]) {
7.             dfs(to);
8.             trn[v] = min(trn[v], trn[to]);
9.         } else if (used_in_cmp.find(to) != used_in_cmp.end()) {
10.            trn[v] = min(trn[v], tin[to]);
11.        }
12.    }
13.    if (tin[v] == trn[v]) {
14.        // Создание новой KCC.
15.        int curr;
16.        do {
17.            curr = component.top();
18.            // Добавление «curr» в новую KCC.
19.            component.pop();
20.            used_in_cmp.erase(curr);
21.        } while (trn[v] == trn[component.top()]);
22.        // Вывод полученной KCC.
23.    }
24. }
```

Алгоритм Тарьяна поиска компонент сильной связности. Пример

Ориентированный граф G



Стек

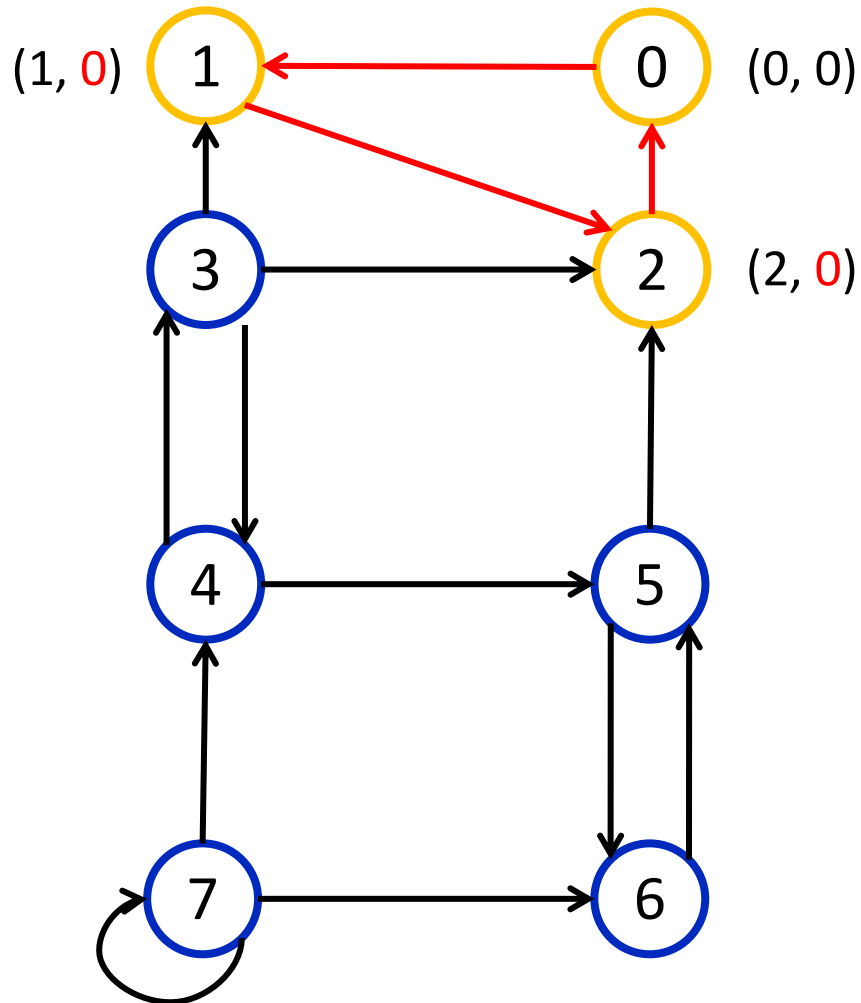


Функция обхода графа в глубину DFS

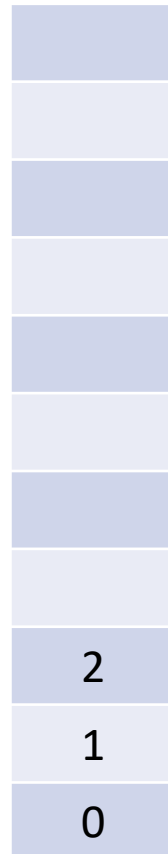
```
1. void dfs(int v) {
2.     used[v] = true;
3.     trn[v] = (tin[v] = timer++);
4.     component.insert(v);
5.     for (auto &to : gr[v]) {
6.         if (!used[to]) {
7.             dfs(to);
8.             trn[v] = min(trn[v], trn[to]);
9.         } else if (used_in_cmp.find(to) != used_in_cmp.end()) {
10.            trn[v] = min(trn[v], tin[to]);
11.        }
12.    }
13.    if (tin[v] == trn[v]) {
14.        // Создание новой KCC.
15.        int curr;
16.        do {
17.            curr = component.top();
18.            // Добавление «curr» в новую KCC.
19.            component.pop();
20.            used_in_cmp.erase(curr);
21.        } while (trn[v] == trn[component.top()]);
22.        // Вывод полученной KCC.
23.    }
24. }
```

Алгоритм Тарьяна поиска компонент сильной связности. Пример

Ориентированный граф G



Стек

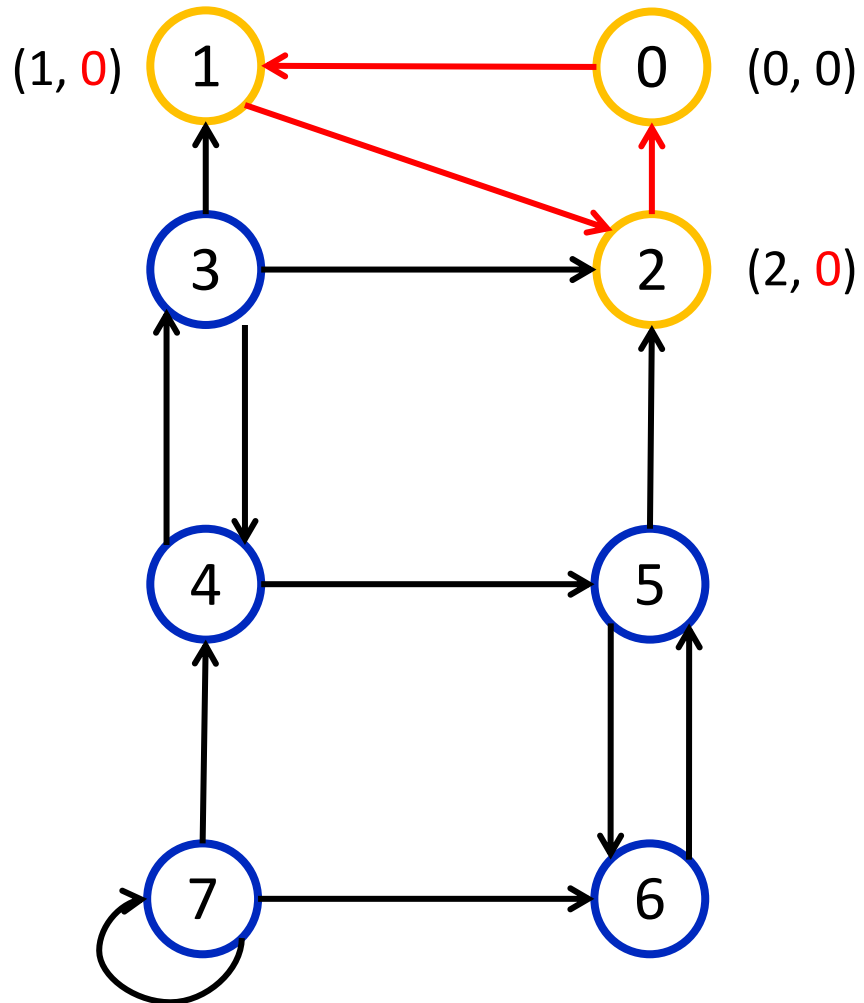


Функция обхода графа в глубину DFS

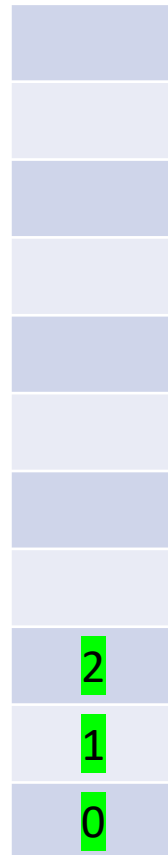
```
1. void dfs(int v) {
2.     used[v] = true;
3.     trn[v] = (tin[v] = timer++);
4.     component.insert(v);
5.     for (auto &to : gr[v]) {
6.         if (!used[to]) {
7.             dfs(to);
8.             trn[v] = min(trn[v], trn[to]);
9.         } else if (used_in_cmp.find(to) != used_in_cmp.end()) {
10.            trn[v] = min(trn[v], tin[to]);
11.        }
12.    }
13.    if (tin[v] == trn[v]) {
14.        // Создание новой KCC.
15.        int curr;
16.        do {
17.            curr = component.top();
18.            // Добавление «curr» в новую KCC.
19.            component.pop();
20.            used_in_cmp.erase(curr);
21.        } while (trn[v] == trn[component.top()]);
22.        // Вывод полученной KCC.
23.    }
24. }
```

Алгоритм Тарьяна поиска компонент сильной связности. Пример

Ориентированный граф G



Стек

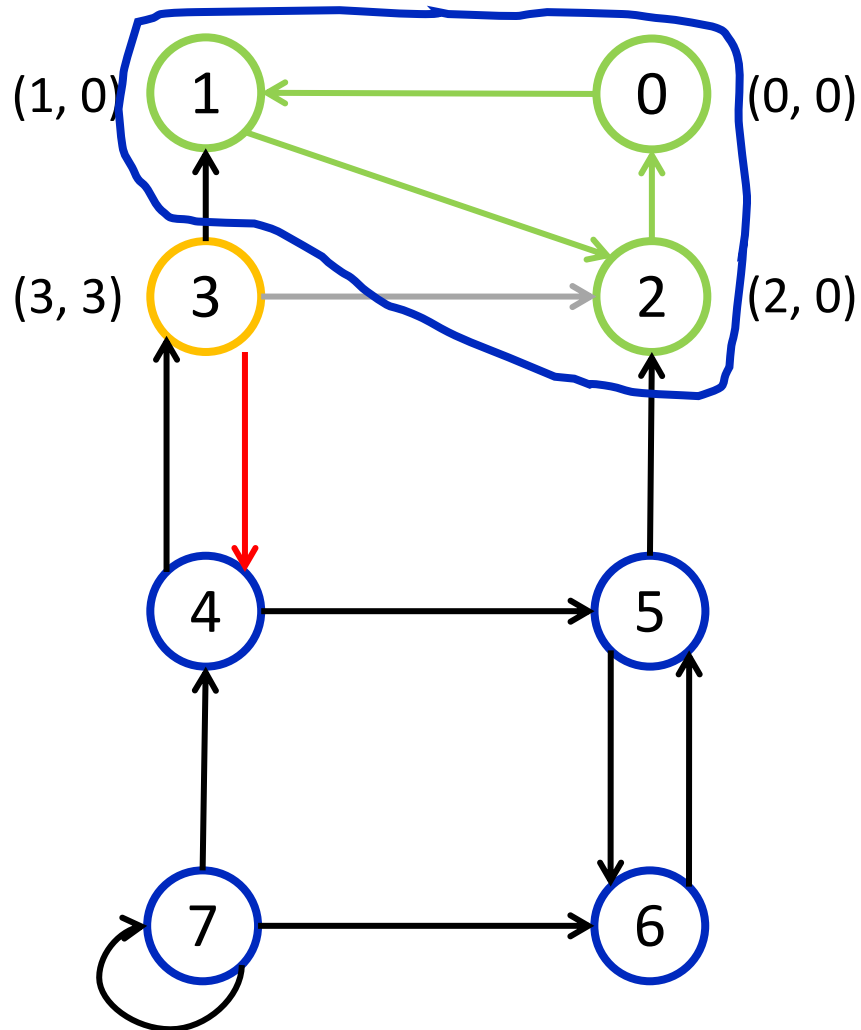


Функция обхода графа в глубину DFS

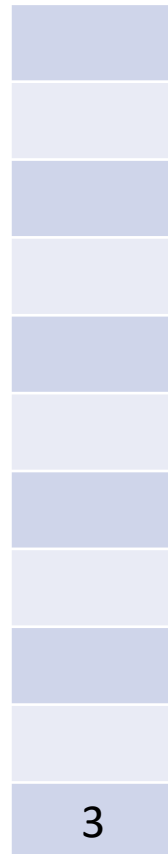
```
1. void dfs(int v) {
2.     used[v] = true;
3.     trn[v] = (tin[v] = timer++);
4.     component.insert(v);
5.     for (auto &to : gr[v]) {
6.         if (!used[to]) {
7.             dfs(to);
8.             trn[v] = min(trn[v], trn[to]);
9.         } else if (used_in_cmp.find(to) != used_in_cmp.end()) {
10.            trn[v] = min(trn[v], tin[to]);
11.        }
12.    }
13.    if (tin[v] == trn[v]) {
14.        // Создание новой KCC.
15.        int curr;
16.        do {
17.            curr = component.top();
18.            // Добавление «curr» в новую KCC.
19.            component.pop();
20.            used_in_cmp.erase(curr);
21.        } while (trn[v] == trn[component.top()]);
22.        // Вывод полученной KCC.
23.    }
24. }
```

Алгоритм Тарьяна поиска компонент сильной связности. Пример

Ориентированный граф G



Стек

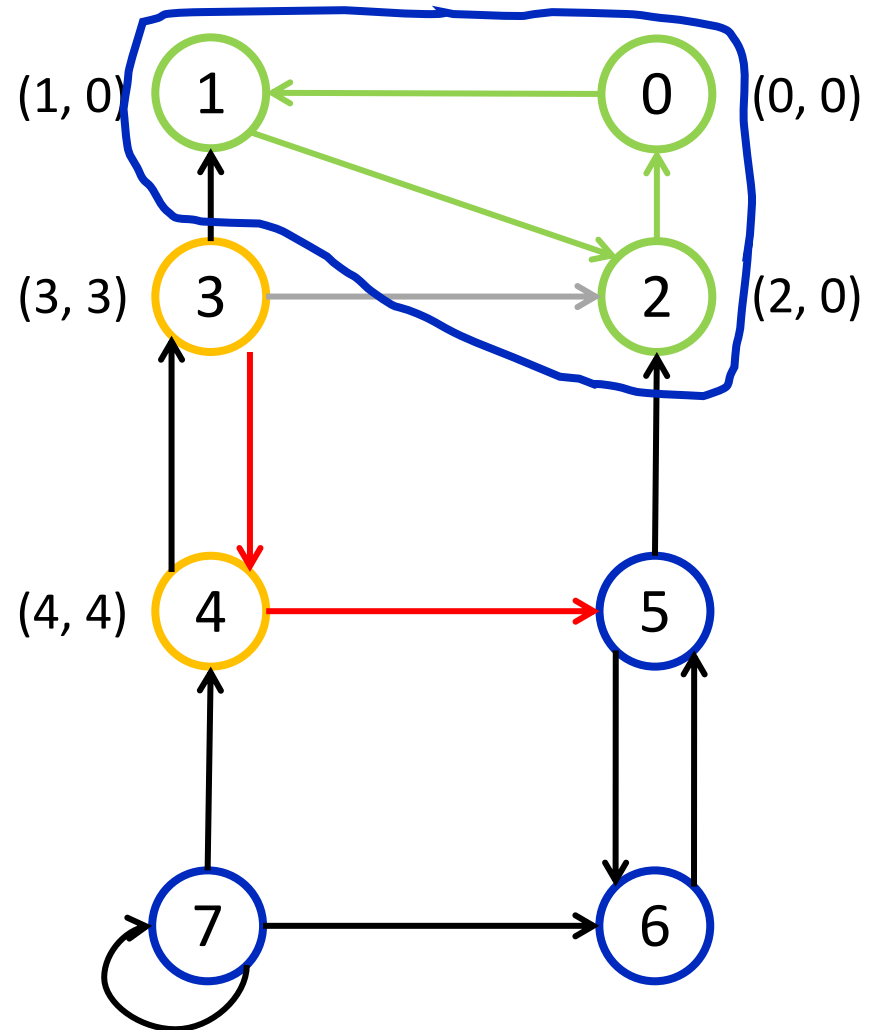


Функция обхода графа в глубину DFS

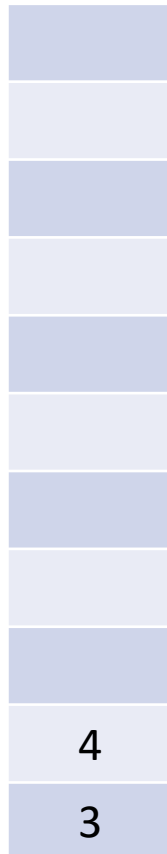
```
1. void dfs(int v) {
2.     used[v] = true;
3.     trn[v] = (tin[v] = timer++);
4.     component.insert(v);
5.     for (auto &to : gr[v]) {
6.         if (!used[to]) {
7.             dfs(to);
8.             trn[v] = min(trn[v], trn[to]);
9.         } else if (used_in_cmp.find(to) != used_in_cmp.end()) {
10.            trn[v] = min(trn[v], tin[to]);
11.        }
12.    }
13.    if (tin[v] == trn[v]) {
14.        // Создание новой KCC.
15.        int curr;
16.        do {
17.            curr = component.top();
18.            // Добавление «curr» в новую KCC.
19.            component.pop();
20.            used_in_cmp.erase(curr);
21.        } while (trn[v] == trn[component.top()]);
22.        // Вывод полученной KCC.
23.    }
24. }
```

Алгоритм Тарьяна поиска компонент сильной связности. Пример

Ориентированный граф G



Стек

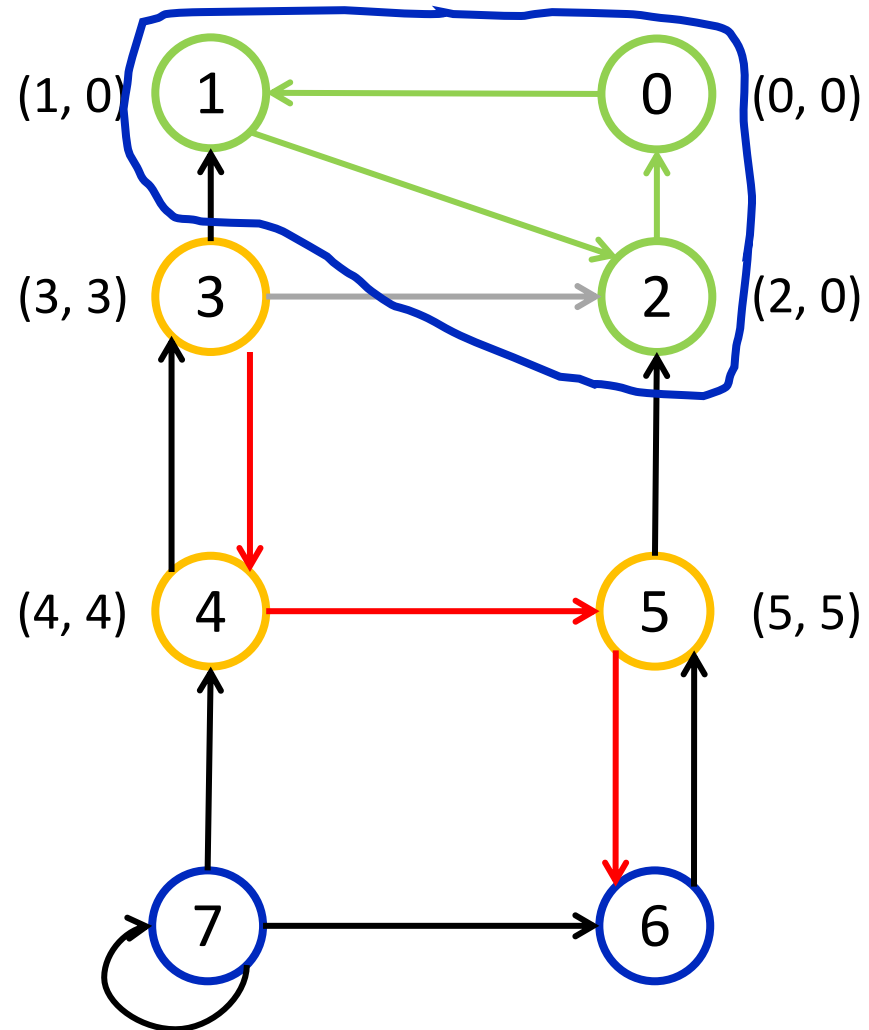


Функция обхода графа в глубину DFS

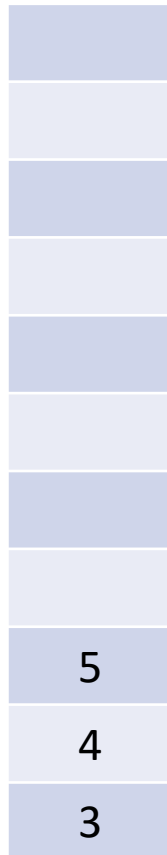
```
1. void dfs(int v) {
2.     used[v] = true;
3.     trn[v] = (tin[v] = timer++);
4.     component.insert(v);
5.     for (auto &to : gr[v]) {
6.         if (!used[to]) {
7.             dfs(to);
8.             trn[v] = min(trn[v], trn[to]);
9.         } else if (used_in_cmp.find(to) != used_in_cmp.end()) {
10.            trn[v] = min(trn[v], tin[to]);
11.        }
12.    }
13.    if (tin[v] == trn[v]) {
14.        // Создание новой KCC.
15.        int curr;
16.        do {
17.            curr = component.top();
18.            // Добавление «curr» в новую KCC.
19.            component.pop();
20.            used_in_cmp.erase(curr);
21.        } while (trn[v] == trn[component.top()]);
22.        // Вывод полученной KCC.
23.    }
24. }
```

Алгоритм Тарьяна поиска компонент сильной связности. Пример

Ориентированный граф G



Стек

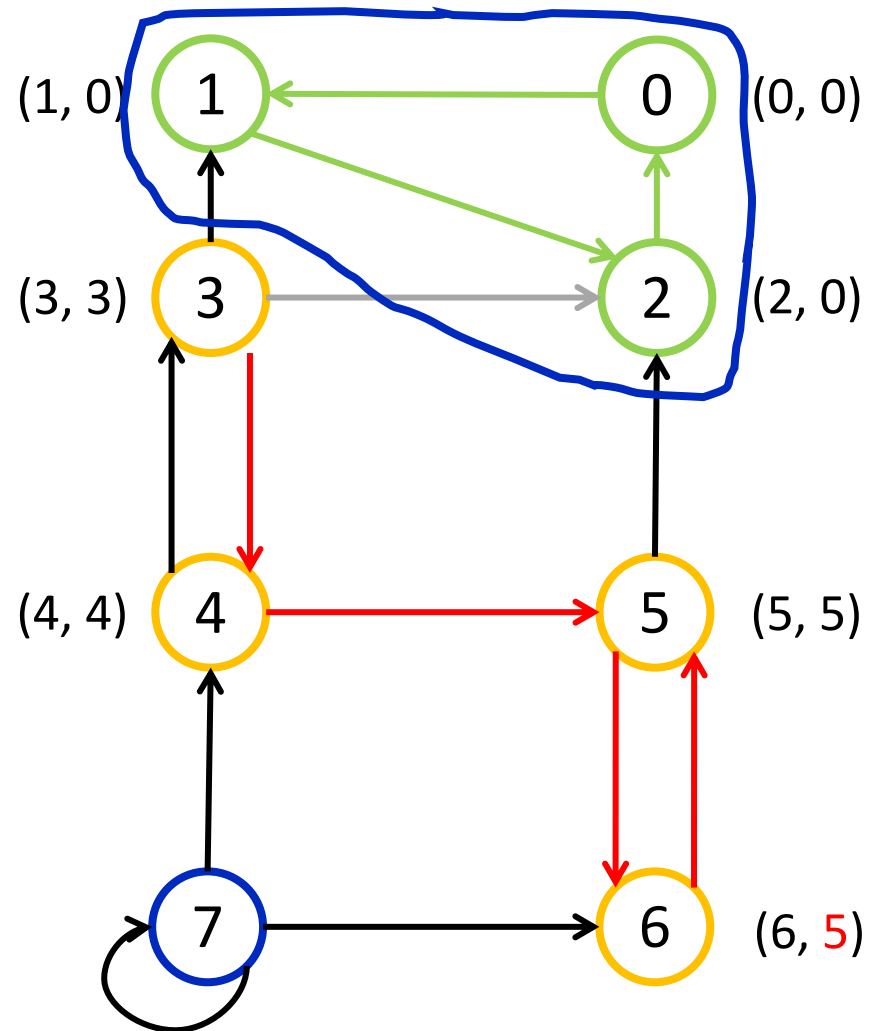


Функция обхода графа в глубину DFS

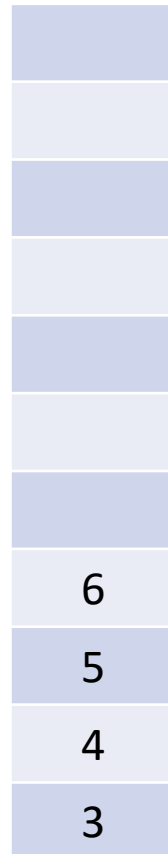
```
1. void dfs(int v) {
2.     used[v] = true;
3.     trn[v] = (tin[v] = timer++);
4.     component.insert(v);
5.     for (auto &to : gr[v]) {
6.         if (!used[to]) {
7.             dfs(to);
8.             trn[v] = min(trn[v], trn[to]);
9.         } else if (used_in_cmp.find(to) != used_in_cmp.end()) {
10.            trn[v] = min(trn[v], tin[to]);
11.        }
12.    }
13.    if (tin[v] == trn[v]) {
14.        // Создание новой KCC.
15.        int curr;
16.        do {
17.            curr = component.top();
18.            // Добавление «curr» в новую KCC.
19.            component.pop();
20.            used_in_cmp.erase(curr);
21.        } while (trn[v] == trn[component.top()]);
22.        // Вывод полученной KCC.
23.    }
24. }
```


Алгоритм Тарьяна поиска компонент сильной связности. Пример

Ориентированный граф G



Стек

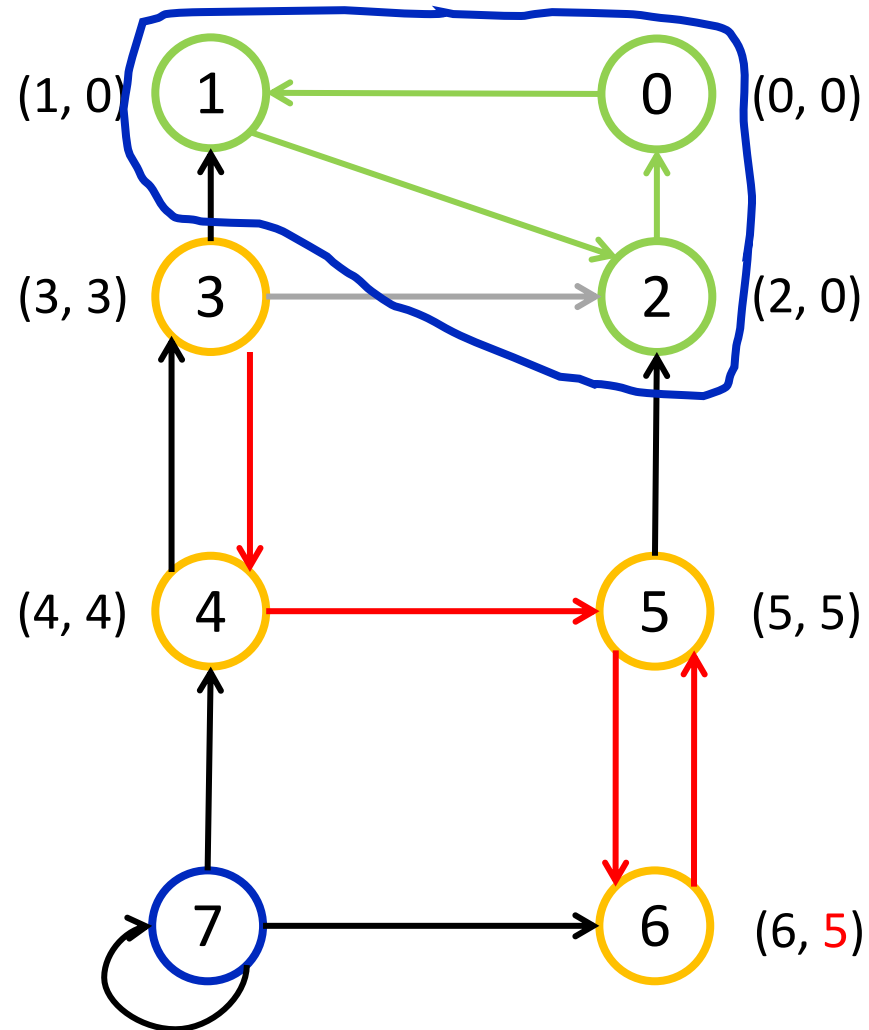


Функция обхода графа в глубину DFS

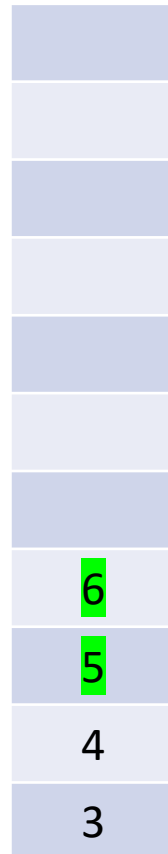
```
1. void dfs(int v) {
2.     used[v] = true;
3.     trn[v] = (tin[v] = timer++);
4.     component.insert(v);
5.     for (auto &to : gr[v]) {
6.         if (!used[to]) {
7.             dfs(to);
8.             trn[v] = min(trn[v], trn[to]);
9.         } else if (used_in_cmp.find(to) != used_in_cmp.end()) {
10.            trn[v] = min(trn[v], tin[to]);
11.        }
12.    }
13.    if (tin[v] == trn[v]) {
14.        // Создание новой KCC.
15.        int curr;
16.        do {
17.            curr = component.top();
18.            // Добавление «curr» в новую KCC.
19.            component.pop();
20.            used_in_cmp.erase(curr);
21.        } while (trn[v] == trn[component.top()]);
22.        // Вывод полученной KCC.
23.    }
24. }
```

Алгоритм Тарьяна поиска компонент сильной связности. Пример

Ориентированный граф G



Стек

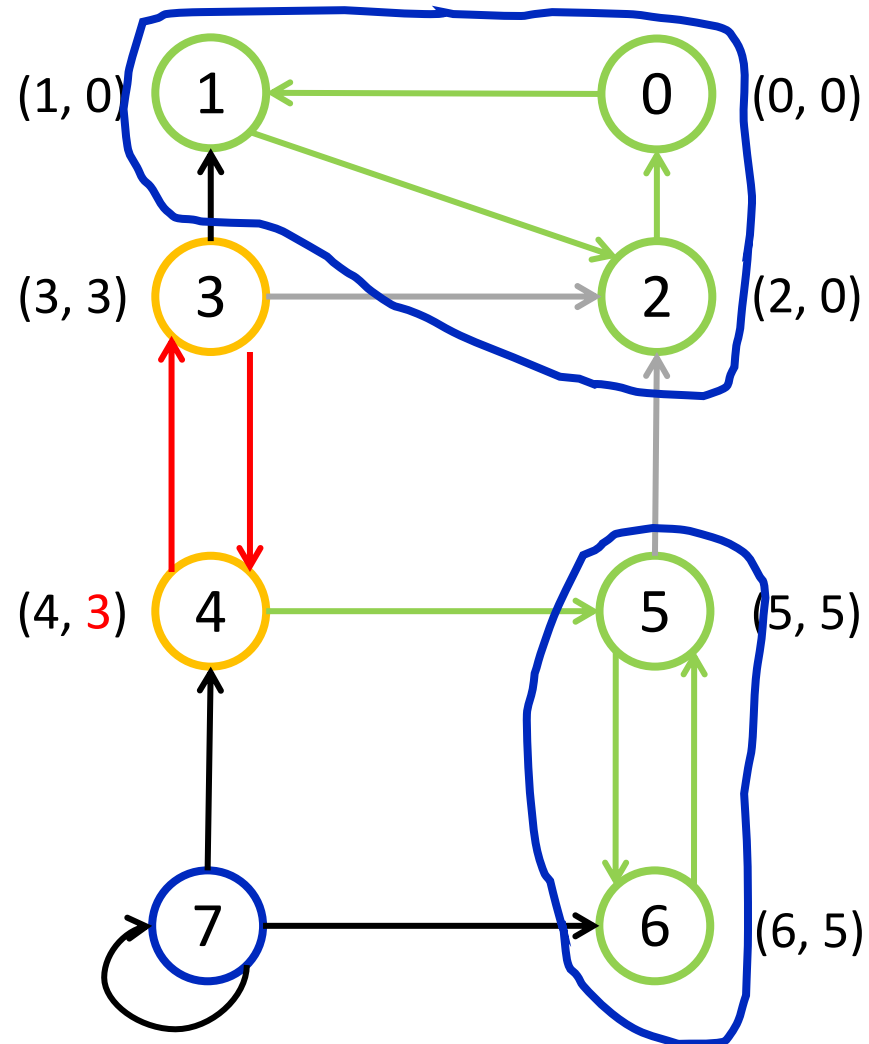


Функция обхода графа в глубину DFS

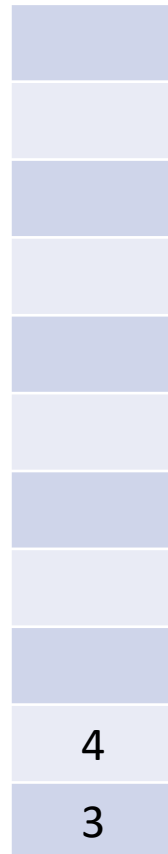
```
1. void dfs(int v) {
2.     used[v] = true;
3.     trn[v] = (tin[v] = timer++);
4.     component.insert(v);
5.     for (auto &to : gr[v]) {
6.         if (!used[to]) {
7.             dfs(to);
8.             trn[v] = min(trn[v], trn[to]);
9.         } else if (used_in_cmp.find(to) != used_in_cmp.end()) {
10.            trn[v] = min(trn[v], tin[to]);
11.        }
12.    }
13.    if (tin[v] == trn[v]) {
14.        // Создание новой KCC.
15.        int curr;
16.        do {
17.            curr = component.top();
18.            // Добавление «curr» в новую KCC.
19.            component.pop();
20.            used_in_cmp.erase(curr);
21.        } while (trn[v] == trn[component.top()]);
22.        // Вывод полученной KCC.
23.    }
24. }
```

Алгоритм Тарьяна поиска компонент сильной связности. Пример

Ориентированный граф G



Стек

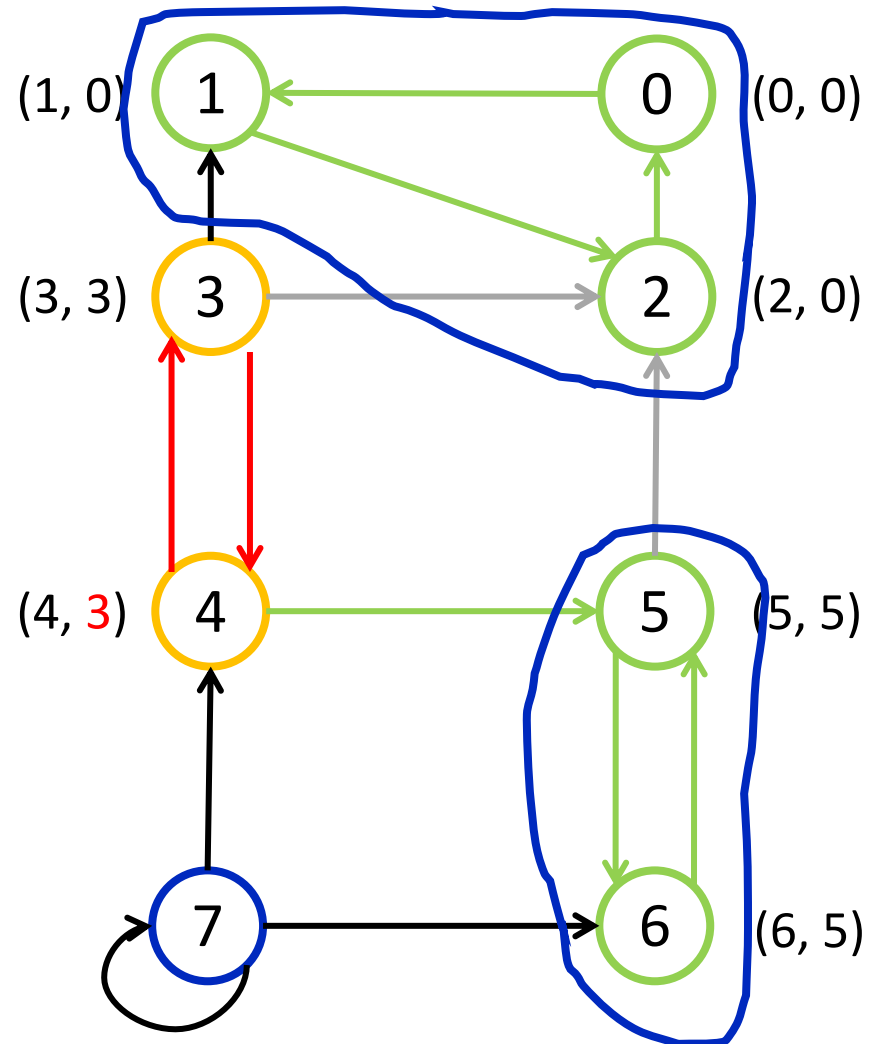


Функция обхода графа в глубину DFS

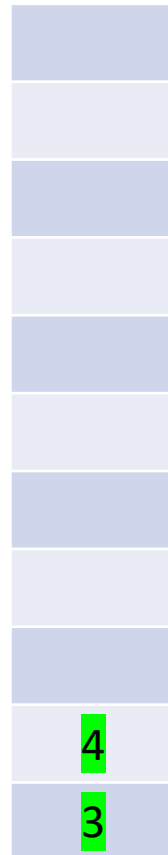
```
1. void dfs(int v) {
2.     used[v] = true;
3.     trn[v] = (tin[v] = timer++);
4.     component.insert(v);
5.     for (auto &to : gr[v]) {
6.         if (!used[to]) {
7.             dfs(to);
8.             trn[v] = min(trn[v], trn[to]);
9.         } else if (used_in_cmp.find(to) != used_in_cmp.end()) {
10.            trn[v] = min(trn[v], tin[to]);
11.        }
12.    }
13.    if (tin[v] == trn[v]) {
14.        // Создание новой KCC.
15.        int curr;
16.        do {
17.            curr = component.top();
18.            // Добавление «curr» в новую KCC.
19.            component.pop();
20.            used_in_cmp.erase(curr);
21.        } while (trn[v] == trn[component.top()]);
22.        // Вывод полученной KCC.
23.    }
24. }
```

Алгоритм Тарьяна поиска компонент сильной связности. Пример

Ориентированный граф G



Стек

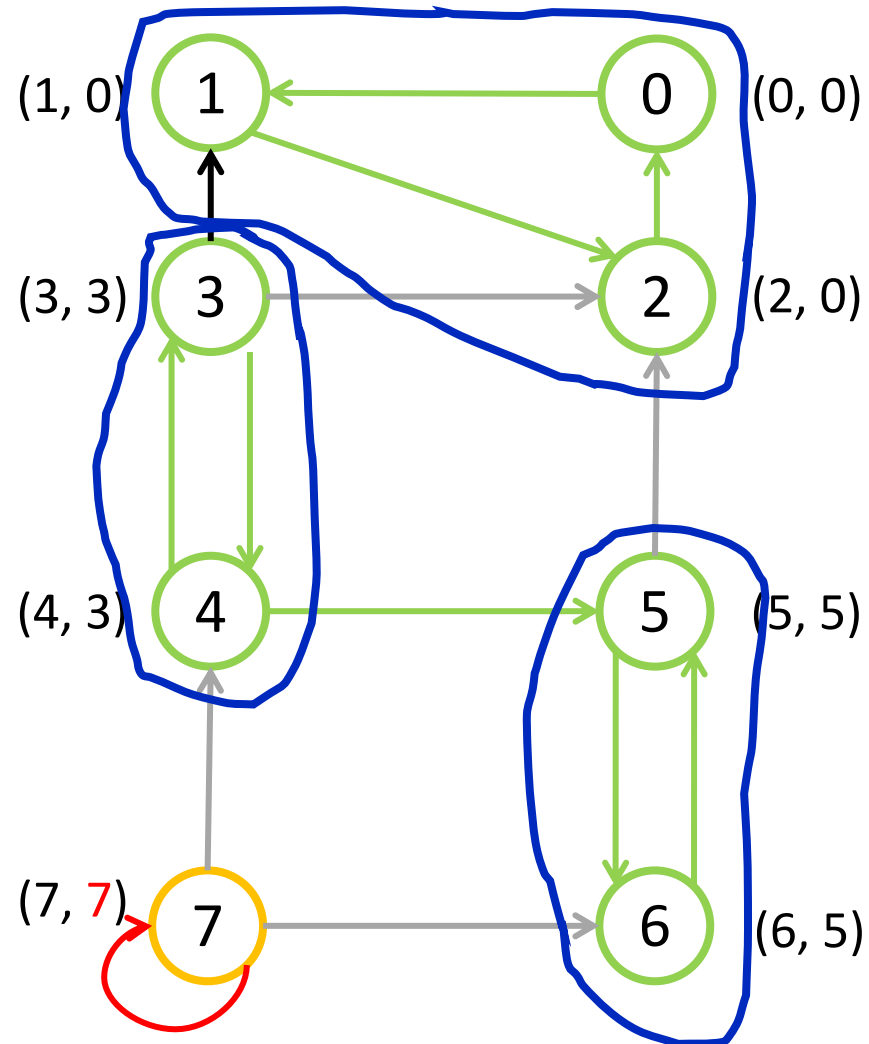


Функция обхода графа в глубину DFS

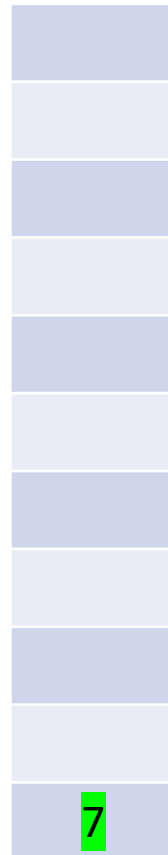
```
1. void dfs(int v) {
2.     used[v] = true;
3.     trn[v] = (tin[v] = timer++);
4.     component.insert(v);
5.     for (auto &to : gr[v]) {
6.         if (!used[to]) {
7.             dfs(to);
8.             trn[v] = min(trn[v], trn[to]);
9.         } else if (used_in_cmp.find(to) != used_in_cmp.end()) {
10.            trn[v] = min(trn[v], tin[to]);
11.        }
12.    }
13.    if (tin[v] == trn[v]) {
14.        // Создание новой KCC.
15.        int curr;
16.        do {
17.            curr = component.top();
18.            // Добавление «curr» в новую KCC.
19.            component.pop();
20.            used_in_cmp.erase(curr);
21.        } while (trn[v] == trn[component.top()]);
22.        // Вывод полученной KCC.
23.    }
24. }
```

Алгоритм Тарьяна поиска компонент сильной связности. Пример

Ориентированный граф G



Стек

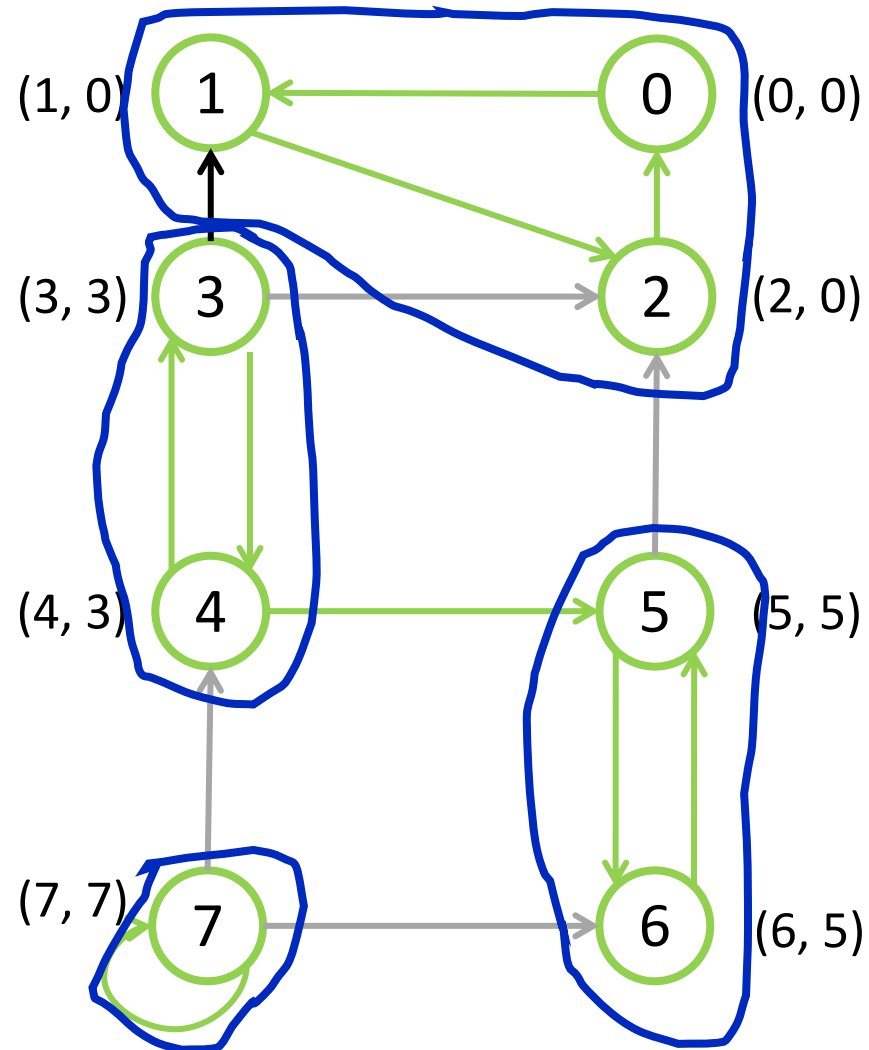


Функция обхода графа в глубину DFS

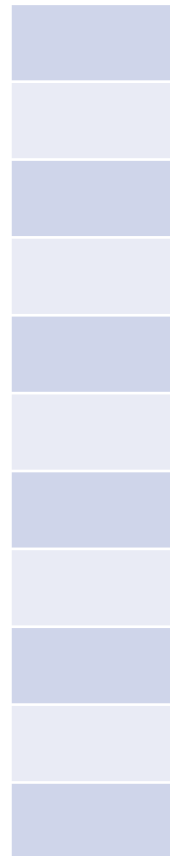
```
1. void dfs(int v) {
2.     used[v] = true;
3.     trn[v] = (tin[v] = timer++);
4.     component.insert(v);
5.     for (auto &to : gr[v]) {
6.         if (!used[to]) {
7.             dfs(to);
8.             trn[v] = min(trn[v], trn[to]);
9.         } else if (used_in_cmp.find(to) != used_in_cmp.end()) {
10.            trn[v] = min(trn[v], tin[to]);
11.        }
12.    }
13.    if (tin[v] == trn[v]) {
14.        // Создание новой KCC.
15.        int curr;
16.        do {
17.            curr = component.top();
18.            // Добавление «curr» в новую KCC.
19.            component.pop();
20.            used_in_cmp.erase(curr);
21.        } while (trn[v] == trn[component.top()]);
22.        // Вывод полученной KCC.
23.    }
24. }
```

Алгоритм Тарьяна поиска компонент сильной связности. Пример

Ориентированный граф G



Стек



Функция обхода графа в глубину DFS

```
1. void dfs(int v) {
2.     used[v] = true;
3.     trn[v] = (tin[v] = timer++);
4.     component.insert(v);
5.     for (auto &to : gr[v]) {
6.         if (!used[to]) {
7.             dfs(to);
8.             trn[v] = min(trn[v], trn[to]);
9.         } else if (used_in_cmp.find(to) != used_in_cmp.end()) {
10.            trn[v] = min(trn[v], tin[to]);
11.        }
12.    }
13.    if (tin[v] == trn[v]) {
14.        // Создание новой KCC.
15.        int curr;
16.        do {
17.            curr = component.top();
18.            // Добавление «curr» в новую KCC.
19.            component.pop();
20.            used_in_cmp.erase(curr);
21.        } while (trn[v] == trn[component.top()]);
22.        // Вывод полученной KCC.
23.    }
24. }
```

Достоинства и недостатки

Алгоритм	Лучший случай	Средний случай	Худший случай	Расходы доп. памяти
Алгоритм поиска мостов	$O(V + E)$	$O(V + E)$	$O(V ^2)$ (полный граф)	$O(N)$ ✓
Алгоритм поиска точек сочленения	$O(V + E)$	$O(V + E)$	$O(V ^2)$ (полный граф)	$O(N)$ ✓
Алгоритм Косарайю	$O(V + E)$	$O(V + E)$	$O(V ^2)$ (полный граф)	$O(N)$ ✓ ²
Алгоритм Тарьяна	$O(V + E)$	$O(V + E)$	$O(V ^2)$ (полный граф)	$O(N)$ ✓

$$E = V^2$$

$$V + E$$